

Universidad de Valladolid

Escuela de Ingeniería Informática

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería de Software

Aplicación móvil para la información y geolocalización de cofradías y procesiones en semana santa.

Autor:
Elena Martínez Vázquez

Tutores:
Juan Alberto Muñoz Cristóbal
Pablo Federico Sanchez Mayoral

Dedicatoria

Agradecimientos

Resumen

Abstract

Índice general

Acrónimos IX

1. Introducción **1**
1.1. Contexto . . . 1
1.2. Motivación . . . 2
1.3. Estado de la cuestión . . . 3
1.3.1. Aplicaciones actuales de Semana Santa . . . 3
1.3.2. Aplicaciones de seguimiento en tiempo real . . . 5
1.4. Objetivos . . . 6
1.5. Estructura de la memoria . . . 7

2. Planificación **9**
2.1. Metodología . . . 9
2.2. Planificación, Fases e iteraciones . . . 9
2.2.1. Planificación inicial . . . 9
2.2.2. Fase de análisis . . . 10
2.2.3. Fase de iteraciones . . . 11
2.2.4. Fase final: documentación y entrega . . . 12
2.3. Análisis de riesgos . . . 12
2.3.1. Planificación de riesgo . . . 13
2.4. Estimación de costes . . . 13
2.4.1. Costes de personal . . . 15
2.4.2. Costes de equipo y amortización . . . 16
2.4.3. Costes de licencias de software . . . 16
2.4.4. Otros costes indirectos . . . 17
2.4.5. Coste total . . . 18
2.5. Seguimiento del proyecto . . . 18

3. Análisis **19**
3.1. Actores del sistema . . . 19
3.1.1. Ciudadano . . . 19
3.1.2. Cofrade . . . 19
3.1.3. Junta de Cofradías . . . 20
3.1.4. Administrador . . . 20

3.2. Requisitos . . . 20
3.2.1. Requisitos funcionales . . . 20
3.2.2. Requisitos de información . . . 24
3.2.3. Requisitos no funcionales . . . 28
3.3. Casos de uso . . . 29
3.4. Modelo de dominio . . . 31
3.4.1. Modelo de interacción . . . 38

4. Descripción de las iteraciones **43**
4.1. 1ª Iteración . . . 43
4.1.1. Analisís . . . 43
4.1.2. Diseño . . . 43
4.1.3. Implementacion . . . 62
4.1.4. Pruebas . . . 63
4.2. 2ª Iteración . . . 64
4.2.1. Analisís . . . 64
4.2.2. Diseño . . . 64
4.2.3. Implementacion . . . 65
4.2.4. Pruebas . . . 66
4.3. 3ª Iteración . . . 66
4.3.1. Analisís . . . 66
4.3.2. Diseño . . . 66
4.3.3. Implementacion . . . 67
4.3.4. Pruebas . . . 68
4.4. 4ª Iteración . . . 68
4.4.1. Analisís . . . 68
4.4.2. Diseño . . . 68
4.4.3. Implementacion . . . 79
4.4.4. Pruebas . . . 79
4.5. 5ª Iteración . . . 79
4.5.1. Analisís . . . 79
4.5.2. Diseño . . . 79
4.5.3. Implementacion . . . 88
4.5.4. Pruebas . . . 88

5. Estado final de la aplicación **89**
5.0.1. Historias de usuario . . . 89

6. Pruebas **91**

7. Conclusiones **93**
7.1. Trabajo futuro . . . 93
7.2. Puntos débiles . . . 94

Bibliografía **99**

Anexos **105**

A. Casos de uso adicionales **105**
A.1. Ciudadano . . . 105
A.2. Cofrade . . . 105
A.3. Miembro de la Junta de Cofradías . . . 105
A.4. Administrador . . . 105

B. Esquema de la base de datos **135**
B.1. Ciudad (`ciudades`) . . . 135
B.2. Junta de Cofradías (`juntas_cofradias`) . . . 135
B.3. Cofradía / Hermandad (`cofradias`) . . . 136
B.4. Paso (`pasos`) . . . 136
B.4.1. Paso-Evento (`pasos_eventos`) — tabla intermedia N:M . . . 137
B.5. Código de Acceso (`codigos_acceso`) . . . 137
B.6. Ubicación (`ubicaciones`) . . . 137
B.7. Evento (`eventos`) . . . 138
B.7.1. Evento-Cofradía (`eventos_cofradias`) — tabla intermedia N:M . . . 138
B.8. Procesión (`procesiones`) . . . 138
B.8.1. Posiciones actuales (`posiciones_actuales`) — histórico de pings anónimos . . . 139
B.9. Recorrido (`recorridos`) . . . 139
B.9.1. Punto de Ruta (`puntos_ruta`) . . . 140
B.9.2. Punto de Interés (`puntos_de_interes`) . . . 140
B.9.3. Recorrido-Punto de Ruta (`recorrido_puntos_ruta`) — tabla intermedia N:M . . . 141
B.10. Usuario (`usuarios`) . . . 141
B.10.1. Miembro de Junta de Cofradías (`miembros_junta_cofradia`) . . . 142
B.10.2. Administrador (`administradores`) . . . 142
B.11. Notificación (`notificaciones`) . . . 143
B.12. Dispositivo Push (`dispositivos_push`) . . . 143
B.13. Relaciones entre tablas . . . 144

C. Manual de despliegue **145**
C.1. Requisitos previos . . . 145
C.2. Despliegue del backend en el servidor . . . 145
C.2.1. Actualización a una nueva versión . . . 147
C.3. Documentación de la API . . . 147
C.4. Configuración del cliente React Native . . . 147
C.5. Generación del ejecutable de la aplicación móvil . . . 148

D. Manual de uso **149**

E. Contenido del TFG **151**

Índice de figuras

1.1. Interfaz de la aplicación El Penitente [4] . . . 3
1.2. Interfaz de la aplicación Valladolid Cofrade [5] . . . 4
1.3. Ejemplo de geolocalización en tiempo real en Google Maps . . . 5
1.4. Seguimiento en tiempo real de un vehículo en Uber [9] . . . 6

2.1. Diagrama de Gantt general completo del proyecto . . . 10

3.1. Diagrama de caso de uso Ciudadano . . . 29
3.2. Diagrama de caso de uso Cofrade . . . 30
3.3. Diagrama de caso de uso Miembro de la Junta de Cofradías . . . 30
3.4. Diagrama de caso de uso Administrador . . . 31
3.5. Diagrama de dominio del proyecto . . . 37
3.6. Diagrama de secuencia del CU “Visualizar procesión en tiempo real” . . . 39
3.7. Diagrama de secuencia del CU “Compartir ubicación durante procesión” . . . 40
3.8. Diagrama de secuencia del CU “Iniciar sesión con código de acceso” . . . 41
3.9. Diagrama de secuencia del CU “Gestionar una ciudad” (alta) . . . 42

4.1. Diagrama de paquetes de la aplicación cliente . . . 47
4.2. Diagrama de paquetes del backend propio . . . 48
4.3. Diagrama de despliegue de la aplicación . . . 49
4.4. Diagrama entidad-relación de la base de datos . . . 52
4.5. Mockup de la pantalla de selección de ciudad . . . 54
4.6. Mockup de la pantalla de inicio: notificación de retraso, procesión en curso, estadísticas y agenda del día . . . 55
4.7. Mockup de la pantalla de calendario: vista mensual y agenda del día seleccionado . . . 56
4.8. Mockup del menú de la pantalla de inicio y de los listados de procesiones, pasos, eventos y cofradías . . . 57
4.9. Mockup del detalle de un paso y de una cofradía . . . 58
4.10. Mockup del detalle de un evento . . . 59
4.11. Mockup del detalle de una procesión (programada y en curso) y de su información ampliada 60
4.12. Mockup de la pantalla de búsqueda con filtros por día y estado . . . 61
4.13. Mockup del mapa en vivo . . . 65
4.14. Mockup de la pantalla de perfil, en modo ciudadano y en modo cofrade . . . 67
4.15. Mockup de la bienvenida y del inicio de sesión, para Junta de Cofradías y Administrador . 69
4.16. Mockup del perfil de la Junta de Cofradías y de la edición de sus datos . . . 70

4.17. Mockup del listado de cofradías y de su creación 71
4.18. Mockup de la edición de una cofradía 72
4.19. Mockup de las procesiones, eventos y pasos de una cofradía, accesibles desde su edición . 73
4.20. Mockup de la creación de una procesión 74
4.21. Mockup de la edición de una procesión 75
4.22. Mockup del listado de procesiones y del envío de una notificación 76
4.23. Mockup del listado de eventos y de su creación 77
4.24. Mockup de la edición de un evento 77
4.25. Mockup del listado de pasos y de su creación 78
4.26. Mockup de la edición de un paso 78
4.27. Mockup del perfil de administrador y de la edición de sus datos 80
4.28. Mockup del listado de ciudades y de su creación 81
4.29. Mockup de la edición de una ciudad 82
4.30. Mockup del listado de Juntas de Cofradías y de su creación 83
4.31. Mockup de la edición de una Junta de Cofradías 84
4.32. Mockup de la creación de un miembro de la Junta de Cofradías 85
4.33. Mockup de la edición de un miembro de la Junta de Cofradías 86
4.34. Mockup del listado de miembros, sin filtrar y filtrado por Junta de Cofradías 87
4.35. Correo de invitación recibido por un nuevo miembro de la Junta de Cofradías 87

Índice de tablas

2.1. Riesgos identificados en el proyecto . . . 13
2.2. Riesgo 1: Estimaciones de tiempo erróneas . . . 13
2.3. Riesgo 2: Problemas con la integración de Google Maps . . . 13
2.4. Riesgo 3: Errores durante el desarrollo . . . 14
2.5. Riesgo 4: Retrasos o errores por falta de experiencia con tecnologías móviles . . . 14
2.6. Riesgo 5: Problemas con la geolocalización en tiempo real . . . 14
2.7. Riesgo 6: Cambios en los requisitos . . . 14
2.8. Riesgo 7: Especificaciones genéricas . . . 14
2.9. Riesgo 8: Problemas con el entorno de desarrollo . . . 15
2.10. Riesgo 9: Indisposición del desarrollador . . . 15
2.11. Riesgo 10: Falta de experiencia con las tecnologías del backend propio . . . 15
2.12. Reparto de horas y coste de personal por perfil . . . 16
2.13. Amortización de los recursos de equipo, prorrateada al uso real del proyecto . . . 17
2.14. Coste de licencias de software . . . 17
2.15. Costes totales del proyecto . . . 18

3.1. Resumen de los actores del sistema . . . 21
3.2. Caso de uso: Visualizar procesión en tiempo real . . . 32
3.3. Caso de uso: Crear notificación . . . 33
3.4. Caso de uso: Iniciar sesión con código de acceso . . . 34
3.5. Caso de uso: Compartir ubicación durante procesión . . . 35
3.6. Caso de uso: Gestionar una ciudad . . . 36

5.1. Historia de usuario HU01 . . . 89

A.1. Caso de uso: Seleccionar ciudad . . . 106
A.2. Caso de uso: Consultar información de una cofradía, procesión, evento o paso . . . 107
A.3. Caso de uso: Acceder a listados de procesiones, pasos, eventos o cofradías . . . 108
A.4. Caso de uso: Buscar cofradías, eventos, procesiones o pasos . . . 109
A.5. Caso de uso: Filtrar elementos . . . 110
A.6. Caso de uso: Consultar calendario de Semana Santa . . . 111
A.7. Caso de uso: Consultar procesiones y eventos de un día . . . 112
A.8. Caso de uso: Añadir favoritos . . . 113
A.9. Caso de uso: Eliminar favorito . . . 113
A.10. Caso de uso: Consultar favoritos . . . 114

A.11.Caso de uso: Consultar estado de las notificaciones . . . 115
A.12.Caso de uso: Generar ruta hacia procesión . . . 116
A.13.Caso de uso: Elegir procesión en curso . . . 117
A.14.Caso de uso: Dejar de compartir ubicación . . . 117
A.15.Caso de uso: Iniciar sesión . . . 118
A.16.Caso de uso: Cambiar contraseña provisional . . . 119
A.17.Caso de uso: Solicitar reactivación de cuenta . . . 120
A.18.Caso de uso: Gestionar su propio perfil . . . 121
A.19.Caso de uso: Editar información de la ciudad . . . 122
A.20.Caso de uso: Gestionar cofradía . . . 123
A.21.Caso de uso: Gestionar pasos . . . 124
A.22.Caso de uso: Gestionar eventos . . . 125
A.23.Caso de uso: Gestionar procesión . . . 126
A.24.Caso de uso: Asignar pasos . . . 127
A.25.Caso de uso: Importar recorrido (GPX) . . . 128
A.26.Caso de uso: Marcar puntos de interés . . . 129
A.27.Caso de uso: Gestionar códigos de acceso . . . 130
A.28.Caso de uso: Gestionar una Junta de Cofradías . . . 131
A.29.Caso de uso: Gestionar a los miembros de las Juntas de Cofradías . . . 132
A.30.Caso de uso: Reenviar invitación . . . 133
A.31.Caso de uso: Gestionar solicitudes de reactivación . . . 134

B.1. Campos de `ciudades` . . . 135
B.2. Campos de `juntas_cofradias` . . . 136
B.3. Campos de `cofradias` . . . 136
B.4. Campos de `pasos` . . . 137
B.5. Campos de `pasos_eventos` . . . 137
B.6. Campos de `codigos_acceso` . . . 137
B.7. Campos de `ubicaciones` . . . 138
B.8. Campos de `eventos` . . . 138
B.9. Campos de `eventos_cofradias` . . . 138
B.10.Campos de `procesiones` . . . 139
B.11.Campos de `posiciones_actuales` . . . 139
B.12.Campos de `recorridos` . . . 140
B.13.Campos de `puntos_ruta` . . . 140
B.14.Campos de `puntos_de_interes` . . . 140
B.15.Campos de `recorrido_puntos_ruta` . . . 141
B.16.Campos de `usuarios` . . . 142
B.17.Campos de `miembros_junta_cofradia` . . . 142
B.18.Campos de `administradores` . . . 142
B.19.Campos de `notificaciones` . . . 143
B.20.Campos de `dispositivos_push` . . . 144
B.21.Relaciones entre tablas de PostgreSQL . . . 144

Acrónimos

- **API**: Application Programming Interface.
- **PWA**: Progressive Web App (ya lo explicas en el texto, pero debería estar en el apéndice)
- **GPS**: Global Positioning System
- **UML**: Unified Modeling Language
- **RF**: Requisito Funcional
- **RI**: Requisito de Información
- **RNF**: Requisito No Funcional
- **HU**: Historia de Usuario
- **TFG**: Trabajo de Fin de Grado

Capítulo 1

Introducción

1.1. Contexto

La Semana Santa es una de las fiestas culturales y religiosas más relevantes del calendario español. Detrás de esta tradición hay siglos de historia, ya que forma parte del patrimonio histórico y artístico del país. En ciudades como Sevilla, Málaga o Valladolid, entre otras, está declarada Fiesta de Interés Turístico Internacional. Esta celebración tiene un gran impacto tanto cultural como económico, debido a los miles de visitantes nacionales e internacionales que atrae cada año.

La estructura organizativa de la Semana Santa está formada por las cofradías o hermandades, las cuales son asociaciones religiosas encargadas de organizar las procesiones y los eventos de estos días. La procesión es el acto central: cada una recorre un itinerario propio por las calles de la ciudad, llevando consigo pasos[1] de gran valor artístico que representan escenas de la Biblia. Junto a las procesiones, las cofradías organizan también otros eventos de carácter más puntual –pregones, traslados u ofrendas florales, entre otros– que completan el programa de la Semana Santa pero, a diferencia de una procesión, no siguen un recorrido propio por la ciudad. Esta organización de horarios, recorridos y actos requiere una gran coordinación por parte de las distintas cofradías y de la Junta de Cofradías de cada ciudad. Durante estos días, la ciudad experimenta imprevistos, como cortes de tráfico, cambios en la movilidad urbana y una gran cantidad de personas en puntos estratégicos del recorrido.

Desde el punto de vista social, la Semana Santa puede implicar dificultades de movilidad debido al desconocimiento de la ubicación de las distintas procesiones, además de los cambios de última hora en cuanto a los horarios de comienzo de cada procesión debido al mal tiempo o incluso a cancelaciones. Como esto se sale de la planificación inicial, los cofrades encuentran dificultades a la hora de comunicar estos cambios y decisiones de última hora a los espectadores y fieles. A día de hoy, son las distintas Juntas de Cofradías las que deben gestionar toda esta información de forma manual o con herramientas anticuadas.

En el contexto tecnológico actual, la mayoría de los turistas y ciudadanos hacen uso de sus teléfonos

[1]Paso: plataforma o andamio, habitualmente de grandes dimensiones y ricamente ornamentado, sobre el que se transportan a hombros o mediante ruedas las imágenes religiosas durante la procesión.

móviles como principal medio para consultar información. Aplicaciones como Google Maps [1] permiten conocer en tiempo real la ubicación (geolocalización [2]) de calles, monumentos, bares, etc. Esta funcionalidad podría aplicarse a la Semana Santa, de forma que facilitaría a los visitantes y ciudadanos información constante y rápida sobre dónde se encuentra una procesión o si ha habido algún cambio o imprevisto en la misma.

Finalmente, debido a la cantidad de dinero que genera el turismo durante la Semana Santa, podemos concluir que es de gran interés para las ciudades que la experiencia del turista y del ciudadano sea positiva y que existan herramientas para que esto se haga realidad, de forma que una aplicación que mejore dicha experiencia repercutirá positivamente.

1.2. Motivación

La principal motivación de este proyecto surge de la dificultad que tienen tanto los ciudadanos como los visitantes a la hora de buscar y obtener información clara y actualizada durante los días de la Semana Santa. Aunque a día de hoy existan programas oficiales, carteles informativos o cuentas oficiales en redes sociales, estos medios no siempre consiguen dar a conocer en tiempo real la situación de las procesiones, ya sea por retrasos o cambios de última hora debidos a factores meteorológicos o a problemas dentro de la propia cofradía.

Debido a esto, los asistentes, en muchas ocasiones, deben desplazarse a los distintos puntos del recorrido sin tener la certeza de que la procesión no haya pasado ya, cuánto tiempo de espera se estima hasta que pase por ese punto o incluso si llegará a pasar debido a cancelaciones. Esto genera desorganización y aglomeraciones innecesarias, además de una experiencia menos satisfactoria para los observadores.

Gracias a estas observaciones, se ve clara la necesidad de contar con un medio común capaz de organizar y difundir la información necesaria para que la experiencia sea lo más clara y accesible posible para cualquier persona.

Como respuesta a estas limitaciones, se plantea la idea de una aplicación móvil que permita la centralización de la información sobre el estado de las procesiones de forma actualizada, además de incluir otras funciones como la visualización de los recorridos y la geolocalización en tiempo real de las mismas. De esta forma, se mejora no solo la experiencia de los usuarios, sino que también se facilita la gestión de la información durante la Semana Santa.

Además, el desarrollo de esta aplicación permite no solo aplicar los conocimientos adquiridos durante el grado sobre análisis de requisitos, diseño de software, desarrollo de aplicaciones móviles y gestión de proyectos, sino también ampliarlos mediante su aplicación en un proyecto real, enfrentándose a problemas prácticos como la integración de distintas tecnologías, el manejo de la geolocalización en tiempo real o la gestión de múltiples tipos de usuarios dentro de un mismo sistema.

1.3. Estado de la cuestión

En la actualidad existen diversas soluciones tecnológicas relacionadas con la Semana Santa, aunque la mayoría de ellas se encuentran limitadas a ámbitos geográficos concretos o se centran únicamente en la difusión de información estática. A continuación, se analizan las principales herramientas disponibles, así como sus características más relevantes.

1.3.1. Aplicaciones actuales de Semana Santa

A día de hoy, existen diversas aplicaciones móviles orientadas a la Semana Santa para distintas ciudades españolas. Entre ellas encontramos *El Penitente* [3], centrada únicamente en las ciudades de Sevilla y Málaga. Esta aplicación ofrece información sobre horarios, itinerarios y una representación del recorrido de las procesiones sobre el mapa, mostrando el trayecto seguido y, en algunos casos, su posición aproximada.

La aplicación presenta una interfaz (Figura 1.1) en la que el usuario puede consultar información sobre las procesiones y sus recorridos, notificaciones, planificaciones para organizar la Semana Santa y horarios de hermandades o cofradías.

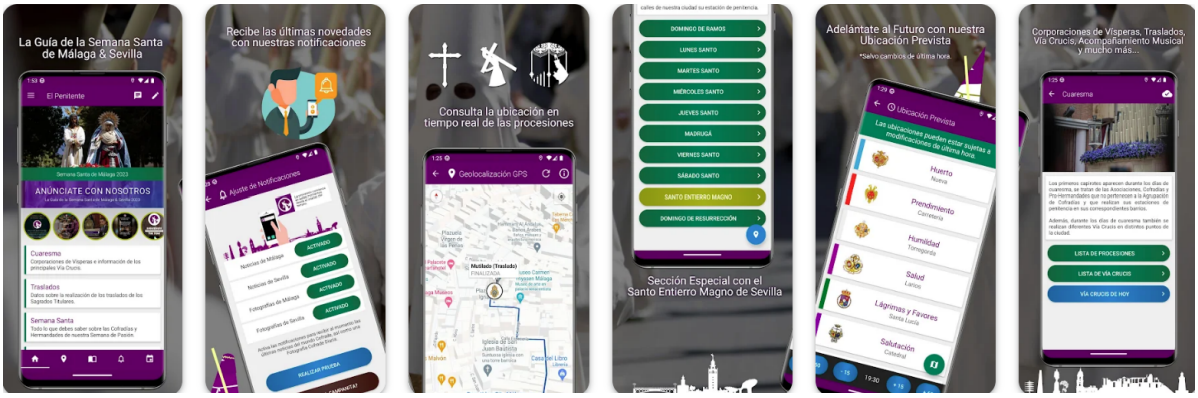


Figura 1.1: Interfaz de la aplicación El Penitente [4]

Su mayor limitación es que está diseñada y pensada para ciudades específicas y no constituye una solución unificada a nivel nacional.

Otra aplicación a analizar es *Valladolid Cofrade* [5], una herramienta reciente desarrollada como aplicación web progresiva (PWA) [6], es decir, una aplicación accesible desde el navegador que puede instalarse en el dispositivo y funcionar de forma similar a una aplicación nativa. Esta permite a los usuarios acceder a información sobre la Semana Santa sin realizar la instalación tradicional desde tiendas oficiales. La aplicación ofrece información sobre programas, recorridos y un sistema de notificaciones en tiempo real ante cambios o incidencias (Figura 1.2).

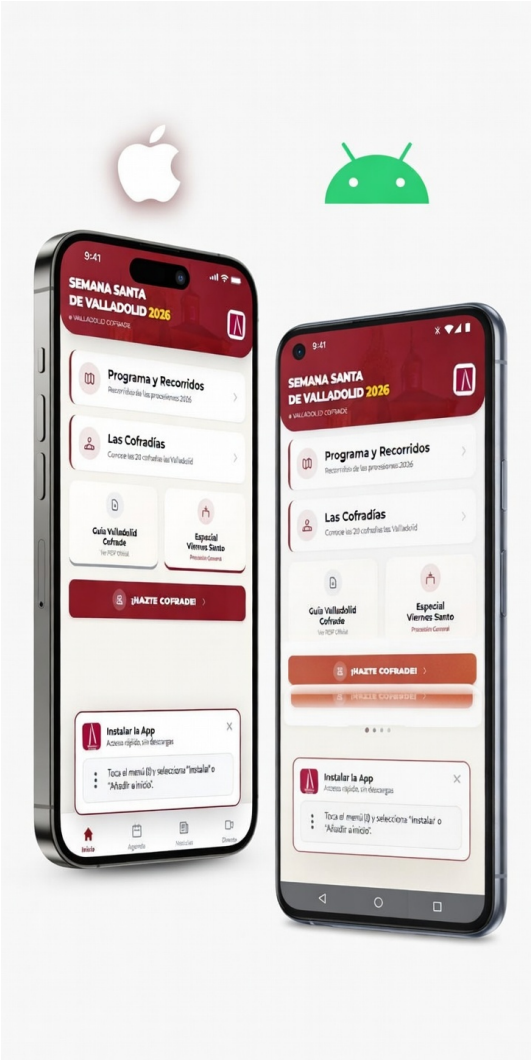


Figura 1.2: Interfaz de la aplicación Valladolid Cofrade [5]

En otras ciudades se cuenta con aplicaciones locales específicas o sitios web adaptados, como sSantaZgz en Zaragoza [7] o Junta Pro Semana Santa de Zamora [8]. Estas aplicaciones muestran el programa oficial, los distintos recorridos e incluso, en algunos casos, como en la de Zamora, incluyen herramientas para conocer la previsión meteorológica.

Más allá de estas limitaciones técnicas, también hay que destacar la diversidad organizativa de la Semana Santa en España, ya que esta depende de la ciudad, de la estructura que se siga en ella y de la terminología utilizada. Por ejemplo, en algunas ciudades, como Valladolid, es común emplear los términos *cofradía* o *cofrade*, mientras que en otras, como Sevilla, se utilizan *hermandad* o *nazareno* para referirse a conceptos equivalentes. Desde el punto de vista organizativo, existen diferentes formas de organizar los eventos, recorridos e itinerarios de las procesiones. Incluso hay diferencias en elementos como pasos, imágenes, homilías, rezos o cánticos que forman parte de estas procesiones o eventos.

Estas diferencias hacen que cada aplicación tenga un diseño distinto, pensado específicamente para una ciudad concreta, dificultando la creación de una solución reutilizable. Por tanto, en la actualidad no existe una plataforma que consiga unificar y generalizar estos conceptos y ofrecer una visión global de la

Semana Santa.

Además de las aplicaciones, los usuarios también hacen uso de las publicaciones oficiales de las cofradías en sus páginas web o de programas impresos. Sin embargo, los medios que realmente resultan útiles para la actualización inmediata de la información son las redes sociales oficiales de las cofradías o de las Juntas de Cofradías, que permiten a los espectadores recibir información sobre cambios o comunicados al instante. Aun así, esta forma de comunicación no siempre resulta fácil de encontrar o de organizar debido a la diversidad de cuentas que gestionan dicha información.

1.3.2. Aplicaciones de seguimiento en tiempo real

En cuanto a la geolocalización y el seguimiento en tiempo real, se trata de una tecnología ya existente que se utiliza en diversas aplicaciones como *Google Maps* [1], que permite conocer la ubicación, cómo llegar a distintos destinos o el estado del transporte público (Figura 1.3).

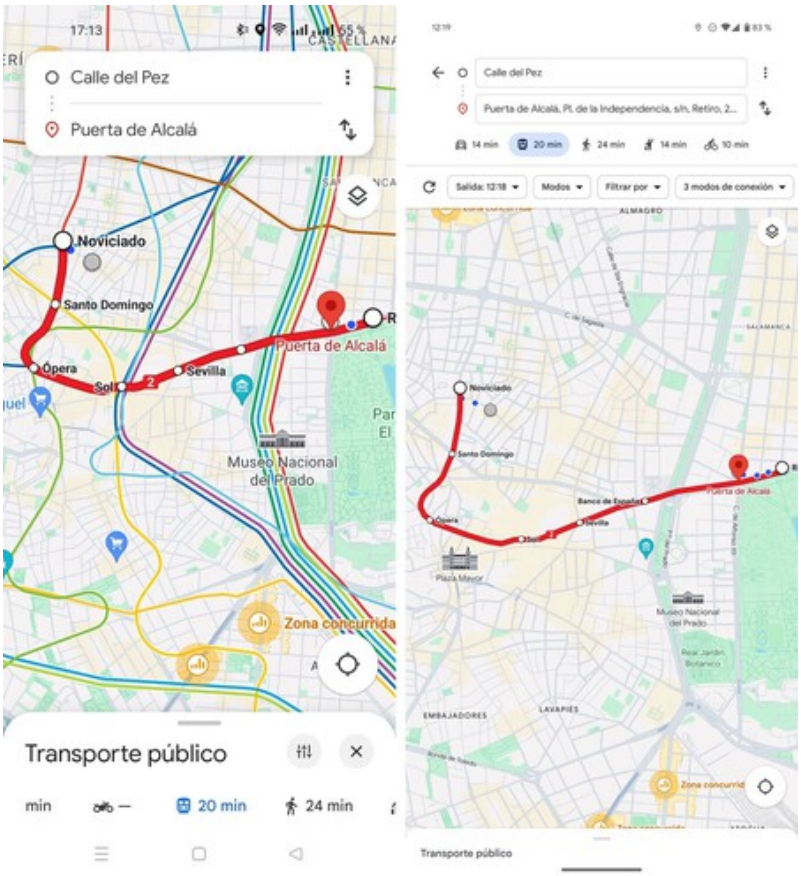


Figura 1.3: Ejemplo de geolocalización en tiempo real en Google Maps

Además, plataformas como *Uber* [9] permiten a los usuarios conocer en todo momento la ubicación exacta del vehículo solicitado (Figura 1.4).

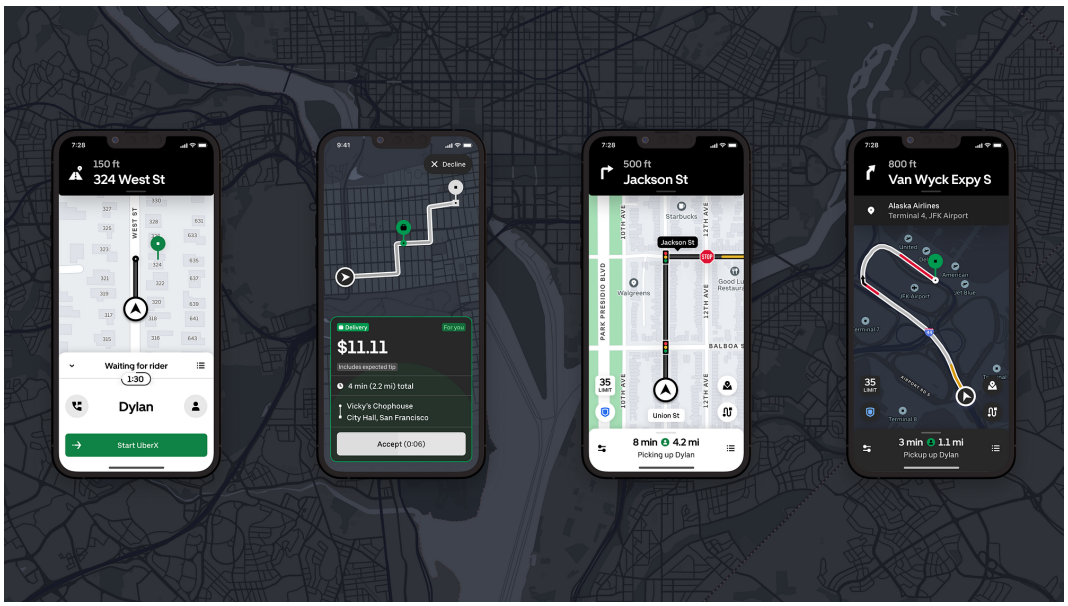


Figura 1.4: Seguimiento en tiempo real de un vehículo en Uber [9]

Además, existen diversos Trabajos de Fin de Grado centrados en el desarrollo de aplicaciones móviles y el uso de tecnologías de geolocalización. Un ejemplo es el proyecto realizado por Mario Llorente Pérez [10], enfocado en una aplicación móvil relacionada con actividades deportivas y el uso de mapas y geolocalización. En trabajos como este puede observarse el importante papel que juega la localización en tiempo real y la necesidad de diseñar interfaces que permitan al usuario interactuar con la aplicación de forma sencilla.

Todas estas aplicaciones y trabajos constituyen ejemplos claros de que el seguimiento en tiempo real es una funcionalidad ampliamente utilizada, a pesar de que en el contexto de la Semana Santa sigue estando limitada y, cuando se ofrece, suele restringirse a entornos locales o presentar un nivel de precisión reducido.

Por tanto, su incorporación en una solución a nivel nacional conseguiría no solo mejorar la experiencia de los usuarios, sino también superar las limitaciones de las soluciones actuales en cuanto a la oferta de información precisa sobre el estado y la ubicación de las procesiones o eventos.

1.4. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es el diseño y desarrollo de una aplicación móvil orientada a la Semana Santa, capaz de centralizar en una única plataforma la información de las distintas celebraciones de Semana Santa a nivel nacional, con independencia de la ciudad, cofradía o terminología empleada en cada una de ellas. En concreto, se busca ofrecer a los usuarios una herramienta que les permita consultar la información relevante de la Semana Santa de una ciudad y visualizar en tiempo real, mediante geolocalización, la ubicación, el recorrido y el estado de sus procesiones y los eventos.

Para alcanzar este objetivo, se plantean los siguientes objetivos específicos:

- Diseñar dicha aplicación permitiendo a los usuarios elegir entre las distintas Semanas Santas dentro del territorio nacional.
- Proporcionar acceso a la información general relacionada con la Semana Santa, incluyendo historia, cofradías o hermandades y elementos representativos.
- Mostrar información detallada sobre los eventos, como nombre, descripción, fechas, horas e itinerarios.
- Añadir funcionalidades de personalización, como la selección de eventos o elementos de interés y la recepción de notificaciones ante cambios o incidencias.
- Implementar un sistema de visualización sobre mapa que permita representar las procesiones y su evolución en tiempo real.
- Desarrollar una solución accesible desde dispositivos móviles que garantice una experiencia de uso adecuada.
- Diseñar una arquitectura escalable que permita la incorporación de nuevas ciudades y eventos sin necesidad de rediseñar el sistema.

1.5. Estructura de la memoria

PENDIENTE: revisar y actualizar este apartado al terminar el TFG.

La memoria se ha organizado en distintos capítulos en los cuales se puede ver la estructura y el desarrollo de dicho proyecto, de forma que se pueda comprender el proyecto de forma clara y completa.

En primer lugar, en este capítulo de introducción se presenta el contexto del proyecto, la motivación que lo impulsa, el estado actual de las distintas soluciones existentes, así como los objetivos que se llevan a cabo.

Después, en el capítulo de planificación se describe la metodología seguida durante el desarrollo de la aplicación, la organización del proyecto, además de las distintas fases de desarrollo y gestión de tareas.

A continuación, en el capítulo de análisis se identifican los actores del sistema, se especifican los requisitos funcionales, de información y no funcionales, y se detallan los casos de uso y el modelo de dominio sobre los que se sustenta el desarrollo de la aplicación.

Seguido por el capítulo de las iteraciones, en el cual se muestran las distintas etapas en las que se ha dividido y repartido el desarrollo de la aplicación, mostrando la evolución que ha ido teniendo el sistema a lo largo del desarrollo del proyecto.

Además, cuenta con un capítulo de estado, en el que se muestra el estado final del sistema desarrollado.

Por último, incluye un capítulo de pruebas en el que se analizan los resultados obtenidos al conseguir construir la versión final y operativa de la aplicación.

Esta memoria concluye con un capítulo de conclusiones, donde se valoran los objetivos conseguidos y se plantean posibles mejoras futuras.

Capítulo 2

Planificación

2.1. Metodología

Para el desarrollo de este proyecto se ha decidido hacer uso de una metodología **iterativa incremental** [11]. Este modelo se ha seleccionado porque permite organizar el proyecto en ciclos o iteraciones sucesivas, donde cada una añade funcionalidad al sistema de forma progresiva. Cada iteración incluye sus propias fases de análisis, diseño, implementación y pruebas, lo que facilita la validación continua del producto y la detección temprana de errores.

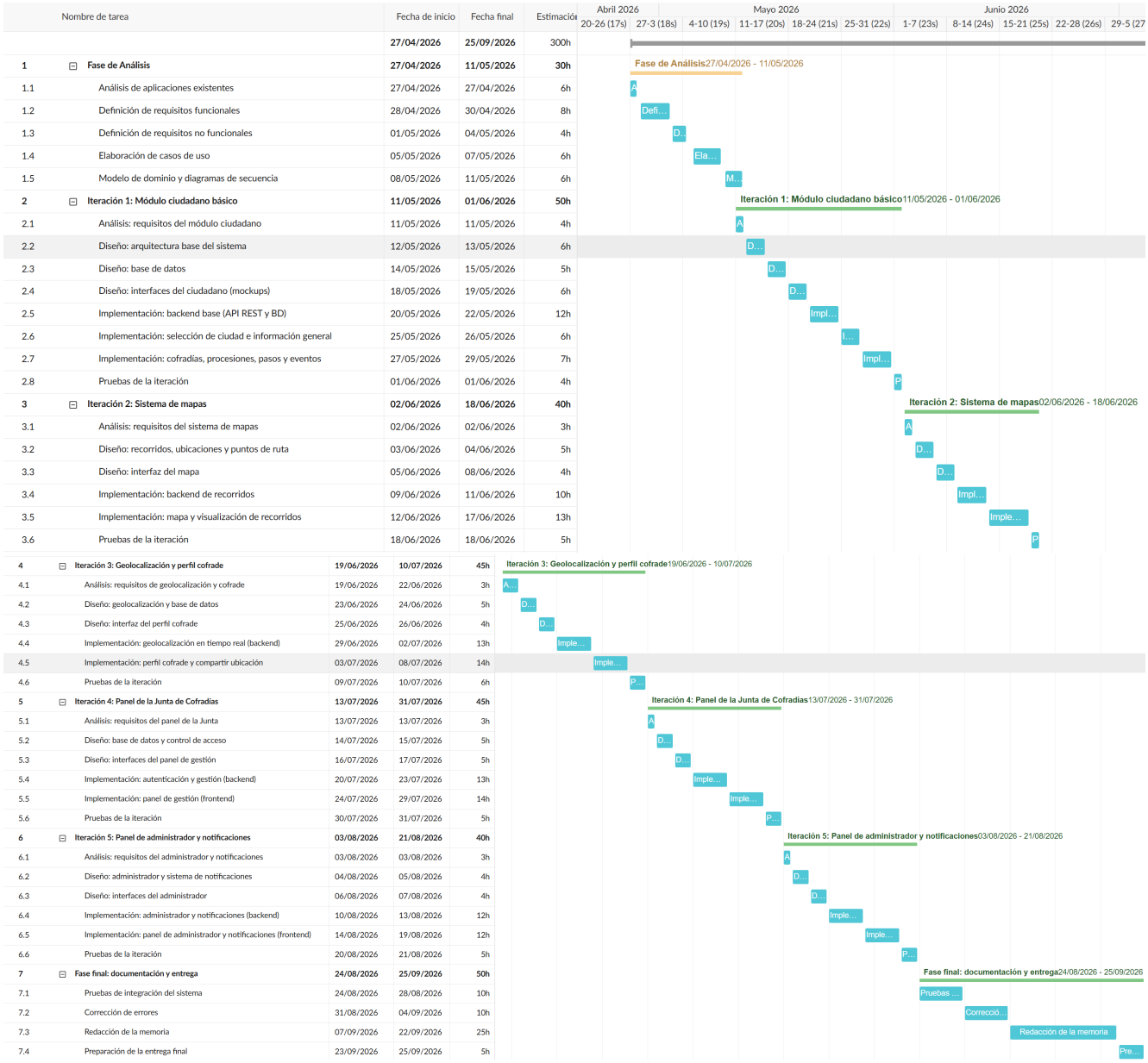
La elección de esta metodología está justificada por la naturaleza del proyecto, ya que permite entregar incrementos funcionales del sistema de manera gradual, comenzando por las funcionalidades básicas del módulo ciudadano y avanzando progresivamente hacia características más complejas como la geolocalización en tiempo real, los paneles de gestión y el sistema de notificaciones. Al ser un proyecto desarrollado por un único desarrollador, este modelo resulta adecuado porque combina la flexibilidad de adaptarse a posibles ajustes durante el desarrollo con una planificación estructurada que facilita el seguimiento del avance.

El proyecto se ha organizado en una **fase inicial de análisis** general, seguida de **cinco iteraciones** que abordan de forma incremental los distintos módulos del sistema, y una **fase final** de documentación, integración y entrega.

2.2. Planificación, Fases e iteraciones

2.2.1. Planificación inicial

Este proyecto consta de tres grandes fases, cuya planificación general se puede observar en el siguiente Diagrama de Gantt [12] (Figura 2.1): una primera fase de análisis, una segunda fase de desarrollo iterativo e incremental, organizada en cinco iteraciones, y una tercera y última fase de documentación y entrega.



	Nombre de tarea	Fecha de inicio	Fecha final	Estimación
		27/04/2026	**25/09/2026**	300h
1	**Fase de Análisis**	**27/04/2026**	**11/05/2026**	**30h**
1.1	Análisis de aplicaciones existentes	27/04/2026	27/04/2026	6h
1.2	Definición de requisitos funcionales	28/04/2026	30/04/2026	8h
1.3	Definición de requisitos no funcionales	01/05/2026	04/05/2026	4h
1.4	Elaboración de casos de uso	05/05/2026	07/05/2026	6h
1.5	Modelo de dominio y diagramas de secuencia	08/05/2026	11/05/2026	6h
2	**Iteración 1: Módulo ciudadano básico**	**11/05/2026**	**01/06/2026**	**50h**
2.1	Análisis: requisitos del módulo ciudadano	11/05/2026	11/05/2026	4h
2.2	Diseño: arquitectura base del sistema	12/05/2026	13/05/2026	6h
2.3	Diseño: base de datos	14/05/2026	15/05/2026	5h
2.4	Diseño: interfaces del ciudadano (mockups)	18/05/2026	19/05/2026	6h
2.5	Implementación: backend base (API REST y BD)	20/05/2026	22/05/2026	12h
2.6	Implementación: selección de ciudad e información general	25/05/2026	26/05/2026	6h
2.7	Implementación: cofradías, procesiones, pasos y eventos	27/05/2026	29/05/2026	7h
2.8	Pruebas de la iteración	01/06/2026	01/06/2026	4h
3	**Iteración 2: Sistema de mapas**	**02/06/2026**	**18/06/2026**	**40h**
3.1	Análisis: requisitos del sistema de mapas	02/06/2026	02/06/2026	3h
3.2	Diseño: recorridos, ubicaciones y puntos de ruta	03/06/2026	04/06/2026	5h
3.3	Diseño: interfaz del mapa	05/06/2026	08/06/2026	4h
3.4	Implementación: backend de recorridos	09/06/2026	11/06/2026	10h
3.5	Implementación: mapa y visualización de recorridos	12/06/2026	17/06/2026	13h
3.6	Pruebas de la iteración	18/06/2026	18/06/2026	5h
4	**Iteración 3: Geolocalización y perfil cofrade**	**19/06/2026**	**10/07/2026**	**45h**
4.1	Análisis: requisitos de geolocalización y cofrade	19/06/2026	22/06/2026	3h
4.2	Diseño: geolocalización y base de datos	23/06/2026	24/06/2026	5h
4.3	Diseño: interfaz del perfil cofrade	25/06/2026	26/06/2026	4h
4.4	Implementación: geolocalización en tiempo real (backend)	29/06/2026	02/07/2026	13h
4.5	Implementación: perfil cofrade y compartir ubicación	03/07/2026	08/07/2026	14h
4.6	Pruebas de la iteración	09/07/2026	10/07/2026	6h
5	**Iteración 4: Panel de la Junta de Cofradías**	**13/07/2026**	**31/07/2026**	**45h**
5.1	Análisis: requisitos del panel de la Junta	13/07/2026	13/07/2026	3h
5.2	Diseño: base de datos y control de acceso	14/07/2026	15/07/2026	5h
5.3	Diseño: interfaces del panel de gestión	16/07/2026	17/07/2026	5h
5.4	Implementación: autenticación y gestión (backend)	20/07/2026	23/07/2026	13h
5.5	Implementación: panel de gestión (frontend)	24/07/2026	29/07/2026	14h
5.6	Pruebas de la iteración	30/07/2026	31/07/2026	5h
6	**Iteración 5: Panel de administrador y notificaciones**	**03/08/2026**	**21/08/2026**	**40h**
6.1	Análisis: requisitos del administrador y notificaciones	03/08/2026	03/08/2026	3h
6.2	Diseño: administrador y sistema de notificaciones	04/08/2026	05/08/2026	4h
6.3	Diseño: interfaces del administrador	06/08/2026	07/08/2026	4h
6.4	Implementación: administrador y notificaciones (backend)	10/08/2026	13/08/2026	12h
6.5	Implementación: panel de administrador y notificaciones (frontend)	14/08/2026	19/08/2026	12h
6.6	Pruebas de la iteración	20/08/2026	21/08/2026	5h
7	**Fase final: documentación y entrega**	**24/08/2026**	**25/09/2026**	**50h**
7.1	Pruebas de integración del sistema	24/08/2026	28/08/2026	10h
7.2	Corrección de errores	31/08/2026	04/09/2026	10h
7.3	Redacción de la memoria	07/09/2026	22/09/2026	25h
7.4	Preparación de la entrega final	23/09/2026	25/09/2026	5h

Figura 2.1: Diagrama de Gantt general completo del proyecto

2.2.2. Fase de análisis

Esta primera fase se centra en el análisis de las aplicaciones relacionadas con la Semana Santa existentes, la identificación de sus carencias y áreas de mejora, y en una versión inicial de los requisitos funcionales y no funcionales [13] del sistema, además de la elaboración de los casos de uso [14] y los distintos diagramas que servirán como base para la realización del desarrollo de la aplicación.

Esta fase comienza con el análisis de las propuestas existentes en el mercado, identificando las características principales de aplicaciones como *El Penitente* [3], *Valladolid Cofrade* [5] o *sSantaZgz* [7].

Seguido de este análisis se realiza la definición de los requisitos funcionales, los cuales describen las funcionalidades concretas que el sistema debe ofrecer, y los requisitos no funcionales, que definen las

condiciones de calidad, rendimiento y seguridad que debe cumplir [13], teniendo en cuenta los cuatro perfiles de usuario: ciudadano, cofrade, Junta de Cofradías y administrador. También se elaboran los casos de uso [14], representaciones estandarizadas de las interacciones entre los usuarios y el sistema, así como el modelo de dominio y los diagramas de secuencia, que servirán para el desarrollo posterior.

2.2.3. Fase de iteraciones

Esta segunda fase está centrada en el desarrollo de la aplicación siguiendo un enfoque iterativo e incremental [15], organizado en cinco iteraciones. En cada una de ellas se repite un mismo ciclo de trabajo formado por las etapas de análisis, diseño, implementación y pruebas, de modo que las funcionalidades correspondientes a esa iteración se analizan, diseñan, implementan y validan antes de darla por finalizada y avanzar a la siguiente. Este enfoque permite ir incorporando las funcionalidades de forma progresiva, obteniendo al final de cada iteración una versión incremental y funcional del sistema. El desglose detallado de tareas y horas de cada iteración puede consultarse en el Diagrama de Gantt (Figura 2.1).

Estas iteraciones abarcan desde las funcionalidades básicas del ciudadano hasta las más complejas, como la geolocalización en tiempo real, es decir, la capacidad de conocer y actualizar la posición geográfica de una procesión de forma continua haciendo uso de la tecnología GPS[1] del móvil, o el sistema de notificaciones (mensajes enviados desde el servidor a los dispositivos de los usuarios sin que estos tengan que abrir la aplicación activamente).

Iteración 1: Módulo ciudadano básico

Esta primera iteración tiene como objetivo establecer la arquitectura base del sistema y desarrollar las funcionalidades fundamentales del perfil de ciudadano, entre las que se incluyen la selección de ciudad, la visualización de la información general de la Semana Santa, el listado de cofradías, procesiones y pasos y la consulta de eventos.

Iteración 2: Sistema de mapas

Esta segunda iteración se centra principalmente en la implementación del sistema de mapas, que permitirá visualizar los distintos recorridos de las procesiones.

Iteración 3: Geolocalización y perfil cofrade

Esta tercera iteración se centra en la implementación completa del sistema de geolocalización en tiempo real, de forma que se pueda conocer y actualizar de manera continua la posición de las procesiones

[1] GPS (Global Positioning System): sistema de navegación por satélite que permite determinar la posición geográfica de un dispositivo en cualquier punto del planeta.

durante su recorrido. Además, se desarrolla el perfil de cofrade con sus correspondientes funcionalidades, incluyendo la posibilidad de compartir la ubicación durante las procesiones.

Iteración 4: Panel de la Junta de Cofradías

Esta cuarta iteración corresponde al desarrollo del panel de gestión para las Juntas de Cofradías, permitiendo administrar la información de la Semana Santa, incluyendo eventos, recorridos, imágenes y participantes. El acceso a este panel está restringido mediante un mecanismo de control de acceso [16], que permite diferenciar los permisos y limitaciones de los distintos roles del sistema.

Iteración 5: Panel de administrador y notificaciones

Esta quinta y última iteración se centra en la implementación del panel de administrador global del sistema, así como del sistema de notificaciones, que permitirá a la Junta de Cofradías comunicar a los usuarios cambios o incidencias en las procesiones de forma inmediata, sin necesidad de que estos tengan la aplicación abierta.

2.2.4. Fase final: documentación y entrega

En esta última fase se realizan las pruebas de integración del sistema completo, la corrección de errores detectados, la finalización de la memoria del proyecto y la preparación de la entrega final.

2.3. Análisis de riesgos

Otra parte importante en la planificación de un proyecto de desarrollo de software es el análisis de riesgos. Un riesgo es un evento de incertidumbre que puede afectar de forma positiva o negativa a los objetivos del proyecto [17]. Para garantizar el alcance de los objetivos del proyecto, es necesario realizar un análisis de estos riesgos.

Los riesgos identificados se organizan en función de su prioridad, la cual se determina mediante la técnica de análisis de exposición al riesgo (*Risk Exposure*, RE) [18], donde el riesgo se calcula mediante un valor numérico que representa la probabilidad e impacto estimados de cada evento. Se define de la siguiente manera:

$$RE = \text{probabilidad} \times \text{impacto} \tag{2.1}$$

La variable probabilidad representa la probabilidad de que el riesgo ocurra en el proyecto (escala de 1 a 10), mientras que el impacto representa la gravedad de sus consecuencias en caso de ocurrir (escala de 1 a 10). En la Tabla 2.1 se muestran los riesgos identificados en el proyecto.

Riesgo	Probabilidad	Impacto	Prioridad
1. Estimaciones de tiempo erróneas	8	7	56
2. Falta de experiencia con las tecnologías del backend propio (Spring Boot)	7	7	49
3. Problemas con la integración de Google Maps	6	8	48
4. Errores durante el desarrollo	7	6	42
5. Retrasos o errores por falta de experiencia con tecnologías móviles	7	6	42
6. Problemas con la geolocalización en tiempo real	5	8	40
7. Cambios en los requisitos	6	6	36
8. Especificaciones ambiguas o poco definidas	5	6	30
9. Problemas con el entorno de desarrollo	4	5	20
10. Indisponibilidad del desarrollador	3	6	18

Tabla 2.1: Riesgos identificados en el proyecto

2.3.1. Planificación de riesgo

Tras realizar la identificación de los riesgos, el siguiente paso es la decisión de qué medidas se deben tomar para reducir la probabilidad de que el riesgo ocurra, de forma que se pueda disminuir su impacto. En las Tablas 2.2 a 2.11 se detallan, para cada uno de los riesgos identificados en la Tabla 2.1, las medidas de reducción propuestas para disminuir su probabilidad y las medidas de contingencia previstas en caso de que finalmente ocurran.

Riesgo	Estimaciones de tiempo erróneas
Reducción	Planificar las tareas con margen por si la estimación ha sido inferior al tiempo necesario. Dividir las tareas grandes en subtareas más pequeñas y manejables.
Contingencia	Cambiar la planificación, reducir el alcance del proyecto priorizando las funcionalidades esenciales.

Tabla 2.2: Riesgo 1: Estimaciones de tiempo erróneas

Riesgo	Problemas con la integración de Google Maps
Reducción	Realizar pruebas de concepto tempranas con la API de Google Maps. Documentarse sobre las limitaciones y cuotas del servicio.
Contingencia	Considerar alternativas como OpenStreetMap [19] o Mapbox en caso de problemas persistentes.

Tabla 2.3: Riesgo 2: Problemas con la integración de Google Maps

2.4. Estimación de costes

A continuación se presenta el presupuesto estimado del proyecto, calculado como si este fuese desarrollado por un equipo profesional completo, siguiendo los perfiles habituales en el desarrollo de una

Riesgo	Errores durante el desarrollo
Reducción	Verificar que la funcionalidad está implementada correctamente antes de dar por finalizado el incremento. Realizar pruebas unitarias[2]Pruebas unitarias: pruebas que verifican el correcto funcionamiento de componentes individuales del código de forma aislada.
Contingencia	Volver al último punto correctamente probado y continuar desde ahí. Utilizar control de versiones [20] para recuperar estados anteriores del código.

Tabla 2.4: Riesgo 3: Errores durante el desarrollo

Riesgo	Retrasos o errores por falta de experiencia con tecnologías móviles
Reducción	Formarse en las tecnologías clave antes de proceder al desarrollo. Consultar documentación oficial y tutoriales.
Contingencia	Aprovechar la experiencia del tutor para solventar los posibles problemas. Buscar soluciones en comunidades de desarrolladores.

Tabla 2.5: Riesgo 4: Retrasos o errores por falta de experiencia con tecnologías móviles

Riesgo	Problemas con la geolocalización en tiempo real
Reducción	Investigar las mejores prácticas para implementar geolocalización en aplicaciones móviles. Realizar prototipos tempranos para validar la viabilidad técnica.
Contingencia	Implementar un modo de actualización manual de la ubicación como alternativa temporal.

Tabla 2.6: Riesgo 5: Problemas con la geolocalización en tiempo real

Riesgo	Cambios en los requisitos
Reducción	El desarrollo iterativo permite adaptar el sistema a cambios de forma gradual.
Contingencia	Reevaluar las prioridades y ajustar el alcance del proyecto según las nuevas necesidades.

Tabla 2.7: Riesgo 6: Cambios en los requisitos

Riesgo	Especificaciones genéricas
Reducción	Reunir toda la información posible antes de abordar una tarea. Clarificar requisitos con el tutor.
Contingencia	Prototipar en caso de ser necesario para validar la interpretación de los requisitos.

Tabla 2.8: Riesgo 7: Especificaciones genéricas

aplicación móvil de estas características. Se trata de una estimación teórica con fines de planificación y no de los costes reales asumidos por el autor, quien ha desarrollado el TFG de forma individual.

Riesgo	Problemas con el entorno de desarrollo
Reducción	Utilizar un repositorio remoto (GitHub [20]) para almacenar copias de seguridad del código.
Contingencia	Utilizar una máquina de respaldo o entorno de desarrollo alternativo.

Tabla 2.9: Riesgo 8: Problemas con el entorno de desarrollo

Riesgo	Indisposición del desarrollador
Reducción	Planificar las tareas con margen por si es necesario extender el tiempo para completar el incremento.
Contingencia	Realizar horas extra en caso necesario y ajustar la planificación.

Tabla 2.10: Riesgo 9: Indisposición del desarrollador

Riesgo	Falta de experiencia con las tecnologías del backend propio (Spring Boot, Spring Security, JPA/Hibernate)
Reducción	Realizar una prueba de concepto acotada (*spike*) antes de comenzar la implementación completa, construyendo un único recurso extremo a extremo (entidad, autenticación JWT y *endpoint* protegido por rol) para validar el ritmo de desarrollo real con la tecnología. Consultar la documentación oficial y limitar el alcance del backend a lo estrictamente necesario para los requisitos, evitando patrones o herramientas adicionales que no aporten valor al proyecto (por ejemplo, no se contemplan microservicios ni colas de mensajería).
Contingencia	Si el retraso es significativo, reducir el alcance del backend priorizando los endpoints imprescindibles para cada iteración y posponiendo funcionalidades secundarias a iteraciones posteriores, tal y como permite la metodología iterativa incremental (Sección 2.1).

Tabla 2.11: Riesgo 10: Falta de experiencia con las tecnologías del backend propio

2.4.1. Costes de personal

Un proyecto de estas características, correspondiente al desarrollo completo de una aplicación móvil sobre la Semana Santa, requeriría en un entorno profesional un equipo multidisciplinar. Se han considerado los siguientes seis perfiles, entre los que se reparten las 300 horas estimadas de duración del proyecto:

- **Jefe de Proyecto**: coordinación general, planificación y seguimiento.
- **Analista Funcional**: recogida y especificación de requisitos, análisis del dominio.
- **Diseñador UX/UI**: diseño de la experiencia e interfaz de usuario.
- **Desarrollador Backend**: implementación del servidor, base de datos y API.

- **Desarrollador Frontend/Mobile**: implementación de la aplicación Android.
- **Tester/QA**: pruebas funcionales, de regresión y control de calidad.

Para cada perfil se ha calculado el coste por hora a partir del sueldo bruto anual medio en España, incrementado en un 30 % en concepto de cotizaciones a la Seguridad Social a cargo de la empresa, y dividido entre 1.800 horas anuales de jornada completa. La Tabla 2.12 recoge el reparto de horas y el coste asociado a cada perfil.

Perfil	Horas	Coste/hora	Coste total
Jefe de Proyecto [21]	45	33€	1.485,00€
Analista Funcional [22]	45	27€	1.215,00€
Diseñador UX/UI [23]	30	27€	810,00€
Desarrollador Backend [24]	75	26€	1.950,00€
Desarrollador Frontend/Mobile [25]	75	19€	1.425,00€
Tester/QA [26]	30	17€	510,00€
Total	**300**		**7.395,00€**

Tabla 2.12: Reparto de horas y coste de personal por perfil

2.4.2. Costes de equipo y amortización

El equipo del proyecto necesitaría un ordenador portátil por persona, además de un dispositivo Android dedicado a las pruebas. Se considera una duración total del proyecto de 5 meses.

- **Ordenadores portátiles**: 6 unidades valoradas en 1.000€ cada una, con una vida útil estimada de 5 años (60 meses). La amortización mensual por unidad es de 16,66€, lo que supone 83,30€ por portátil durante los 5 meses del proyecto, y 500,00€ en total para los 6 puestos de trabajo.
- **Dispositivo Android de pruebas**: 1 unidad valorada en 300€, con una vida útil estimada de 3 años (36 meses). La amortización mensual es de 8,33€, lo que supone 41,65€ para los 5 meses del proyecto.

Ahora bien, estos importes corresponden a la ocupación de los equipos durante los 5 meses completos del proyecto. Dado que el trabajo real asciende a 300 horas, frente a una capacidad disponible de 750 horas en ese periodo (150 horas/mes de jornada completa, según el criterio de 1.800 horas anuales aplicado en el apartado anterior), el resto del tiempo los equipos podrían destinarse a otro uso. Por ello, la amortización se imputa de forma proporcional al uso real del proyecto (300/750 = 40 %).

El coste de amortización así calculado asciende a **216,66€**, tal y como recoge la Tabla 2.13.

2.4.3. Costes de licencias de software

Buena parte de las herramientas empleadas disponen de planes gratuitos o licencias universitarias suficientes para un proyecto de este tamaño. Sin embargo, un equipo profesional de 6 personas necesitaría

Recurso	Valor	Vida útil	Amort. (5 meses)	Amort. en el proyecto (300h, 40 %)
Portátiles (x6)	6.000,00€	5 años	500,00€	200,00€
Dispositivo Android (x1)	300,00€	3 años	41,65€	16,66€
Total amortización			541,65€	**216,66€**

Tabla 2.13: Amortización de los recursos de equipo, prorrateada al uso real del proyecto

además herramientas colaborativas de pago para la gestión del proyecto y el diseño. La Tabla 2.14 recoge el coste y la fuente de cada licencia.

El backend se ha desplegado en un servidor virtual cedido por la Escuela de Ingeniería Informática de la Universidad de Valladolid, sin coste para el proyecto. Para reflejar lo que costaría fuera de ese entorno, se ha valorado un servidor virtual comercial de prestaciones similares (Hetzner Cloud CX23: 2 vCPU, 4 GB de RAM y 40 GB de disco NVMe), a 5,99€ al mes sin IVA con la dirección IPv4 pública incluida [27], lo que supone 29,95€ para los 5 meses del proyecto.

Herramienta	Coste	Fuente
Android Studio	0,00€	[28]
Visual Studio Code	0,00€	[29]
Overleaf (plan gratuito)	0,00€	—
Expo Push Notifications (plan gratuito)	0,00€	[30]
Expo Application Services, EAS Build (plan gratuito)	0,00€	[30]
Servidor virtual (equivalente: Hetzner Cloud CX23, 5 meses)	29,95€	[27]
Google Maps API (crédito gratuito)	0,00€	[1]
GitHub (plan gratuito)	0,00€	[20]
Cuenta de desarrollador Google Play (pago único)	23,00€	[31]
Figma Professional (1 licencia, 5 meses)	69,00€	[32]
Jira Software Standard (6 usuarios, 5 meses)	218,40€	[33]
Total licencias	**340,35€**	

Tabla 2.14: Coste de licencias de software

2.4.4. Otros costes indirectos

Un equipo de 6 personas trabajando de forma presencial o híbrida necesitaría un espacio de oficina. Se ha tomado como referencia el alquiler de un despacho de 22 m^2 en un espacio de coworking en el centro de Valladolid, con todos los suministros incluidos (calefacción, refrigeración, electricidad, wifi y limpieza), por un importe de 475€ al mes [34]. Para los 5 meses de duración del proyecto, el coste estructural asciende a 2.375,00€.

Al igual que con la amortización de equipos, este importe corresponde a la disponibilidad del espacio durante los 5 meses completos, mientras que el uso real del proyecto es de 300 horas frente a una capacidad de 750 horas en ese periodo (40 %); el resto del tiempo el espacio podría destinarse a otro uso. Aplicando este mismo criterio de proporcionalidad, el coste estructural imputado al proyecto asciende a **950,00€** (2.375,00€ × 40 %).

2.4.5. Coste total

La Tabla 2.15 resume el presupuesto completo del proyecto, considerando personal, amortización de equipo, licencias de software y costes estructurales —estos dos últimos prorrateados a las 300 horas reales de uso— para una duración de 300 horas repartidas en 5 meses.

Elemento	**Coste**
Personal (Tabla 2.12)	7.395,00€
Amortización de equipo, prorrateada (Tabla 2.13)	216,66€
Licencias de software (Tabla 2.14)	340,35€
Costes estructurales, prorrateados (oficina y suministros)	950,00€
Total	**8.902,01€**

Tabla 2.15: Costes totales del proyecto

2.5. Seguimiento del proyecto

Nota: esta sección se completará al finalizar el desarrollo del proyecto, comparando la planificación inicial con la duración real del proyecto. Se comentará que ha habido distintas replanificaciones debido a diversos recortes de tiempo que han surgido a lo largo de los meses de desarrollo.

Capítulo 3

Análisis

3.1. Actores del sistema

En el desarrollo de cualquier sistema software es fundamental identificar y definir los actores que interactuarán con la plataforma. Los actores representan a las entidades externas, ya sean personas u otros sistemas, que se comunican con la aplicación y hacen uso de sus funcionalidades.

En el caso de esta aplicación, orientada a la gestión y consulta de información sobre la Semana Santa, se han identificado cuatro actores principales: el ciudadano, el cofrade, la Junta de Cofradías y el administrador. Cada uno de ellos dispone de un nivel de acceso y unas funcionalidades diferenciadas, que se detallan a continuación y que sirven de base para la definición de los requisitos funcionales (Sección 3.2).

3.1.1. Ciudadano

El ciudadano es el actor principal de la aplicación y representa a cualquier persona interesada en seguir la Semana Santa, ya sea de forma presencial en la ciudad o de manera remota. No requiere registro previo para utilizar la aplicación.

Puede consultar la información detallada de cofradías, eventos, procesiones y pasos, realizar búsquedas y aplicar filtros, y visualizar en tiempo real la posición de las procesiones mediante geolocalización. Además, puede marcar contenido como favorito y recibir notificaciones sobre la ciudad que tiene seleccionada, activables o no desde el propio permiso de notificaciones del dispositivo.

3.1.2. Cofrade

El cofrade es la persona que comparte su ubicación en tiempo real mientras está procesionando. Accede a la aplicación introduciendo el código de acceso de su cofradía, generado por la Junta de Cofradía correspondiente, sin necesidad de registrarse.

Su única función es compartir su ubicación durante una procesión en curso de su cofradía: puede empezar a compartirla cuando la procesión comienza y dejar de hacerlo en cualquier momento, y la aplicación deja de compartirla automáticamente cuando la procesión finaliza. Esta ubicación es la que alimenta la geolocalización de la procesión que consulta el resto de usuarios.

3.1.3. Junta de Cofradías

La Junta de Cofradías es el actor responsable de la gestión del contenido asociado a la Semana Santa de su ciudad dentro de la aplicación. Accede a un panel de gestión desde el que puede crear y editar cofradías, eventos y procesiones. El recorrido de cada procesión se importa desde un archivo GPX, preparado previamente con una herramienta externa de trazado de rutas: de ese archivo la aplicación solo toma la secuencia ordenada de coordenadas, que forma el trazado de la procesión, sin ningún nombre ni información adicional. Una vez importado, la Junta revisa ese recorrido desde la propia aplicación y marca a mano cuáles de esos puntos son puntos de interés[1].

Asimismo, se encarga de actualizar el estado de las procesiones y de los eventos, de gestionar la información de ambos y de los pasos de cada cofradía, y de crear las notificaciones que reciben los ciudadanos y cofrades. También es responsable de generar, para cada cofradía, su propio código de acceso, el cual permite a los cofrades compartir su ubicación en la procesión de cofradía correspondiente.

3.1.4. Administrador

El administrador es el actor con mayor nivel de privilegios dentro del sistema y se encarga de su configuración y mantenimiento a nivel global. Su principal responsabilidad es la gestión de las ciudades disponibles en la aplicación, así como de la Junta de Cofradías de cada una, dando de alta, editando o dando de baja aquellas en las que el sistema ofrece cobertura. Además gestiona a los miembros de las distintas Juntas de Cofradías –dando de alta, editando o dando de baja su acceso, y resolviendo las solicitudes de reactivación de los miembros desactivados– y su propio perfil.

La Tabla 3.1 resume los cuatro actores identificados y su rol dentro del sistema.

3.2. Requisitos

3.2.1. Requisitos funcionales

Los requisitos funcionales definen las diferentes funciones del sistema, además de cómo debe ser su comportamiento en ciertas situaciones. Cada requisito incluye, junto a su descripción, el nivel de prioridad asignado (Alta, Media o Baja), que indica su importancia relativa dentro del proyecto.

[1]Punto de interés: lugar concreto del recorrido de una procesión al que se le da un significado propio dentro de la celebración, como un encuentro con otra procesión, la entrada a una iglesia de la que la procesión sale más tarde, o una parada para una lectura o un cántico.

Actor	Rol
Ciudadano	Consulta información y sigue la Semana Santa sin necesidad de registro.
Cofrade	Ciudadano perteneciente a una cofradía; comparte su ubicación durante las procesiones.
Junta de Cofradías	Gestiona todo el contenido de la Semana Santa de su ciudad: cofradías, eventos, procesiones, recorridos y notificaciones.
Administrador	Gestiona la configuración global del sistema, en particular las ciudades disponibles y sus juntas de cofradias.

Tabla 3.1: Resumen de los actores del sistema

Para facilitar su lectura, los requisitos se han agrupado según el perfil de usuario (Sección 3.1) al que van dirigidos –ciudadano, cofrade, Junta de Cofradías o administrador–. Dentro de cada grupo se han ordenado de mayor a menor prioridad, salvo en el del ciudadano: al ser el grupo más numeroso, sus requisitos se listan siguiendo en su lugar el flujo de uso habitual de la aplicación (desde que se accede hasta la geolocalización en tiempo real y las notificaciones), para que el lector pueda seguirlos en el mismo orden en que se encontraría con ellos.

Ciudadano

- **RF-01** *(Prioridad: Alta)* El sistema debe permitir el uso de la aplicación como ciudadano sin la necesidad de realizar un registro previo.
- **RF-02** *(Prioridad: Alta)* El sistema debe permitir al usuario seleccionar la ciudad de la que desea consultar la Semana Santa.
- **RF-03** *(Prioridad: Alta)* El sistema debe solicitar permiso a los usuarios para acceder a su ubicación en tiempo real.
- **RF-04** *(Prioridad: Media)* El sistema debe seleccionar automáticamente la ciudad más cercana en función de la ubicación del usuario.
- **RF-05** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de las cofradías.
- **RF-06** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de los eventos.
- **RF-07** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de las procesiones.
- **RF-08** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de los pasos.
- **RF-09** *(Prioridad: Alta)* El sistema debe mostrar un calendario con los días de la Semana Santa, indicando en cada día si hay eventos o procesiones programadas.
- **RF-10** *(Prioridad: Alta)* El sistema debe permitir a los usuarios seleccionar un día concreto del calendario para consultar el listado de procesiones y eventos programados ese día.

- **RF-11** *(Prioridad: Media)* El sistema debe permitir a los usuarios acceder, desde un menú principal, a los listados completos de procesiones, pasos, eventos y cofradías.
- **RF-12** *(Prioridad: Alta)* El sistema debe permitir a los usuarios realizar búsquedas sobre eventos, procesiones, pasos o cofradías específicas.
- **RF-13** *(Prioridad: Media)* El sistema debe permitir filtrar eventos y procesiones por día y por estado.
- **RF-14** *(Prioridad: Media)* El sistema debe permitir a los usuarios marcar cofradías, procesiones, eventos y pasos como favoritos, y consultarlos reunidos en un listado propio de favoritos.
- **RF-15** *(Prioridad: Media)* El sistema debe permitir al usuario interactuar con el mapa (zoom, desplazamiento, ...).
- **RF-16** *(Prioridad: Alta)* Sin que el usuario tenga que seleccionar ninguna cofradía, evento o procesión, el sistema debe mostrar por defecto en el mapa las procesiones que están actualmente en curso.
- **RF-17** *(Prioridad: Baja)* El sistema debe mostrar en el mapa los puntos de interés del recorrido de las procesiones que se estén mostrando.
- **RF-18** *(Prioridad: Alta)* El sistema debe permitir visualizar la posición de las procesiones en tiempo real mediante geolocalización.
- **RF-19** *(Prioridad: Baja)* El sistema debe poder sugerir rutas hacia las ubicaciones más relevantes de una procesión (inicio, puntos de interés, fin, ...).
- **RF-20** *(Prioridad: Baja)* El sistema debe poder generar rutas entre dos puntos evitando las procesiones activas.
- **RF-21** *(Prioridad: Alta)* El sistema debe enviar a cada usuario las notificaciones de la ciudad que tiene seleccionada en la aplicación (RI-12), siempre que no haya desactivado su recepción (RF-22).
- **RF-22** *(Prioridad: Media)* El sistema debe permitir al usuario desactivar la recepción de notificaciones a través del propio permiso de notificaciones del dispositivo, mostrando en su perfil si las tiene activadas y dando acceso directo a los ajustes del dispositivo para cambiarlo.

Cofrade

- **RF-23** *(Prioridad: Alta)* El sistema debe permitir a los cofrades acceder al modo cofrade, desde el inicio de sesión, introduciendo el código de acceso de su cofradía generado por la Junta de Cofradías, sin necesidad de registrarse.
- **RF-24** *(Prioridad: Alta)* El sistema debe permitir a los cofrades compartir su ubicación en tiempo real únicamente mientras una procesión de su cofradía esté en curso –eligiendo cuál si hay varias en curso a la vez–, y dejar de compartirla en cualquier momento. El sistema debe dejar de compartirla automáticamente, informando al cofrade del motivo, cuando la procesión finalice, cuando se revoque su código de acceso o cuando caduque su sesión. Si el cofrade concede el permiso de

ubicación permanente, la ubicación se debe seguir enviando aunque la pantalla esté bloqueada; en caso contrario, solo mientras la aplicación esté abierta.

- **RF-25** *(Prioridad: Media)* El sistema debe descartar, sin guardarlas ni representarlas, las posiciones recibidas de los cofrades que se encuentren a más de 50 metros del recorrido de la procesión.

Miembro de la Junta de Cofradías

- **RF-26** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías iniciar sesión con su correo electrónico y su contraseña. Al ser dado de alta, el miembro recibe por correo electrónico una contraseña provisional generada por el sistema, y el sistema debe obligarle a sustituirla por una propia la primera vez que inicie sesión, sin permitirle acceder al panel de gestión ni realizar ningún cambio hasta hacerlo.
- **RF-27** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías gestionar la información de su propio perfil.
- **RF-28** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías crear y eliminar cofradías, eventos y procesiones.
- **RF-29** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías editar información de cofradías, eventos, procesiones y de la Semana Santa de su ciudad.
- **RF-30** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías gestionar (dar de alta, editar y dar de baja) los pasos que pertenecen a cada una de las cofradías de su ciudad –un paso pertenece siempre a una única cofradía (RI-10), aunque después pueda participar en varios eventos y procesiones de esa misma cofradía–.
- **RF-31** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías emitir, cuando lo necesite, un código de acceso para una cofradía, para que sus cofrades puedan acceder al modo cofrade; el valor del código lo genera automáticamente el propio sistema, sin que tenga que escribirlo a mano. También debe permitirle revocar un código emitido, de modo que deje de dar acceso al modo cofrade.
- **RF-32** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías importar el recorrido de una procesión desde un archivo GPX, del que se toma la secuencia ordenada de coordenadas que forma el trazado.
- **RF-33** *(Prioridad: Baja)* El sistema debe permitir al miembro de la Junta de Cofradías marcar como punto de interés cualquier punto del recorrido importado de una procesión, indicando su tipo, su nombre y, opcionalmente, una descripción y una imagen, y editar después esa información.
- **RF-34** *(Prioridad: Alta)* El sistema debe actualizar automáticamente el estado de las procesiones y de los eventos según su horario: deben pasar a en curso al llegar su hora de inicio y a finalizado al llegar su hora de fin. Además, el sistema debe permitir al miembro de la Junta de Cofradías corregir ese estado a mano (programado, en curso, finalizado o cancelado), salvo cuando el cambio

contradiga el horario –volver a programado una vez pasada la hora de inicio, o a en curso una vez pasada la hora de fin–, en cuyo caso debe indicarle que modifique la hora en lugar del estado.

- **RF-35** *(Prioridad: Alta)* Cuando una procesión o un evento pase a en curso o a finalizado, ya sea automáticamente o porque lo cambie a mano el miembro de la Junta de Cofradías, el sistema debe generar y enviar la notificación correspondiente (RI-08) sin que tenga que crearla. Cuando el cambio sea a cancelado, el sistema no debe avisar por sí solo, sino abrir a continuación la creación de una notificación de cancelación para que el miembro de la Junta indique el motivo (RF-36).
- **RF-36** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías crear notificaciones dirigidas a todos los usuarios sobre incidencias o cambios en eventos y procesiones.

Administrador

- **RF-37** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar y dar de baja) las ciudades disponibles.
- **RF-38** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar y dar de baja) las Juntas de Cofradías del sistema.
- **RF-39** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar, desactivar y dar de baja) a los miembros de las Juntas de Cofradías. Al dar de alta a un miembro, el sistema debe generar una contraseña provisional y enviársela por correo electrónico junto a un mensaje de bienvenida. Mientras el miembro no haya cambiado esa contraseña, el administrador debe verlo como invitación pendiente y poder reenviarle la invitación, lo que genera una contraseña provisional nueva e invalida la anterior.
- **RF-40** *(Prioridad: Media)* El sistema debe permitir al administrador gestionar la información de su propio perfil.
- **RF-41** *(Prioridad: Media)* El sistema debe permitir a un miembro desactivado de una Junta de Cofradías solicitar su reactivación, y al administrador aceptarla o rechazarla.

3.2.2. Requisitos de información

Los requisitos de información definen los datos que el sistema debe almacenar y gestionar para su correcto funcionamiento. Al igual que en los requisitos funcionales, se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. En este caso no se agrupan por perfil de usuario, ya que describen el modelo de datos del sistema en su conjunto y no están asociados a un único actor.

- **RI-01** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las ciudades:
  - Identificador.
  - Nombre.

  - Comunidad autónoma.
  - Provincia.
  - Historia.
  - Patrimonio.
  - Si está activa (visible para el ciudadano) o no.
  - Latitud y longitud (opcionales), para poder preseleccionar la ciudad más cercana al Ciudadano por GPS.
  - Junta de Cofradías que la gestiona.

- **RI-02** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las Juntas de Cofradías/hermandades
  - Identificador.
  - Nombre.
  - Email.
  - Teléfono.
  - Si está activa o desactivada (una Junta desactivada no puede hacer cambios).
  - Ciudad que gestiona (relación 1:1; no agrupa directamente a las cofradías de esa ciudad, ver RI-04).
  - Miembros que la componen.

- **RI-03** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los usuarios con credenciales propias –miembros de Junta de Cofradías y administradores; el ciudadano y el cofrade nunca se registran, ver Sección B.10–:
  - Identificador.
  - Email.
  - Contraseña, como *hash* (nunca en texto plano).
  - Fecha de ingreso.
  - Si es miembro de Junta de Cofradías, además: nombre, teléfono, Junta de Cofradías a la que pertenece, si está activo o desactivado, si sigue usando la contraseña provisional que se le generó al darlo de alta, y si tiene una solicitud de reactivación pendiente.
  - Si es administrador, además: nombre y teléfono (opcionales, se completan al editar su perfil). Un administrador no está atado a ninguna ciudad en concreto: cualquier administrador puede gestionar cualquier ciudad.

- **RI-04** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las cofradías/hermandades:
  - Identificador.
  - Nombre.
  - Historia.
  - Web oficial.

  - Imagen.
  - Fecha de creación.
  - Si está activa (visible para el ciudadano) o todavía en preparación.
  - Ciudad a la que pertenece –nunca a la Junta de Cofradías que la gestiona, ver Apéndice B–.
  - Pasos que posee, y eventos y procesiones en los que participa.

- **RI-05** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los eventos:
  - Identificador.
  - Nombre.
  - Historia.
  - Tradición.
  - Fecha y hora de inicio.
  - Fecha y hora de fin (opcional; sin ella, el evento no se finaliza automáticamente).
  - Estado (programado, en curso, finalizado o cancelado).
  - Web oficial.
  - Ubicación en la que se celebra.
  - Cofradías que participan en él (puede ser más de una).
  - Pasos que participan en él.
- **RI-06** *(Prioridad: Alta)* El sistema debe almacenar el histórico de posiciones que comparten los cofrades durante una procesión, como lecturas **anónimas** –sin ningún dato que identifique a la persona que las envía, ver Sección B.10–:
  - Identificador.
  - Latitud y longitud.
  - Instante de la lectura.
  - Procesión a la que pertenece.
- **RI-07** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los recorridos:
  - Identificador.
  - Distancia total.
  - Tiempo estimado.
  - Puntos de ruta que lo componen, cada uno con su orden dentro del recorrido y la hora prevista de paso.
- **RI-08** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las notificaciones:
  - Identificador.
  - Título.
  - Mensaje.

  - Fecha de creación.
  - Fecha de expiración (opcional).
  - Tipo (inicio, fin, incidencia, cambio de horario, cancelación o actualización).
  - Prioridad (baja, media o alta; obligatoria salvo en las de tipo inicio y fin, que genera el propio sistema).
  - Ciudad a la que pertenece.

- **RI-09** *(Prioridad: Media)* Al ser una procesión un tipo concreto de evento, el sistema debe almacenar sobre ella, además de toda la información de RI-05 (incluidos los pasos que participan en ella):
  - Recorrido asociado (opcional hasta que se importe).
  - Histórico de posiciones anónimas de sus cofrades (RI-06), a partir del cual se calcula al vuelo su posición actual mientras está en curso.
  - Progreso alcanzado, a lo largo del recorrido, por el cofrade más adelantado y por el más atrasado que están compartiendo ubicación, para pintar la estela en vivo de la procesión.
- **RI-10** *(Prioridad: Media)* El sistema debe almacenar la información sobre los pasos:
  - Identificador.
  - Nombre.
  - Historia.
  - Análisis artístico.
  - Imagen.
  - Cofradía a la que pertenece.
  - Eventos y procesiones en los que participa.
- **RI-11** *(Prioridad: Media)* El sistema debe almacenar la información sobre los puntos de un recorrido y demás ubicaciones de referencia utilizadas en el mapa:
  - Latitud.
  - Longitud.
  - Dirección (opcional).
  - Si el punto se destaca como punto de interés (un encuentro, la entrada a una iglesia, una parada para una lectura u oración, un monumento...), además: tipo (monumento, iglesia, encuentro, oración, entrada de procesión o salida de procesión), nombre, descripción e imagen.
- **RI-12** *(Prioridad: Media)* El sistema debe almacenar la información sobre los dispositivos registrados para recibir notificaciones *push*:
  - Identificador.
  - Token de Expo Push del dispositivo.
  - Ciudad de la que quiere recibir notificaciones.

  - Fecha de registro y de última actualización.

- **RI-13** *(Prioridad: Media)* El sistema debe almacenar la información sobre los códigos de acceso de las cofradías:
  - Identificador.
  - Código, generado automáticamente por el sistema y único.
  - Estado (emitido, validado –ya se ha usado al menos una vez para entrar– o revocado).
  - Cofradía a la que da acceso.

3.2.3. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad del sistema, así como restricciones sobre su funcionamiento. También se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. Al igual que los requisitos de información, no se agrupan por perfil de usuario, ya que son de aplicación transversal a todo el sistema.

- **RNF-01** *(Prioridad: Alta)* El sistema debe proteger los datos de los usuarios, especialmente la información de ubicación: las posiciones que comparten los cofrades deben guardarse de forma anónima, sin ningún dato que identifique a quien las envía (RI-06), y las contraseñas no deben almacenarse en texto plano (RI-03).
- **RNF-02** *(Prioridad: Alta)* La aplicación debe ser accesible y fácil de usar para usuarios sin experiencia previa.
- **RNF-03** *(Prioridad: Alta)* El sistema debe implementar mecanismos de control de acceso que diferencien entre ciudadanos, cofrades, miembros de la junta de cofradías y administrador.
- **RNF-04** *(Prioridad: Alta)* El sistema debe ser compatible tanto con dispositivos Android como con dispositivos iOS.
- **RNF-05** *(Prioridad: Alta)* El sistema debe actualizar la posición de las procesiones con una frecuencia mínima de 30 segundos[2].
- **RNF-06** *(Prioridad: Media)* El sistema debe funcionar correctamente en dispositivos móviles con conexión a internet. Sin conexión, la aplicación debe seguir mostrando, durante un máximo de 24 horas desde que se consultaron, la lista de ciudades, la información de la ciudad y de las cofradías ya visitadas, y la ubicación de los eventos y el recorrido de las procesiones ya consultados. El resto de funcionalidades, como la geolocalización en tiempo real, las notificaciones o la consulta de procesiones y eventos, requieren conexión.
- **RNF-07** *(Prioridad: Media)* El sistema debe gestionar de forma eficiente la carga del mapa, garantizando tiempos de respuesta menores a 10 segundos[3].

[2]Este intervalo se ha elegido como equilibrio entre ofrecer una posición suficientemente precisa para el usuario y evitar un consumo excesivo de batería y datos móviles en los dispositivos de los cofrades que comparten su ubicación.

[3]Valor tomado como referencia habitual de usabilidad para evitar que el usuario perciba la aplicación como lenta al cargar el mapa.

- **RNF-08** *(Prioridad: Media)* El sistema debe ser capaz de enviar notificaciones de manera fiable y en tiempo adecuado (menos de 60 segundos de latencia).

3.3. Casos de uso

Los casos de uso son los escenarios detallados que describen cómo interactúan los diferentes usuarios con la aplicación, desde la consulta de información hasta la gestión administrativa de la Semana Santa. Debido a que los usuarios pueden comportarse con diferentes roles (ciudadano, cofrade, miembro de la Junta de Cofradías o administrador), es importante diferenciar los casos de uso entre cada uno de ellos. Para ello se han dividido los mismos en cuatro diagramas (ver figuras 3.1, 3.2, 3.3 y 3.4), uno para cada rol.

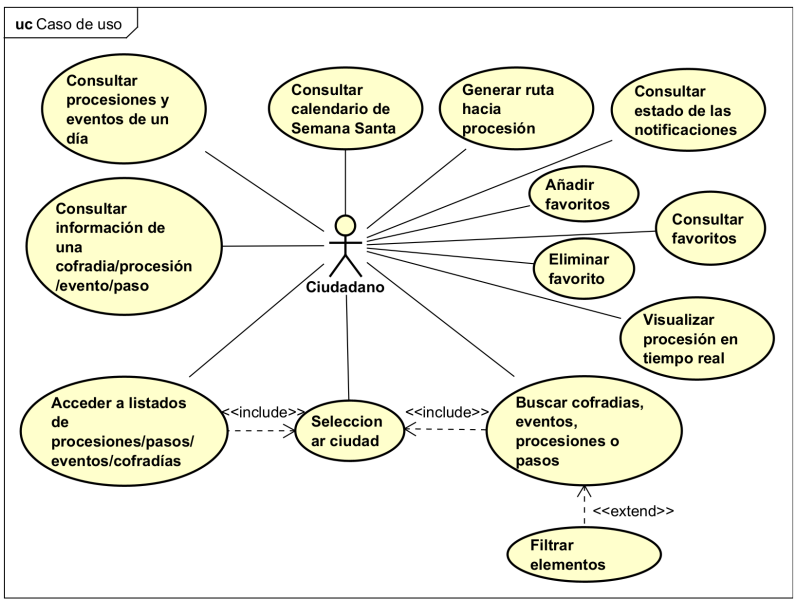


Figura 3.1: Diagrama de caso de uso Ciudadano

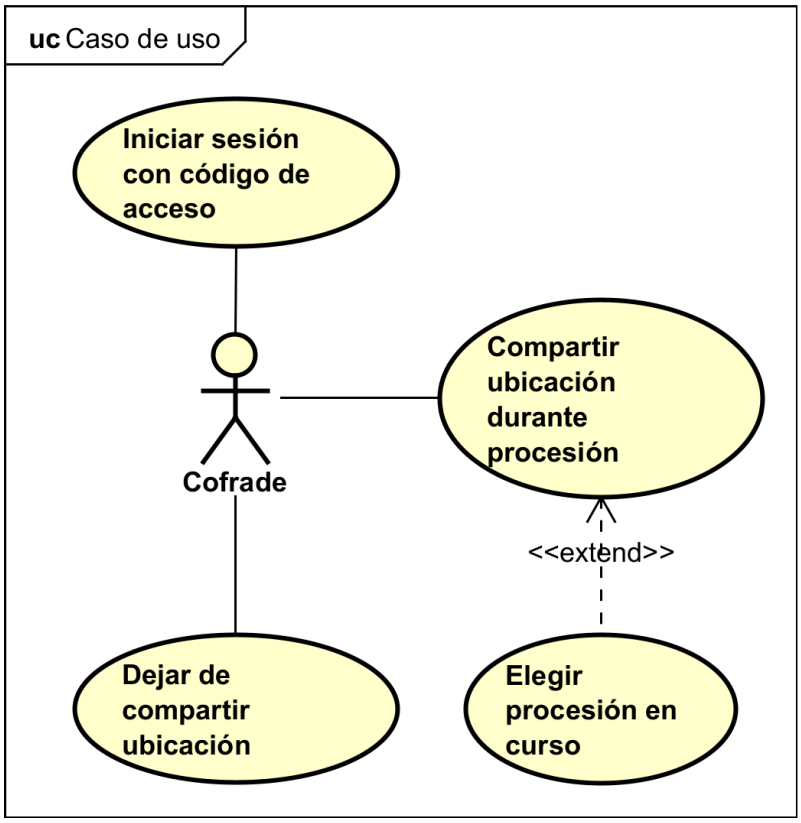


Figura 3.2: Diagrama de caso de uso Cofrade

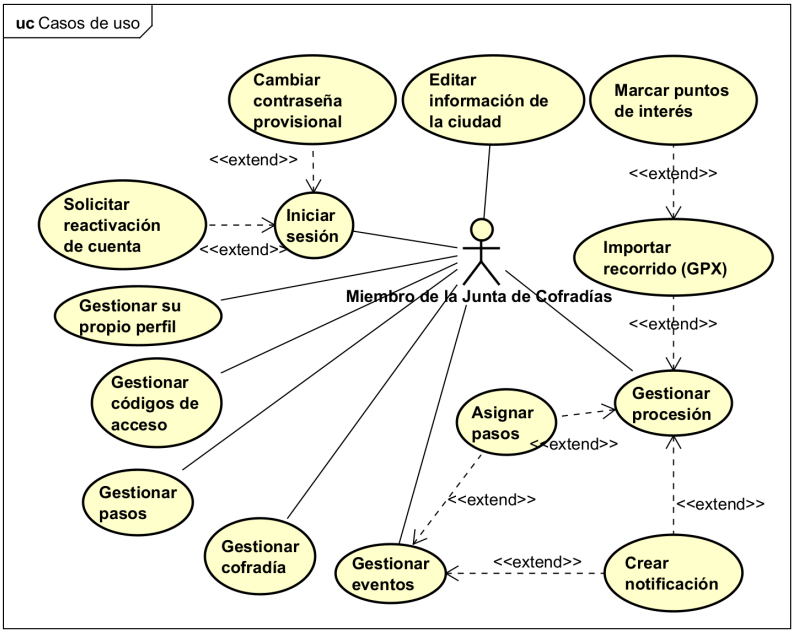


Figura 3.3: Diagrama de caso de uso Miembro de la Junta de Cofradías

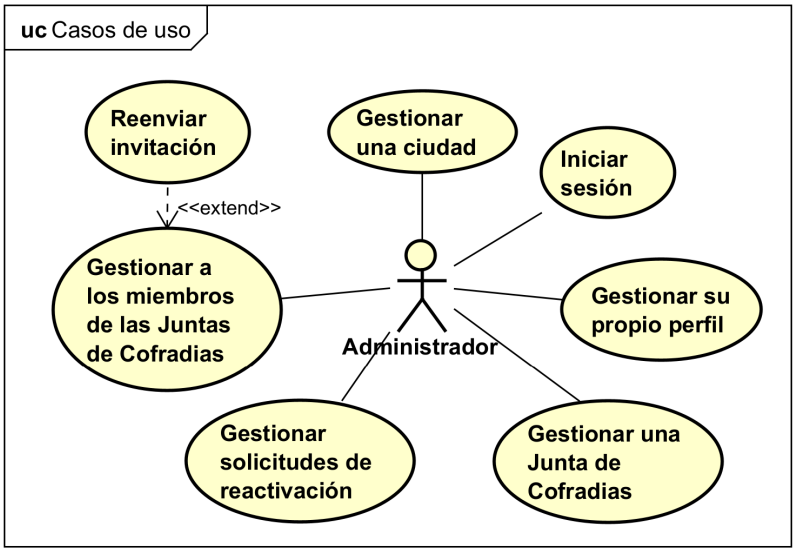


Figura 3.4: Diagrama de caso de uso Administrador

En las siguientes tablas se exponen los cinco casos de uso más representativos del sistema, que cubren a los cuatro roles implicados (ciudadano, cofrade, miembro de la Junta de Cofradías y administrador). En cada uno se detallan: el caso de uso, los actores que lo realizan, las precondiciones necesarias para iniciarlo, las postcondiciones que deben cumplirse al completarse, una secuencia base de pasos, las secuencias alternativas que pueden darse durante su ejecución y los requisitos relacionados. El resto de los casos de uso identificados, de carácter más secundario, se recogen en el Apéndice A.

3.4. Modelo de dominio

En esta sección se detalla el diseño conceptual del sistema, incluyendo las entidades clave y sus relaciones. El modelo refleja la estructura lógica del dominio de la Semana Santa, garantizando una representación fiel de sus procesos, actores y reglas de negocio. Se explican brevemente las abstracciones incluidas en la Figura 3.5.

Caso de uso: Visualizar procesión en tiempo real	
Resumen	El usuario consulta en el mapa la posición en tiempo real de las procesiones que están en curso en la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad y dispone de conexión a internet.
Postcondición	Se muestran sobre el mapa el recorrido de las procesiones en curso y el tramo que ocupa cada una en ese momento.
Secuencia Base	
[1] El usuario solicita consultar las procesiones en tiempo real.	
[2] El sistema muestra el mapa de la ciudad seleccionada con las procesiones que están en curso en ese momento, sin que el usuario tenga que elegir ninguna.	
[3] El sistema dibuja el recorrido de cada procesión en curso y, sobre él, el tramo que ocupa en ese momento, desde el cofrade más atrasado hasta el más adelantado que comparten su ubicación.	
[4] El sistema actualiza periódicamente la posición de cada procesión, al menos cada 30 segundos.	
[5] El usuario interactúa con el mapa (zoom y desplazamiento) o elige una de las procesiones en curso para centrar el mapa en ella.	
Secuencia Alternativa	
[2.a.1] No hay ninguna procesión en curso: el sistema muestra el mapa de la ciudad e informa de que no hay procesiones en movimiento.	
[3.a.1] Ningún cofrade comparte su ubicación todavía: el sistema muestra el recorrido de la procesión sin el tramo ocupado.	
[4.a.1] Pérdida de conexión: el sistema mantiene la última posición conocida hasta que la conexión se recupera.	
Requisitos Relacionados	RF-15, RF-16 y RF-18.

Tabla 3.2: Caso de uso: Visualizar procesión en tiempo real

Caso de uso: Crear notificación	
Resumen	El miembro de la Junta de Cofradías informa a los ciudadanos de su ciudad de una incidencia o un cambio en una procesión o un evento.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Acaba de guardar cambios en una procesión o un evento de su ciudad (casos de uso «Gestionar procesión» y «Gestionar eventos»).
Postcondición	La notificación queda registrada en el sistema y se envía como notificación *push* a los dispositivos registrados en la ciudad.
Secuencia Base	
[1] El usuario indica que quiere notificar el cambio realizado.	
[2] El sistema solicita el tipo de notificación (actualización de información, cambio de horario, incidencia o cancelación), su prioridad (baja, media o alta) y, opcionalmente, un mensaje.	
[3] El usuario introduce los datos de la notificación.	
[4] El usuario confirma el envío.	
[5] El sistema registra la notificación con el nombre de la procesión o del evento como título.	
[6] El sistema envía la notificación *push* a todos los dispositivos registrados en la ciudad.	
Secuencia Alternativa	
[1.a.1] El cambio guardado ha sido una cancelación: el sistema inicia directamente este caso de uso con el tipo cancelación ya elegido, para que el usuario indique el motivo.	
[1.a.2] El usuario no quiere notificar el cambio: el caso de uso termina sin crear ninguna notificación.	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Datos incompletos: falta el tipo o la prioridad; el sistema indica qué datos son obligatorios y no la envía.	
[5.a.1] Tipo cancelación: además de registrar la notificación, el sistema cambia el estado de la procesión o del evento a cancelado.	
[6.a.1] Error al enviar la notificación *push*: la notificación queda registrada y consultable en la aplicación igualmente.	
Requisitos Relacionados	RF-35 y RF-36.

Tabla 3.3: Caso de uso: Crear notificación

Caso de uso: Iniciar sesión con código de acceso	
Resumen	Un cofrade accede al modo cofrade introduciendo el código de acceso de su cofradía, sin necesidad de registrarse.
Actor	Cofrade
Precondición	La Junta de Cofradías ha emitido un código de acceso para la cofradía del cofrade y no lo ha revocado.
Postcondición	El cofrade queda identificado como cofrade de esa cofradía y puede compartir su ubicación durante sus procesiones en curso.
Secuencia Base	
[1] El usuario solicita acceder como cofrade.	
[2] El sistema solicita el código de acceso de la cofradía.	
[3] El usuario introduce el código de acceso.	
[4] El sistema comprueba que el código existe, que no ha sido revocado y que la cofradía está activa.	
[5] El sistema identifica al usuario como cofrade de esa cofradía, sin crear ningún usuario ni guardar ningún dato que lo identifique, y le da acceso al modo cofrade.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Código inexistente: el sistema informa de que el código no es válido y no concede el acceso.	
[4.a.2] Código revocado: el sistema informa de que el código ha sido revocado y no concede el acceso.	
[4.a.3] Cofradía desactivada: el sistema informa de que la cofradía está desactivada y no concede el acceso.	
Requisitos Relacionados	RF-23.

Tabla 3.4: Caso de uso: Iniciar sesión con código de acceso

Caso de uso: Compartir ubicación durante procesión	
Resumen	Un cofrade comparte su ubicación en tiempo real mientras participa en una procesión en curso de su cofradía, para alimentar la posición en vivo que consultan los ciudadanos.
Actor	Cofrade
Precondición	El cofrade ha accedido al modo cofrade (caso de uso «Iniciar sesión con código de acceso»).
Postcondición	El sistema recibe periódicamente la ubicación del cofrade, de forma anónima, hasta que deja de compartirla o la procesión finaliza.
Secuencia Base	
[1] El cofrade solicita compartir su ubicación.	
[2] El sistema comprueba que la cofradía tiene una procesión en curso en ese momento.	
[3] El sistema solicita permiso para acceder a la ubicación del dispositivo, si todavía no lo tiene concedido.	
[4] El cofrade concede el permiso.	
[5] El sistema envía la ubicación del cofrade de forma periódica, al menos cada 30 segundos, y sigue haciéndolo aunque la pantalla del dispositivo esté bloqueada.	
[6] El sistema descarta las posiciones que se encuentran a más de 50 metros del recorrido de la procesión, y guarda el resto como lecturas anónimas.	
[7] El sistema informa al cofrade de que está compartiendo su ubicación y en qué procesión.	
Secuencia Alternativa	
[2.a.1] La cofradía no tiene ninguna procesión en curso: el sistema informa de ello y no comparte la ubicación.	
[2.a.2] La cofradía tiene varias procesiones en curso a la vez: se realiza el caso de uso «Elegir procesión en curso».	
[3.a.1] Permiso ya concedido anteriormente: el sistema pasa directamente al paso 5.	
[4.a.1] El cofrade deniega el permiso: el sistema informa de que no puede compartir la ubicación sin él, y el caso de uso termina.	
[5.a.1] El cofrade solo concede el permiso mientras la aplicación está abierta: el sistema le avisa de que únicamente compartirá la ubicación con la aplicación abierta.	
[5.a.2] Pérdida de conexión: el sistema vuelve a intentar el envío en el siguiente ciclo.	
[5.a.3] La procesión finaliza, la Junta revoca el código de acceso o caduca la sesión: el sistema deja de compartir la ubicación automáticamente e informa al cofrade del motivo.	
[7.a.1] El cofrade decide dejar de compartir su ubicación: se realiza el caso de uso «Dejar de compartir ubicación».	
Requisitos Relacionados	RF-24 y RF-25.

Tabla 3.5: Caso de uso: Compartir ubicación durante procesión

Caso de uso: Gestionar una ciudad	
Resumen	El administrador da de alta, edita o da de baja las ciudades disponibles en la aplicación.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador.
Postcondición	Las ciudades disponibles quedan actualizadas en el sistema.
Secuencia Base	
[1] El usuario solicita gestionar las ciudades.	
[2] El sistema muestra las ciudades registradas, activas o no.	
[3] El usuario indica si quiere dar de alta una ciudad nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la ciudad: nombre, comunidad autónoma, provincia, historia, patrimonio, si está activa (visible para los ciudadanos) y, opcionalmente, su latitud y longitud.	
[5] El usuario introduce o modifica los datos y confirma la operación.	
[6] El sistema valida los datos y guarda la ciudad.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la ciudad junto con sus notificaciones y sus dispositivos registrados.	
[3.a.2] La ciudad tiene una Junta de Cofradías o cofradías asociadas: el sistema no permite darla de baja e informa de que antes hay que eliminarlas, o bien desactivar la ciudad.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la ciudad.	
Requisitos Relacionados	RF-37.

Tabla 3.6: Caso de uso: Gestionar una ciudad

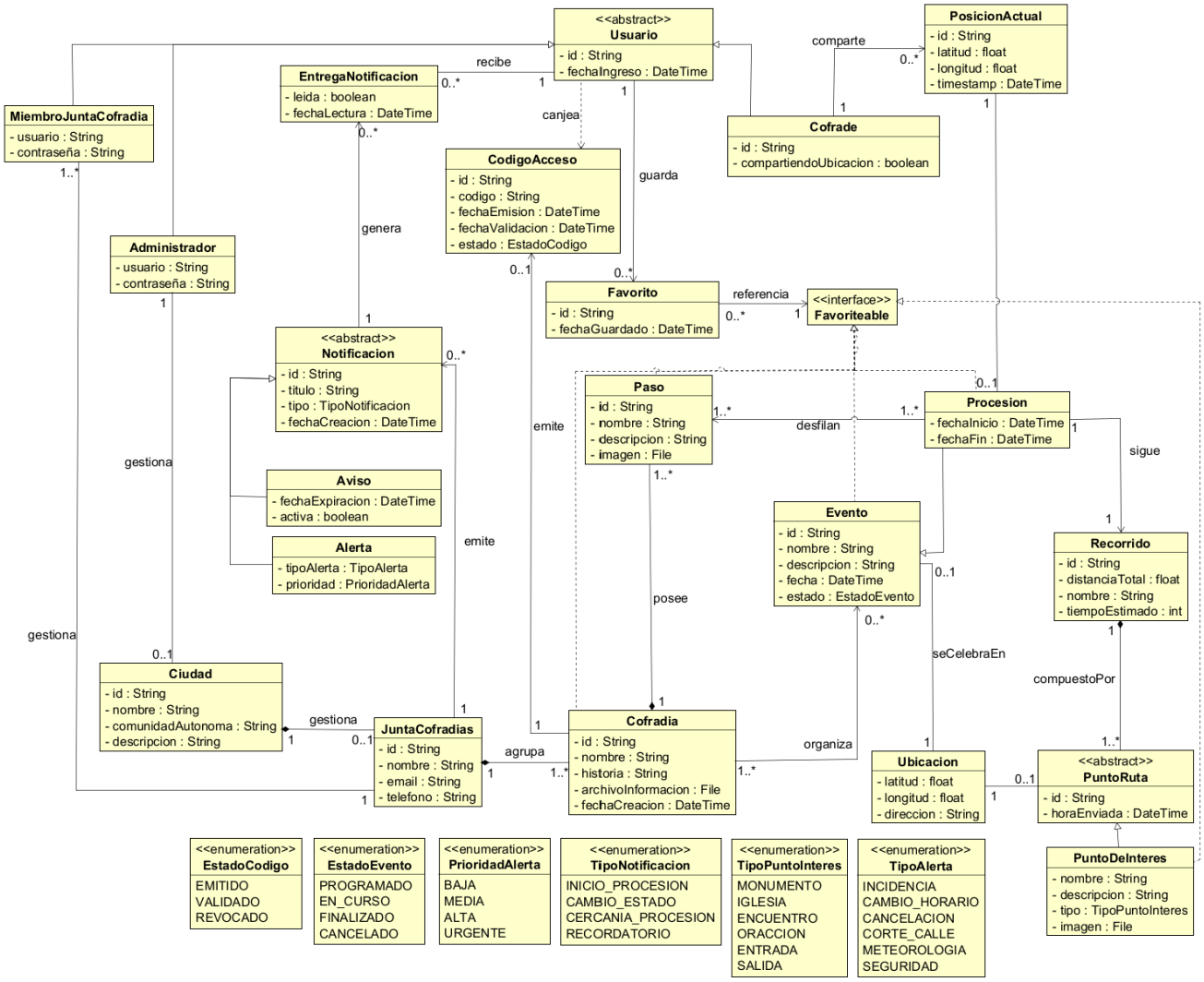


Figura 3.5: Diagrama de dominio del proyecto

Nota: la construcción del backend (Sección 4.1) obligó a revisar varias relaciones de este diagrama conforme se traducían a un modelo relacional; el texto de esta sección ya refleja esos cambios, pero la imagen de la Figura 3.5 está pendiente de redibujarse en consecuencia. Estoy pidiendo permisos para la licencia de Astah para poder actualizarla.

El modelo de dominio representa las principales entidades del sistema y las relaciones existentes entre ellas. La clase `Ciudad` agrupa las procesiones y actos propios de cada localidad, y agrupa también directamente las distintas `Cofradia` de esa ciudad; `JuntaCofradias` es quien gestiona esa ciudad (relación 1:1), pero no es ella quien agrupa a las cofradías, de forma que una renovación de la Junta no afecta a los datos de las cofradías que gestiona. Cada `Cofradia` puede participar en varios `Evento` y un mismo `Evento` puede tener más de una `Cofradia` participando –relación de muchos a muchos, pensada para actos conjuntos entre varias hermandades–, y posee uno o varios `Paso`, que pueden participar en varios eventos y procesiones distintos –no solo procesiones, "los eventos también pueden tener pasos"–, de ahí también la relación de muchos a muchos entre `Paso` y `Evento`, que `Procesion` hereda al extenderlo. `Procesion` **extiende** a `Evento` –hereda sus atributos y sus cofradías participantes, y añade únicamente su fecha de inicio y fin propias– en lugar de referenciarlo mediante una simple asociación, y tiene asocia-

do un Recorrido, compuesto a su vez por distintos PuntoRuta: un mismo punto puede formar parte de varios recorridos, por ejemplo cuando dos procesiones comparten un tramo de calle. PuntoDeInteres extiende a PuntoRuta añadiendo nombre, descripción, imagen y una categoría, para los puntos que merece la pena mostrar con detalle; Ubicacion ya no es un subtipo de PuntoRuta, sino una clase independiente y reutilizable (latitud, longitud y dirección) referenciada tanto por PuntoRuta como por Evento.

Para la geolocalización en tiempo real, un Cofrade autenticado con el código de acceso de su cofradía puede compartir su ubicación mientras sigue una procesión; cada lectura se registra como una PosicionActual, sin ninguna referencia a la persona que la envió. La posición que consultan el resto de usuarios de la aplicación no es ninguna PosicionActual guardada en concreto, sino el promedio calculado a partir de las lecturas más recientes de esa procesión.

La comunicación con los usuarios se modela mediante una única clase Notificacion –ya no hay una jerarquía Aviso/Alerta como se planteó inicialmente: al revisar qué tipos se usaban de verdad se vio que el cliente decide el color con el que mostrar cada notificación solo a partir de su prioridad, nunca de una distinción más fina entre subtipos, así que esa jerarquía no se estaba ganando su sitio frente a la complejidad de mantenerla–. Cada Notificacion pertenece a una Ciudad y tiene un tipo (INICIO, FIN, INCIDENCIA, CAMBIO_HORARIO o CANCELACION: los dos primeros los genera el propio sistema al cambiar el estado de un Evento, el resto los crea la Junta de Cofradías a mano) y, salvo en INICIO/FIN, una prioridad (BAJA, MEDIA o ALTA). Sin EntregaNotificacion: no se registra si un usuario concreto la ha leído o no –el ciudadano no tiene cuenta con la que asociar esa lectura–, simplemente se consulta la lista de notificaciones vigentes de la ciudad. Además de poder consultarse dentro de la aplicación, cada Notificacion se reenvía como notificación *push* a los dispositivos registrados en su ciudad (clase DispositivoPush, un token de Expo Push asociado a una Ciudad, sin ningún Usuario al que pertenecer).

Por último, el acceso al modo cofrade se controla mediante la clase CodigoAcceso: la Junta de Cofradías de la ciudad emite, cuando lo necesita, un código en nombre de una de sus cofradías –Cofradia no tiene cuenta propia con la que emitirlo ella misma–, y quien lo valida obtiene los privilegios de cofrade sin llegar a registrarse como Usuario. De forma similar, los elementos marcados como favoritos por un usuario (cofradías, eventos, procesiones o pasos) se representan mediante la clase Favorito, que hace referencia a cualquier elemento que implemente la interfaz Favoriteable.

3.4.1. Modelo de interacción

En esta sección se describen los principales casos de uso de la aplicación mediante diagramas de secuencia obtenidos a partir del análisis. Se han excluido las secuencias alternativas de cancelar con el fin de simplificar los diagramas.

Para el caso de uso *Visualizar procesión en tiempo real* (Figura 3.6), el ciudadano selecciona una procesión activa y el sistema obtiene la instancia de Procesion correspondiente, junto con su recorrido planificado, y se suscribe a las actualizaciones de su posición actual. Mientras la procesión está en curso, el sistema recalcula sobre el recorrido el tramo ya realizado y el tramo restante en función de la posición recibida, para mantener el mapa del usuario actualizado.

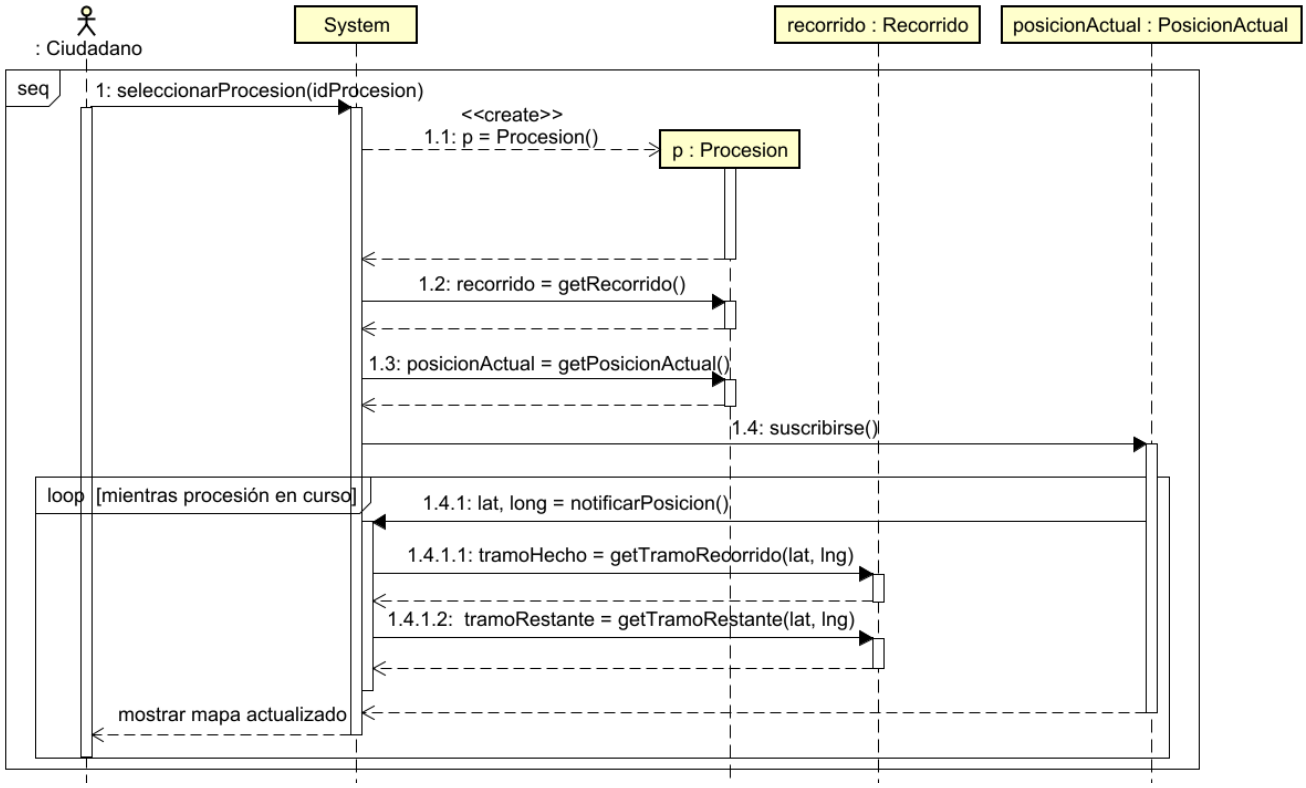


Figura 3.6: Diagrama de secuencia del CU “Visualizar procesión en tiempo real”

En el caso de uso *Compartir ubicación durante procesión* (Figura 3.7), el cofrade solicita compartir su ubicación en la procesión en curso de su cofradía. El sistema solicita los permisos de localización si no los tiene concedidos: si el cofrade los deniega, se le informa de que no es posible compartir la ubicación sin permiso; si los concede, el sistema comienza a actualizar de forma periódica la posición del cofrade mientras la procesión sigue en curso.

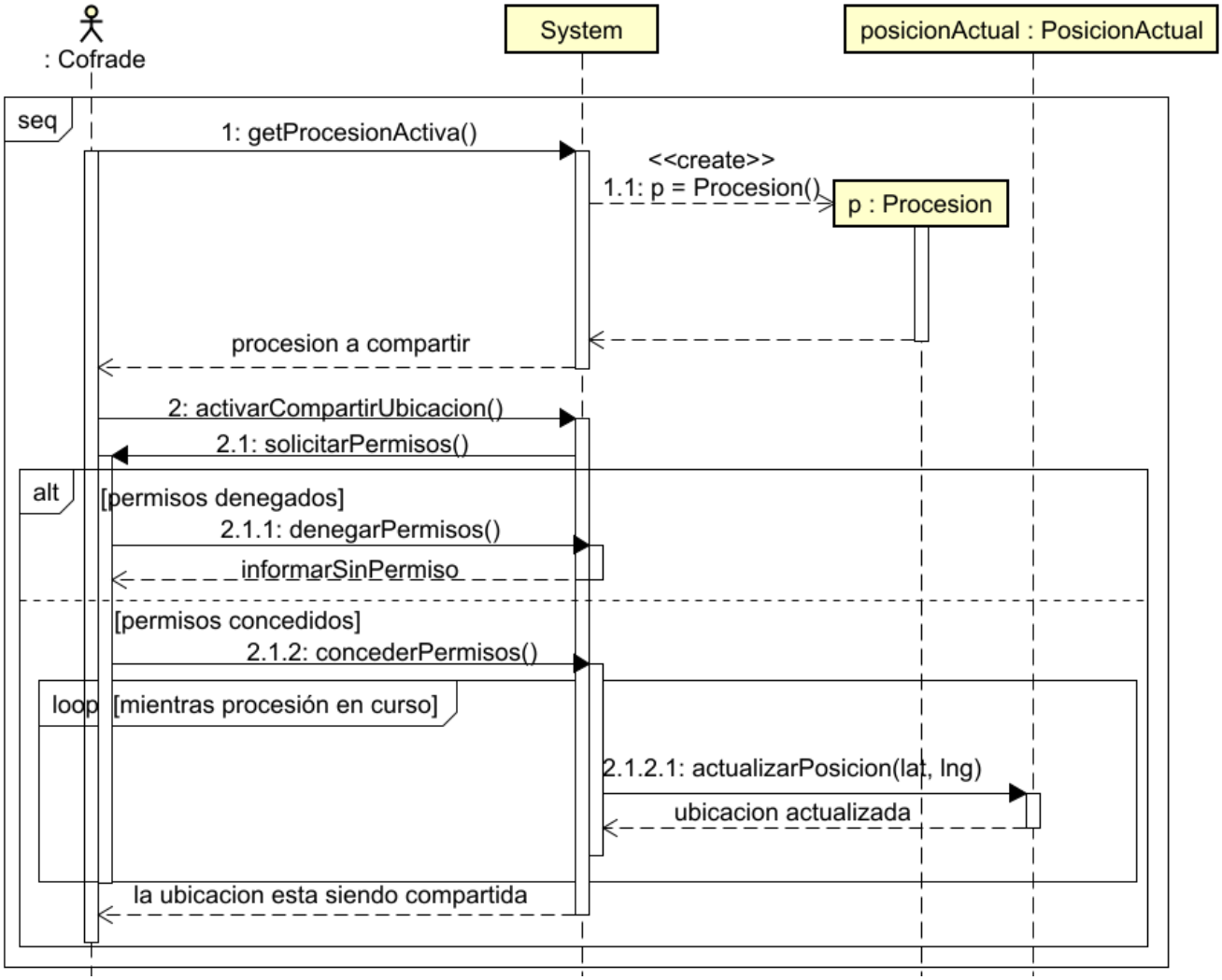


Figura 3.7: Diagrama de secuencia del CU “Compartir ubicación durante procesión”

En el caso de uso *Iniciar sesión con código de acceso* (Figura 3.8), el usuario solicita acceder como cofrade y el sistema le pide el código de acceso de su cofradía. Al introducirlo, el sistema obtiene el `CodigoAcceso` correspondiente y lo valida, informando al usuario del acceso correcto o de un error de validación según el resultado –sin que ello registre ningún `Usuario` nuevo (Sección B.10)–.

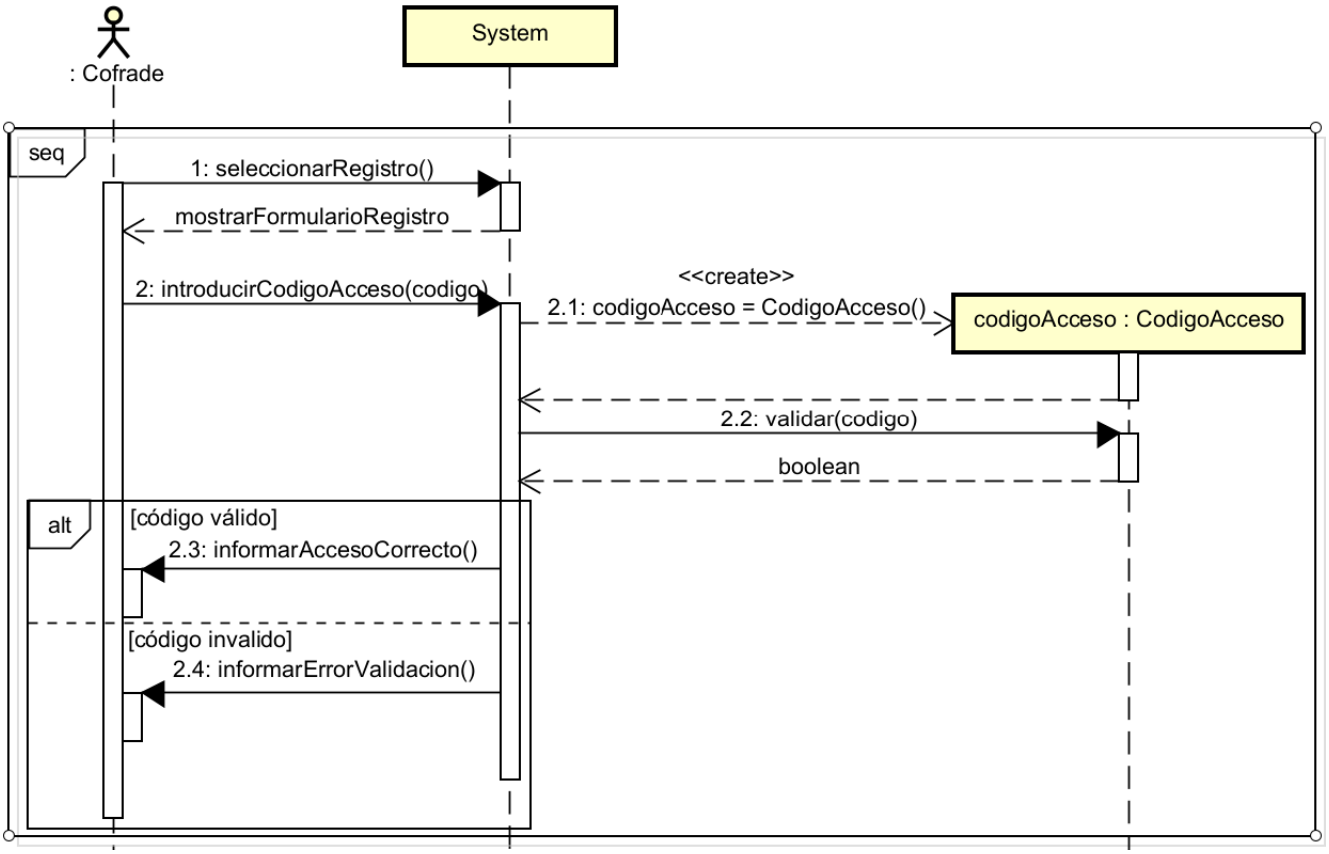


Figura 3.8: Diagrama de secuencia del CU “Iniciar sesión con código de acceso”

Nota: este diagrama todavía dibuja el flujo antiguo de registro del cofrade, ya descartado (el cofrade nunca se registra como `Usuario`). Estoy pidiendo permisos para la licencia de Astah para poder actualizarlo.

En el caso de uso *Gestionar una ciudad*, en su variante de alta (Figura 3.9), el administrador solicita gestionar las ciudades, el sistema le muestra las ciudades existentes y el administrador indica que quiere dar de alta una nueva. Tras introducir los datos, el sistema construye la instancia de `Ciudad` correspondiente y valida los datos introducidos, repitiendo la solicitud hasta que estos son válidos, momento en el que la ciudad queda dada de alta.

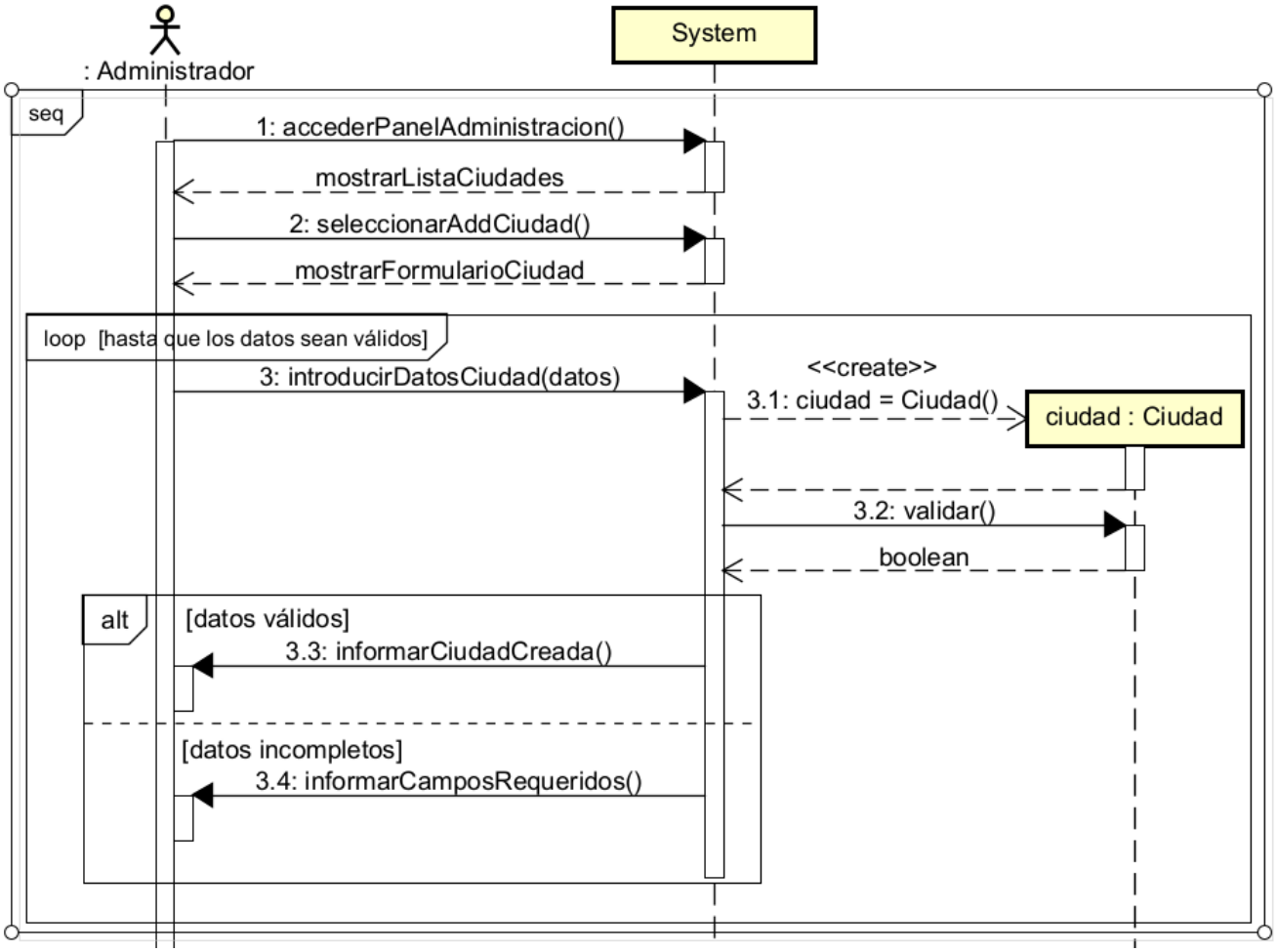


Figura 3.9: Diagrama de secuencia del CU “Gestionar una ciudad” (alta)

Capítulo 4

Descripción de las iteraciones

4.1. 1ª Iteración

4.1.1. Analisís

4.1.2. Diseño

Antes de comenzar la implementación de esta primera iteración ha sido necesario tomar una serie de decisiones de diseño que condicionan el resto del proyecto: qué tecnologías se van a usar, cómo se organiza el código, cómo se guardan los datos y qué soluciones ya conocidas (patrones de diseño) conviene aplicar. Estas decisiones se explican a continuación con el detalle suficiente para que puedan entenderse sin conocimientos previos de React Native o Firebase, ya que sientan la base para el resto de iteraciones del proyecto.

Decisiones tecnológicas

Para el cliente[1] móvil se ha optado por **React Native** [35], un *framework*[2] que permite escribir una única aplicación y ejecutarla tanto en Android como en iOS, en lugar de programar dos aplicaciones nativas independientes (una en Kotlin/Java para Android y otra en Swift para iOS). Se descarta el desarrollo nativo duplicado por dos motivos: el requisito RNF-04 exige que la aplicación funcione en ambas plataformas, y al tratarse de un proyecto desarrollado por una única persona, mantener y probar dos bases de código distintas multiplicaría el tiempo de desarrollo sin aportar ninguna ventaja funcional.

Dentro de React Native, el código se construye con **Expo** [30], siguiendo la recomendación de la propia documentación oficial de React Native, que aconseja apoyarse en un *framework* sobre React Native –en lugar de configurar manualmente el entorno nativo de cada plataforma– y señala a Ex-

[1]Cliente: en una arquitectura cliente-servidor, es la parte del sistema con la que interactúa directamente el usuario; en este caso, la propia aplicación instalada en el teléfono móvil.

[2]Framework: conjunto de herramientas y bibliotecas de código que facilitan el desarrollo de un tipo concreto de aplicación, proporcionando una estructura base sobre la que trabajar.

po como la vía más sencilla para empezar, especialmente para quien tiene poca experiencia previa en desarrollo móvil [35]. Expo proporciona, entre otras cosas, una biblioteca estándar de módulos nativos ya integrados (cámara, notificaciones, geolocalización), un proceso de compilación gestionado (*EAS Build*) que evita instalar y mantener Android Studio y Xcode de forma completa para generar el ejecutable, y la posibilidad de publicar actualizaciones del código JavaScript sin pasar de nuevo por las tiendas de aplicaciones. Por esta razón, para la geolocalización del dispositivo se usa concretamente `expo-location`, el módulo de la propia biblioteca estándar de Expo, en lugar de una alternativa externa como `react-native-geolocation-service`. Para la integración con mapas se ha previsto `react-native-maps`, compatible con proyectos Expo, aunque su integración queda pendiente para la Iteración 2 (Sección 7.1), junto con la propia pantalla de mapa en vivo.

Como *backend*[3] se ha optado por desarrollar una **API REST propia** con **Spring Boot** [36], en lugar de apoyarse en una plataforma *Backend-as-a-Service* (BaaS) como Firebase. Durante la fase de diseño de la primera iteración se planteó una plataforma BaaS como posible forma de arquitectura, ya que agiliza notablemente el desarrollo al resolver de fábrica aspectos como el modelo de datos, la autenticación o el control de acceso; sin embargo, precisamente por eso le quitaba al proyecto buena parte del aprendizaje de diseñar y justificar esas mismas decisiones por cuenta propia –en particular, el de realizar yo misma el modelado de la base de datos–, que son el núcleo de la formación de la mención de Ingeniería del Software cursada en este Grado: arquitectura en capas, modelado relacional de los datos, autenticación y autorización propias, y diseño de una API. Se ha optado por tanto por construir esa capa de backend desde cero. El planteamiento inicial reservaba servicios de terceros (Firebase) para lo que no aportaba valor académico programar desde cero –el almacenamiento de imágenes con **Firebase Storage** [37] y el envío de notificaciones *push* con **Firebase Cloud Messaging** [38]–, pero en la práctica ninguno de los dos ha terminado siendo necesario: las notificaciones *push* se resuelven con **Expo Push Notifications** (Capítulo **??**), ya que el cliente móvil corre sobre Expo y añadir Firebase habría supuesto un segundo proveedor de *push* con sus propias credenciales; y la imagen de un `Paso` se guarda como una URL de texto libre sin infraestructura de subida propia, tal y como se recoge como línea de trabajo futuro en el Apéndice B y en la Sección 7.1. El resultado es un backend propio sin ninguna dependencia de Firebase, lo que simplifica el despliegue al no requerir gestionar un proyecto ni unas credenciales adicionales.

La persistencia de los datos se apoya en **PostgreSQL** [39], un sistema gestor de bases de datos **relacional**, frente a alternativas NoSQL como MongoDB o Firestore. Se ha preferido un modelo relacional porque el dominio del proyecto (Sección 3) está compuesto por entidades fuertemente relacionadas entre sí –una ciudad agrupa cofradías, una cofradía organiza eventos y procesiones, una procesión sigue un recorrido compuesto por puntos de ruta, etc.– con relaciones que conviene que la propia base de datos garantice mediante claves foráneas y restricciones de integridad referencial, en lugar de resolverlas a mano en el código de la aplicación como sería necesario en un modelo documental. El acceso a la base de datos se realiza mediante **Spring Data JPA** [40] sobre **Hibernate** [41], un *ORM*[4] que evita escribir a mano la mayor parte de las consultas SQL.

[3] Backend: la parte del sistema que no ve el usuario directamente, encargada de almacenar los datos, gestionar la lógica de negocio y responder a las peticiones que le hace el cliente.

[4] ORM (*Object-Relational Mapping*): técnica que traduce automáticamente entre los objetos de un lenguaje de programación orientado a objetos (Java, en este caso) y las tablas de una base de datos relacional, evitando escribir manualmente la mayoría de las sentencias SQL.

La autenticación y autorización de la API se resuelve con **Spring Security** [42] y tokens **JWT** (*JSON Web Token*) [43]: al validar sus credenciales (obtenidas, como se explica en el apartado Diseño de la base de datos, al introducir un código de acceso válido), el usuario recibe un token firmado por el servidor que incluye su identificador y su rol; ese token se envía en cada petición posterior a un recurso protegido, y es el propio servidor quien lo valida y comprueba el rol antes de autorizar la operación. Se trata de un mecanismo sin estado (*stateless*): el servidor no necesita guardar la sesión de cada usuario, lo que simplifica el despliegue y la futura escalabilidad de la API. Este mecanismo cubre el requisito RNF-03 (control de acceso diferenciado por rol).

Para los mapas se ha elegido **Google Maps Platform** [1], ya justificado en el estado de la cuestión (Sección 1.3.2), por ser la solución de mapas más extendida y con mejor documentación; su integración, junto con la pantalla de mapa en vivo, queda pendiente para la Iteración 2 (Sección 7.1). La contingencia prevista ante el riesgo "Problemas con la integración de Google Maps" (Sección 2.3) sigue siendo migrar a OpenStreetMap [19] o Mapbox si en la práctica surgen problemas de cuota o de coste.

Por último, la posición en vivo de una procesión (RNF-05: actualización cada 30 segundos como mínimo) no exige por sí misma un mecanismo de *push* en tiempo real: se resuelve mediante una API REST convencional, en la que la aplicación del cofrade envía su posición con una petición `POST /procesiones/{id}/ubicacion` cada 30 segundos, y la aplicación del ciudadano consulta la posición agregada con una petición `GET /procesiones/{id}/ubicacion` con la misma cadencia. Esta solución es más sencilla de construir y de defender que montar un servidor de *sockets* propio (WebSockets o *Server-Sent Events*), y es suficiente para el requisito tal y como está definido; si en el futuro se quisiera ofrecer una actualización percibida como instantánea o escalar a un número de usuarios muy superior, esa migración se recoge como línea de trabajo futuro (Sección 7.1).

Arquitectura del sistema

La arquitectura de un sistema describe, a grandes rasgos, en qué piezas se divide y cómo se comunican entre sí. En este proyecto se ha definido una arquitectura **cliente-servidor con servidor propio**: la aplicación React Native (el cliente) se comunica mediante peticiones HTTPS[5] a la API REST propia, desarrollada con Spring Boot, que expone un conjunto de *endpoints*[6] organizados por recurso (`/ciudades`, `/cofradias`, `/procesiones`, `/usuarios`, `/notificaciones`, etc). Es la propia API quien centraliza toda la lógica de negocio y quien, y solo quien, se comunica con la base de datos PostgreSQL: en ningún caso el cliente accede directamente a la base de datos. El cliente no se comunica directamente con ningún otro servicio externo: recibe las notificaciones *push* a través del propio SDK[7] de `expo-notifications`, que obtiene un token de Expo Push y lo registra en la API propia (`POST /dispositivos-push`) –es la API, no el cliente, quien luego llama al servicio de Expo Push para enviar cada notificación–. La integración con el SDK de Google Maps Platform (visualización de mapas y cálculo de rutas) queda pendiente para la

[5]HTTPS: versión segura (cifrada) del protocolo HTTP, usado habitualmente para la comunicación entre aplicaciones y servicios en internet.

[6]Endpoint: dirección concreta de una API a la que el cliente envía una petición para realizar una operación determinada (por ejemplo, obtener el listado de procesiones de una ciudad).

[7]SDK (*Software Development Kit*): conjunto de herramientas que proporciona un servicio para que otras aplicaciones puedan integrarse con él fácilmente.

Iteración 2, como se explica más arriba.

Estructura del front-end Dentro del propio cliente, el código se organiza siguiendo una **arquitectura en capas** [11]: el proyecto se divide en varios grupos de carpetas (o *paquetes*), cada uno con una responsabilidad concreta, de forma que cada capa solo puede depender de la capa inmediatamente inferior y nunca al revés. Esta regla se sigue para que un cambio en una capa concreta afecte al menor número posible de partes del código, y para poder probar cada capa de forma aislada. Las cuatro capas definidas son:

- **UI** (`screens`, `components`, `navigation`): las pantallas que ve el usuario, agrupadas por su rol (ciudadano, cofrade, junta de cofradías, administrador), siguiendo los cuatro diagramas de casos de uso del Capítulo 3; los componentes visuales reutilizables entre pantallas; y la navegación entre pantallas.

- **Application** (`context`, `hooks`): el estado que se comparte entre varias pantallas a la vez, como la sesión del usuario (token JWT y rol activo), la ciudad seleccionada o si está compartiendo su ubicación en ese momento.

- **Data** (`services`, `models`): un servicio por cada tipo de información del dominio (`cofradiaService`, `procesionService`, `geoService`, `pushService`...) que es el único responsable de hablar con la API propia –mediante peticiones HTTP gestionadas y cacheadas con TanStack Query [44]–; además de los tipos de datos que describen cómo es cada objeto del dominio dentro de la aplicación.

- **Infrastructure** (`api`): la configuración e inicialización del cliente HTTP, es decir, el código mínimo necesario para que el resto de la aplicación pueda empezar a usarlo. Contiene el cliente HTTP (basado en `fetch`) configurado para incluir automáticamente el token JWT en cada petición y para redirigir al *login* si el servidor responde con un error de autenticación.

La Figura 4.1 muestra esta organización en forma de diagrama de paquetes, donde las flechas discontinuas indican qué paquete depende de (necesita) cuál.



Figura 4.1: Diagrama de paquetes de la aplicación cliente

Nota: este diagrama todavía muestra los paquetes `firebase` y `maps` dentro de `Infrastructure`, que no llegaron a usarse (solo existe `api`, Sección 4.1.2). Estoy pidiendo permisos para la licencia de Astah para poder actualizarlo.

Estructura del back-end A diferencia del planteamiento inicial basado en Firebase, el back-end de este proyecto sí incluye código propio, organizado también en capas siguiendo el patrón habitual de una API Spring Boot [11]:

- **Controller**: expone los *endpoints* REST de cada recurso, recibe las peticiones HTTP, delega la lógica en la capa de servicio y traduce el resultado a la respuesta HTTP correspondiente (código de estado y cuerpo en JSON).
- **Service**: contiene la lógica de negocio propiamente dicha –por ejemplo, comprobar que un código de acceso es válido y no ha sido ya utilizado antes de vincular a un usuario con una cofradía, o comprobar que quien solicita cancelar una procesión pertenece a la Junta de Cofradías de esa ciudad–. Es en esta capa donde se aplican las comprobaciones de autorización por rol (RNF-03) que antes resolvían las reglas de seguridad de Firestore.
- **Repository**: interfaces de Spring Data JPA, una por entidad, que abstraen el acceso a PostgreSQL y de las que Spring genera automáticamente la implementación.
- **Security**: configuración de Spring Security, el filtro que intercepta cada petición para validar el token JWT, y la generación de tokens en el proceso de autenticación.
- **DTO** (*Data Transfer Object*): clases que definen qué datos entran y salen exactamente de cada *endpoint*, evitando exponer directamente las entidades de persistencia y desacoplando el modelo interno de la base de datos del contrato público de la API.

Todas las entidades del dominio –`Ciudad`, `Cofradia`, `Paso`, `Evento`, `Procesion`, `Recorrido`, `PuntoRuta`, `Usuario`, `Notificacion`, `CodigoAcceso` y `DispositivoPush`– siguen esta misma estructura de cuatro capas, tal y como recoge la Figura 4.2. El esquema completo de la base de datos subyacente se detalla en el Apéndice B, y el razonamiento seguido para diseñarla se explica en el apartado Diseño de la base de datos más adelante en esta misma sección.

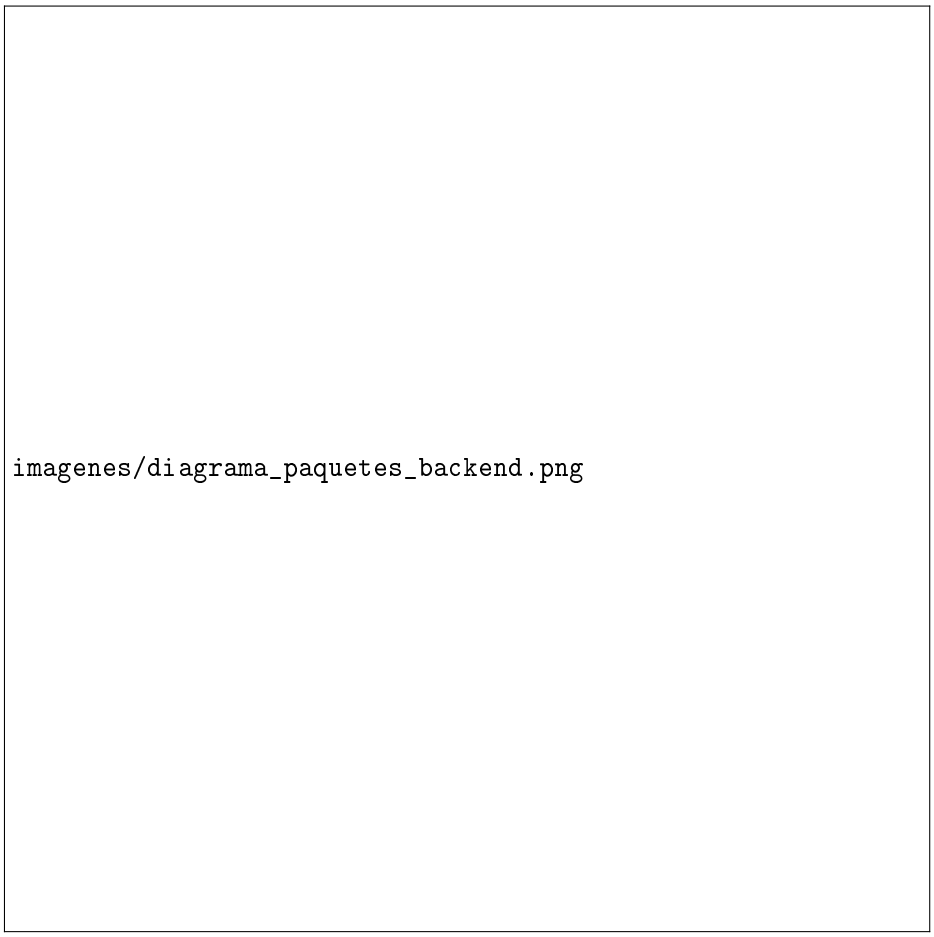


Figura 4.2: Diagrama de paquetes del backend propio

Nota: a este diagrama (y a la lista de entidades del párrafo anterior) le faltan `CodigoAcceso` y `DispositivoPush`, añadidas más adelante con su propio Controller/Service/Repository/DTO. Estoy pidiendo permisos para la licencia de Astah para poder actualizarlo.

Diagrama de despliegue Un diagrama de despliegue representa, de forma física, dónde se ejecuta cada parte del sistema y cómo se comunican entre sí esos entornos. La Figura 4.3 recoge el despliegue completo de este proyecto: el dispositivo móvil ejecuta la aplicación React Native, que se comunica únicamente con la API propia (Spring Boot, desplegada en el servidor virtual que proporciona la Escuela de Ingeniería Informática de la Universidad de Valladolid). Es la propia API quien, a su vez, es el único componente que accede a la base de datos PostgreSQL (alojada en ese mismo servidor) y quien dispara el envío de notificaciones *push* mediante una petición HTTP corriente al servicio de Expo Push (`exp.host`), sin depender de Firebase Cloud Messaging ni del Firebase Admin SDK. La integración con Google Maps Platform, para la visualización de mapas y el cálculo de rutas, queda pendiente para la Iteración 2, como

se explica en la Sección 4.1.2.

La disponibilidad de la información estática sin conexión a internet se resuelve en el cliente mediante TanStack Query [44], que persiste en el dispositivo la última respuesta recibida de cada *endpoint* y la sirve como datos disponibles mientras no hay conexión, revalidándola en segundo plano en cuanto la conexión se recupera.



Figura 4.3: Diagrama de despliegue de la aplicación

Diseño de la base de datos

A diferencia de una base de datos documental como Cloud Firestore, PostgreSQL [39] es una base de datos **relacional**: la información se organiza en *tablas* con filas y columnas, y las relaciones entre entidades se expresan mediante **claves foráneas**[8], que el propio motor de base de datos se encarga de validar. El modelo de dominio de la Figura 3.5 se ha traducido a este modelo relacional siguiendo estas decisiones:

- `pasos` es una tabla independiente con una clave foránea a su tabla "padre" (`cofradia_id`), en lugar de una subcolección anidada como en el planteamiento inicial con Firestore. Al tratarse de un modelo relacional, no es necesario un mecanismo especial para expresar que "siempre se consulta junto a su padre": basta con una consulta que una (*join*) ambas tablas por esa clave foránea.
- `puntos_ruta` no referencia directamente a un recorrido: durante la construcción del backend se detectó que un mismo punto (la misma iglesia, la misma plaza) puede formar parte de varios recorridos –varias procesiones pueden compartir tramo–, así que la relación con `recorridos` pasó de 1:N a N:M, resuelta con la tabla intermedia `recorrido_puntos_ruta`, que además guarda el orden y la hora prevista de paso *de esa relación concreta* y no del punto en sí (Apéndice B).

[8]Clave foránea (*foreign key*): columna de una tabla que referencia la clave primaria de otra tabla, y que la propia base de datos utiliza para garantizar que esa relación siempre apunta a un registro que realmente existe (integridad referencial).

- `ubicaciones` es una tabla nueva, sin equivalente directo en el modelo de dominio original: extrae a una tabla propia (latitud, longitud, dirección) el punto geográfico que necesitan tanto `eventos` como `puntos_ruta`, en lugar de repetir esas tres columnas en cada una. Al ser una tabla independiente, una misma ubicación –por ejemplo, la iglesia donde se celebra un evento y donde además hay un punto de interés del recorrido– puede reutilizarse desde varias entidades sin duplicar datos.

- `posiciones_actuales` se modela como una tabla independiente, en lugar de como columnas dentro de la propia tabla `procesiones` –el motivo es el mismo que en el planteamiento inicial: estas escrituras frecuentes de posición (cada 30 segundos) no deben afectar a los datos más estables de la procesión (horario, recorrido, pasos), que rara vez cambian–, pero a diferencia del planteamiento inicial ya no es una única fila por procesión que se sobrescribe con cada actualización (`UPSERT`), sino un **histórico de lecturas GPS sueltas** (*pings*), cada una **anónima** –sin ninguna referencia a la persona que la envió, ver el punto sobre `usuarios` más abajo–. La "posición actual" de una procesión deja de ser un dato guardado como tal y pasa a ser un valor **calculado** en el momento de la consulta: el promedio de los *pings* recibidos en los últimos 60 segundos. La aplicación cliente sigue consultando este valor mediante la API propia (`GET /procesiones/{id}/ubicacion`) con la cadencia que exige el requisito RNF-05, tal y como se explica en el apartado Decisiones tecnológicas.

- `favoritos` sigue sin guardarse en la base de datos, sino de forma local en el propio dispositivo (por ejemplo, con `AsyncStorage`), ya que el ciudadano no tiene cuenta ni sesión en la que persistir esta información en el servidor; únicamente se guarda el identificador de la cofradía, evento, procesión o paso marcado como favorito.

- `codigos_acceso` es una tabla con una clave foránea a la cofradía correspondiente. A diferencia del planteamiento inicial con Firestore, donde estas referencias debían gestionarse manualmente para evitar duplicar información y que se desactualizara, aquí es la propia base de datos –mediante las restricciones de clave foránea– quien garantiza que una referencia nunca pueda apuntar a un registro inexistente. Sin `notificaciones_entregadas`: a diferencia del planteamiento inicial, no se registra si un usuario concreto ha leído cada notificación –el ciudadano no tiene cuenta con la que asociar esa lectura (Sección B.10)–, simplemente se consulta la lista de notificaciones vigentes de su ciudad (Apéndice B).

- La tabla `usuarios` solo contiene los perfiles con credenciales propias –junta de cofradías y administrador–: el ciudadano sigue sin registrarse, y **el cofrade dejó de generar ninguna fila** durante la construcción del backend. Validar un código de acceso (tabla `codigos_acceso`, con estados `EMITIDO`, `VALIDADO` y `REVOCADO`, tal y como recoge el caso de uso "Iniciar sesión con código de acceso", Sección 3.4.1) ya no crea ninguna fila en `usuarios`: el servidor comprueba que el código no está revocado y devuelve directamente un token JWT identificado con la **cofradía** cuyo código se validó, no con una persona –el propio código de acceso actúa como credencial, sin que en ningún momento se le pida al cofrade establecer una contraseña–. Esta decisión evita cargar el sistema con el registro de personas cuya participación es, además, estacional. `JuntaCofradias` y `Administrador` sí tienen credenciales propias (`email` y una contraseña almacenada como *hash*[9] mediante BCrypt, el algoritmo utilizado por defecto por Spring Security [42]), y se comprueba su rol en la capa de servicio

[9] Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

de la API (*Service*, Sección 4.1) antes de autorizar cualquier operación protegida, cubriendo así el requisito RNF-03. Cada código de acceso lleva una **referencia a la cofradía** en cuyo nombre lo emite la Junta de esa ciudad, de forma que un cofrade que lo valida puede compartir su ubicación en tiempo real durante las procesiones en las que participa esa cofradía, que es la única acción que el modelo de dominio le permite realizar directamente sobre el sistema (Figura 3.5).

- Tanto `usuarios` como `eventos` son, además, la raíz de una **herencia real de tablas** (estrategia `JOINED` de JPA), no solo una traducción directa del modelo de dominio: `procesiones` extiende a `eventos` y `puntos_de_interes` extiende a `puntos_ruta` de la misma forma en que `miembros_junta_cofradia` y `administradores` extienden a `usuarios` –cada subtipo en su propia tabla, cuyo `id` es a la vez su clave primaria y una clave foránea a la tabla raíz–, en lugar de una única tabla con muchas columnas que quedan nulas según el caso, como habría ocurrido de traducir la herencia del modelo de dominio de la forma más directa posible.

- Las relaciones N:M del modelo de dominio –pasos que participan en varios eventos (y las procesiones que los extienden), y cofradías que participan en varios eventos– se resuelven mediante **tablas intermedias** (`pasos_eventos` y `eventos_cofradias`), la forma habitual de modelar relaciones muchos-a-muchos en un esquema relacional.

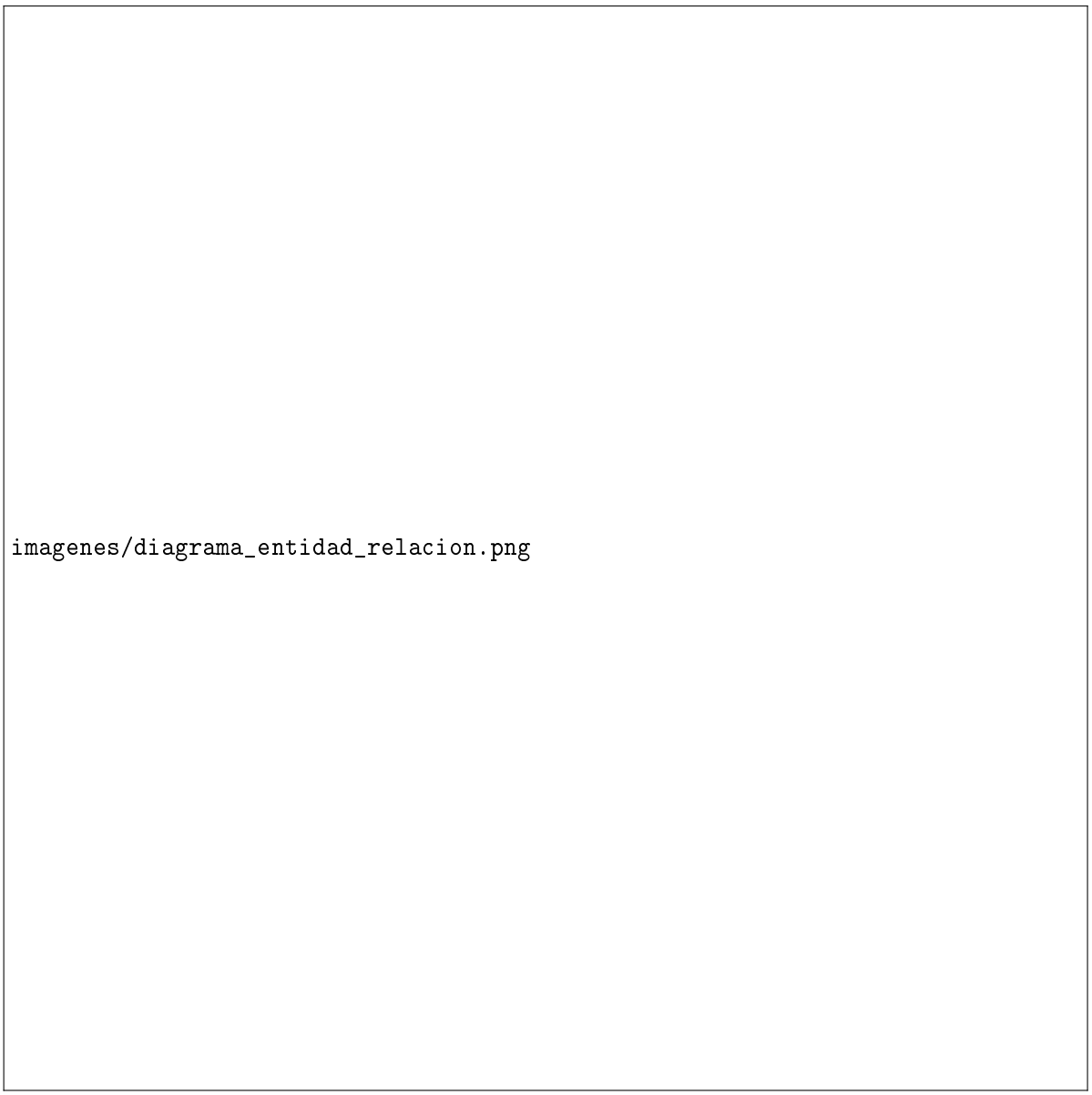


Figura 4.4: Diagrama entidad-relación de la base de datos

Nota: este diagrama está desactualizado respecto al esquema real (Apéndice B): entre otras cosas, no tiene tabla `dispositivos_push`, sigue llamándose `pasos_procesiones` en vez de `pasos_eventos`, y le faltan columnas añadidas más adelante (`activa` en `ciudades` y `cofradias`, `latitud`/`longitud` en `ciudades`, y varias en `miembros_junta_cofradia`). Estoy pidiendo permisos para la licencia de Astah para poder actualizarlo.

El Apéndice B recoge, a nivel de campo, el esquema completo de cada tabla –tipos de dato, claves primarias y foráneas incluidas– para quien necesite el detalle completo de la base de datos.

Interfaces de usuario

Paralelamente a las decisiones de arquitectura y de base de datos, se ha diseñado en Figma [45] mediante *mockups*[10] el conjunto de pantallas correspondiente al alcance de esta primera iteración según la planificación (Sección 2.2): las funcionalidades fundamentales del perfil de ciudadano, es decir, la visualización de la información general de la Semana Santa, el listado de organizaciones y la consulta de eventos. Siguiendo ese mismo criterio, el resto de pantallas de la aplicación –el mapa en vivo, el modo cofrade y la ubicación compartida del perfil, y el panel de la Junta de Cofradías y el de administrador– se diseñan en la iteración a la que corresponden según la planificación, y se documentan en el apartado de Diseño de esa misma iteración.

Antes de entrar en el detalle de cada pantalla, esta primera iteración fija también los criterios visuales comunes a toda la aplicación, para que el resto de iteraciones los reutilicen sin tener que redefinirlos: una estética oscura con acentos dorados, que evoca la iluminación de cirios y el ambiente nocturno característico de las procesiones, y una barra de navegación inferior fija con cinco secciones (*Inicio*, *Calendario*, *Mapa*, *Buscar* y *Perfil*), que se corresponde directamente con el paquete `navigation/` descrito en el apartado de arquitectura. Los listados de cofradías, procesiones, pasos y eventos ya no ocupan una sección propia de la barra de navegación, sino que se accede a ellos desde un menú contextual en la pantalla de *Inicio*, tal y como se explica más adelante.

La paleta de color se apoya en tonos marrón muy oscuro, casi negro, para el fondo (`#1E1008`, `#120A06`, `#34150E`, `#472F17`), dorados como color de acento para títulos, iconos activos y elementos interactivos (`#C9A84C`, `#9A7D60`), tonos crema para tarjetas y textos secundarios (`#F0E0C8`, `#E3D595`) y blanco (`#FFFFFF`) para el texto principal sobre fondo oscuro; un verde puntual (`#37E37E`) se reserva para indicadores de estado, como una procesión en curso. En cuanto a la tipografía, se combinan tres familias: **Cormorant Garamond**, una serifa elegante, para los títulos, coherente con la estética evocada; **Lora**, también serifa pero más neutra, para el texto de lectura extensa (historia de una cofradía, descripción de un paso); y **Sora**, de palo seco, para los elementos de interfaz (botones, etiquetas, barra de navegación), donde interesa más la legibilidad a tamaños pequeños que el carácter estético. Estas decisiones se apoyan, además del criterio propio, en las pautas de *Material Design 3* [46], la guía de diseño de interfaz de Google, en particular en lo relativo al contraste de color, el tamaño mínimo táctil de los elementos interactivos y la estructura de la barra de navegación inferior.

Selección de ciudad Es la primera pantalla que ve cualquier usuario, ya que toda la información de la aplicación (cofradías, procesiones, eventos) está filtrada por la ciudad seleccionada, tal y como se refleja en la colección `ciudades` descrita en el apartado anterior. El usuario puede buscar su ciudad o elegirla de una lista con el número de procesiones disponibles en cada una.

[10] Mockup: representación visual estática de la interfaz de una aplicación, sin funcionalidad real, utilizada para planificar el diseño antes de su implementación.



Figura 4.5: Mockup de la pantalla de selección de ciudad

Inicio Muestra la información general del día de Semana Santa seleccionado: una notificación destaca si existe alguna incidencia relevante (por ejemplo, un retraso en el horario de salida), la procesión que está en curso en ese momento si la hay, unas estadísticas generales de la ciudad (número de cofradías, procesiones y nazarenos) y el listado de procesiones y eventos programados para el día, marcando cuáles son favoritos del ciudadano. Desde el menú contextual de esta pantalla (icono *overflow* superior derecho) se accede, además, a los listados independientes de cofradías, procesiones, pasos y eventos.

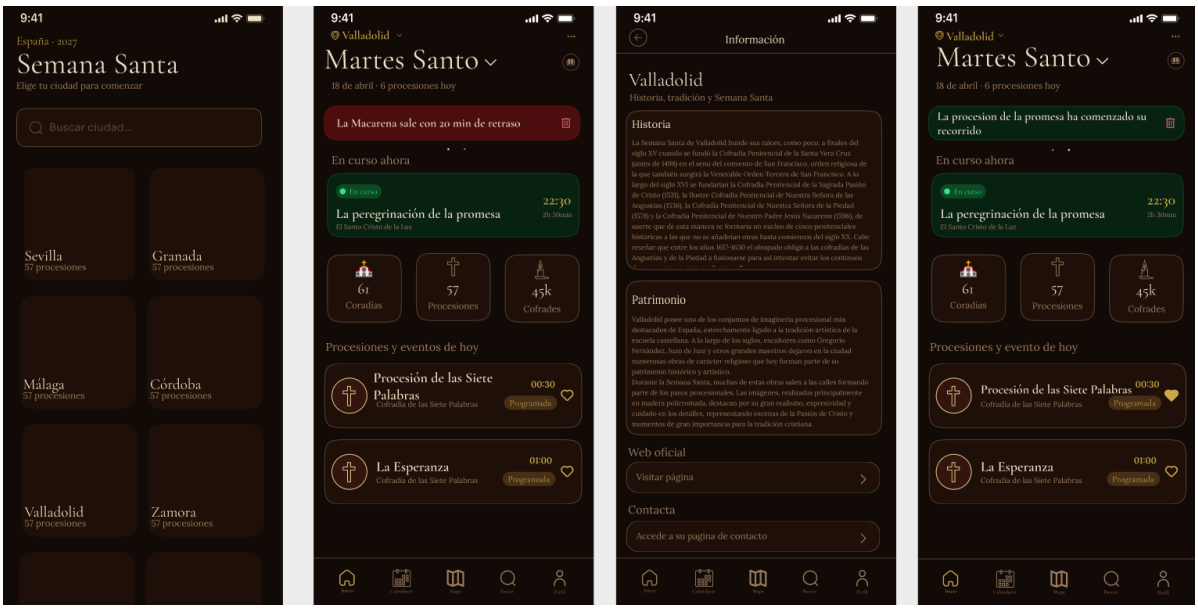


Figura 4.6: Mockup de la pantalla de inicio: notificación de retraso, procesión en curso, estadísticas y agenda del día

Calendario Complementa a la pantalla de inicio permitiendo consultar cualquier otro día de la Semana Santa, no solo el actual. Ofrece dos niveles de detalle: una vista mensual completa, con un indicador en los días que tienen procesiones programadas, y una vista semanal con la agenda del día seleccionado, listando sus procesiones con horario y duración.



Figura 4.7: Mockup de la pantalla de calendario: vista mensual y agenda del día seleccionado

Listados de cofradías, procesiones, pasos y eventos Cada uno de estos cuatro elementos del dominio cuenta con su propio listado, accesible desde el menú de la pantalla de Inicio, con buscador propio y sin más información que el nombre de cada elemento, ya que su detalle se consulta accediendo a la ficha correspondiente.

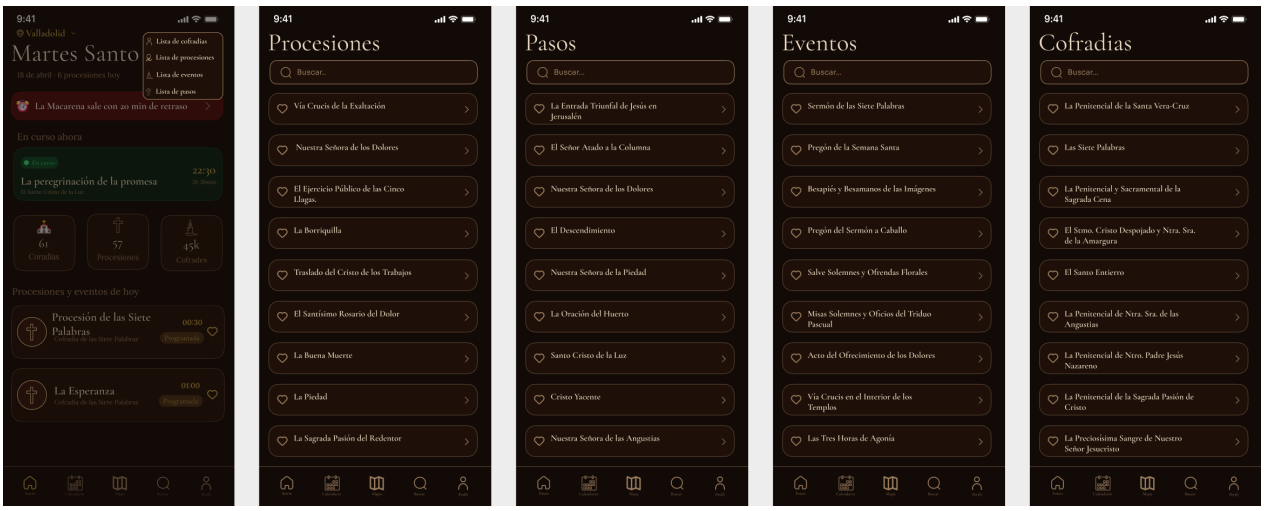


Figura 4.8: Mockup del menú de la pantalla de inicio y de los listados de procesiones, pasos, eventos y cofradías

Detalle de cofradía y de paso La ficha de una cofradía muestra su historia, su web oficial, las procesiones y el evento que organiza y los pasos que la componen. De forma análoga, la ficha de un paso muestra una imagen, su historia y origen, un análisis artístico y su web oficial, siguiendo la misma estructura visual que la ficha de cofradía para mantener la coherencia del conjunto.

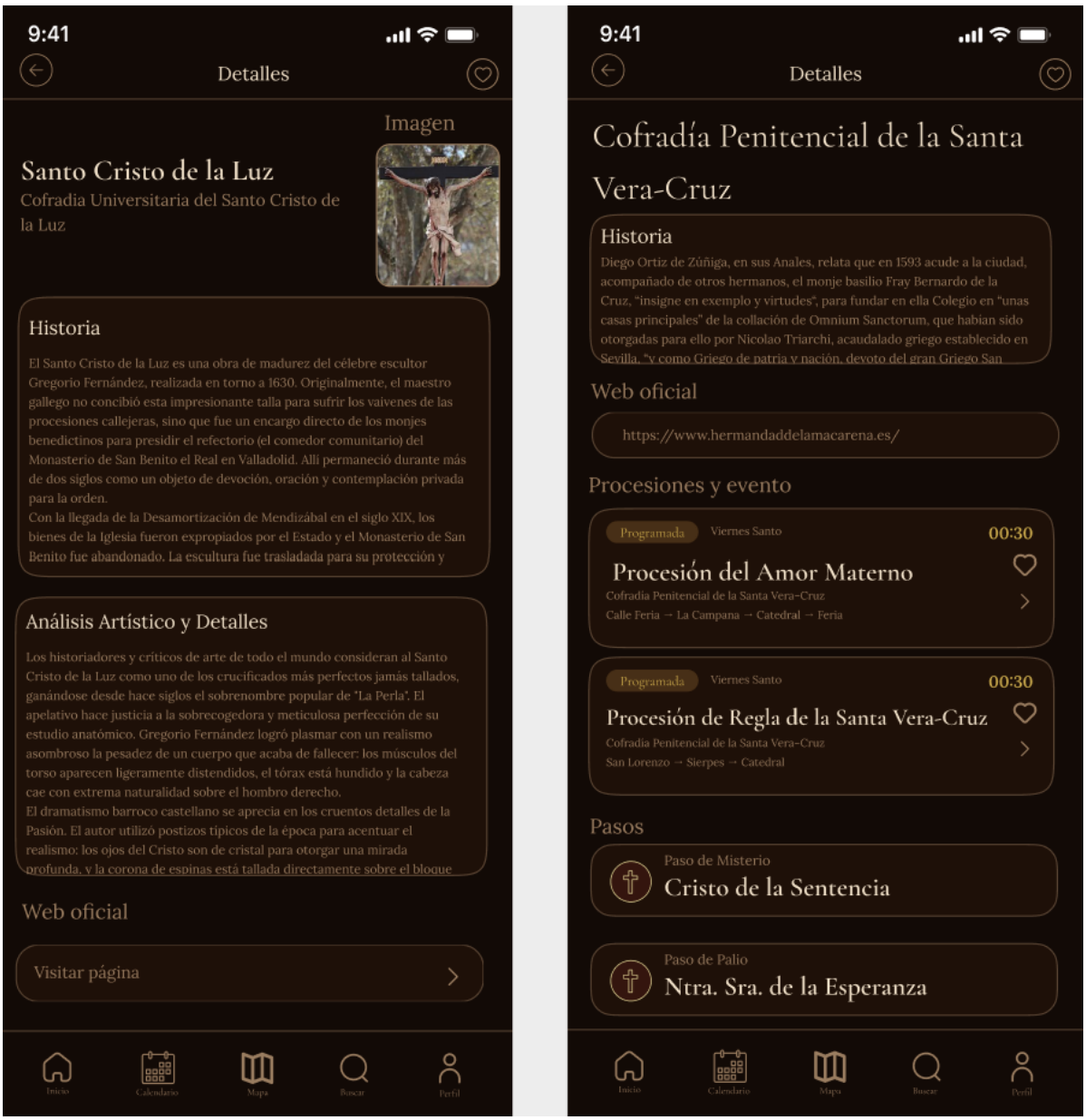


Figura 4.9: Mockup del detalle de un paso y de una cofradía

Detalle de evento La ficha de un evento muestra su nombre, día y hora, duración, ubicación y los pasos que participan en él, siguiendo la misma estructura visual que la ficha de cofradía y de paso.



Figura 4.10: Mockup del detalle de un evento

Detalle de procesión Muestra el estado de la procesión (programada, en curso o finalizada), su día y hora de salida, duración, número de nazarenos, los pasos que participan en ella y su recorrido sobre un mapa, con un botón de acceso directo al mapa en vivo si la procesión está en curso. Además, de forma similar a la ficha de una cofradía, cada procesión tiene una pantalla de información ampliada con la historia y el origen del desfile.

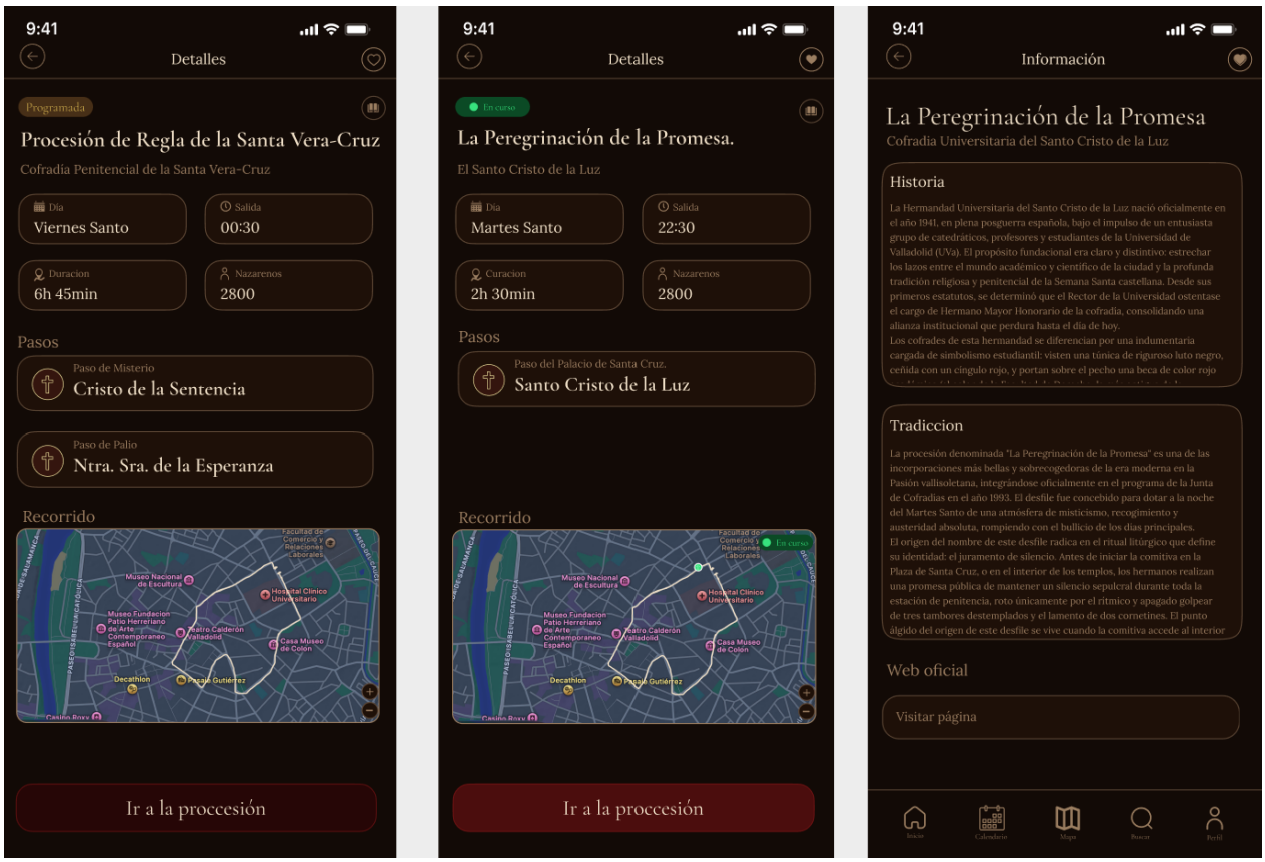


Figura 4.11: Mockup del detalle de una procesión (programada y en curso) y de su información ampliada

Buscar Permite localizar procesiones, pasos y eventos combinando texto libre con filtros por día de Semana Santa y por estado (programada, en curso o finalizada), mostrando un mensaje cuando ningún resultado cumple los filtros seleccionados. Su funcionalidad se implementa en una iteración posterior.

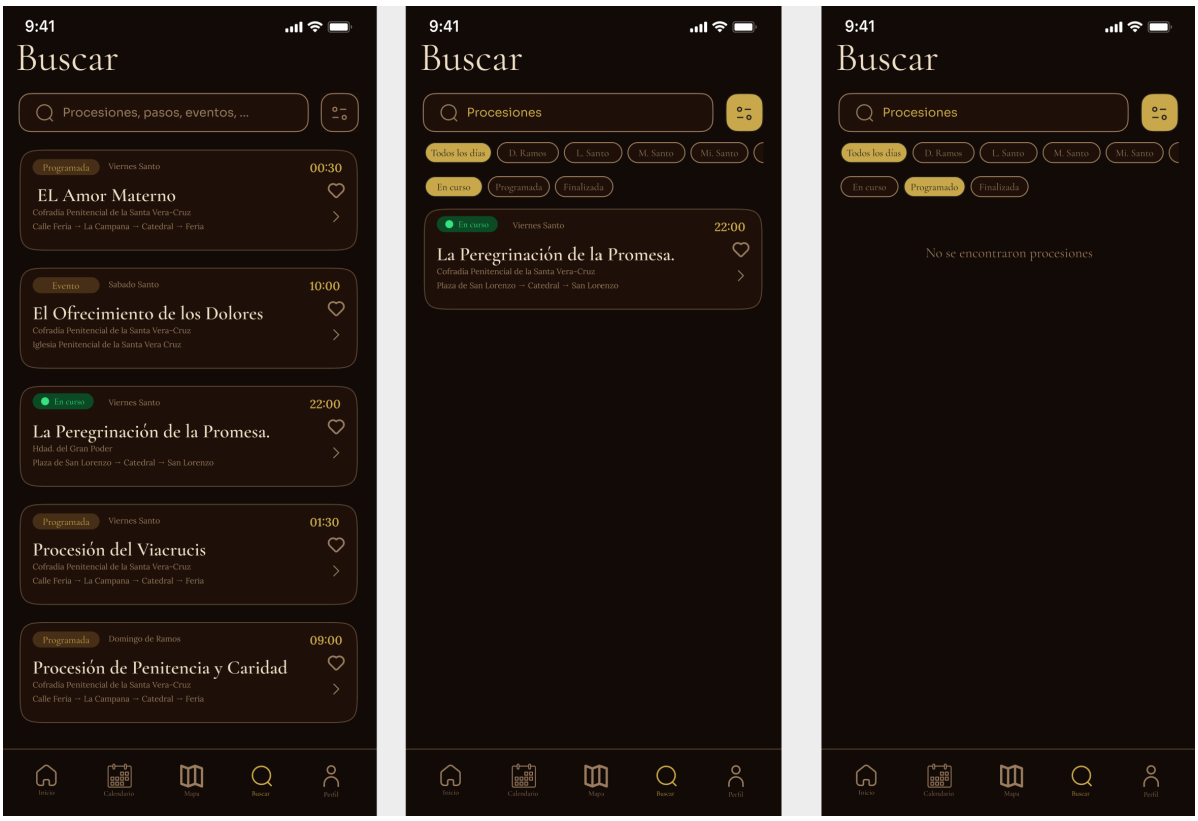


Figura 4.12: Mockup de la pantalla de búsqueda con filtros por día y estado

Patrones de diseño

Un patrón de diseño [15] es una solución ya conocida y reutilizable a un problema que se repite con frecuencia en el desarrollo de software; su uso facilita la legibilidad, el mantenimiento y la escalabilidad del código. Para seleccionar los patrones aplicables a este proyecto se han revisado dos TFG previos de la escuela con una arquitectura similar (aplicación móvil apoyada en Firebase), de los que se han descartado los patrones *Builder* y *Abstract Factory*: en Java se justifican porque los constructores de clases no admiten bien parámetros opcionales, pero en JavaScript/TypeScript un objeto literal con esos mismos campos opcionales ya resuelve el mismo problema de forma nativa, y en ningún punto del sistema es necesario intercambiar familias completas de implementaciones. Los seis patrones finalmente aplicados son los siguientes:

- **Singleton**: patrón que garantiza que una clase tenga una única instancia en toda la aplicación y un punto de acceso global a ella, evitando crear varias copias innecesarias. En este proyecto, el cliente HTTP de la API propia (`api/client`, configurado una única vez con el interceptor que añade el token JWT a cada petición) se instancia una sola vez y esa misma instancia se reutiliza en el resto de la aplicación.
- **Repository**: patrón que separa la lógica de acceso a los datos del resto de la aplicación, ocultando los detalles de cómo y dónde se almacenan. En el cliente, la capa `services` es la única parte del código que conoce el cliente HTTP de la API propia; el resto de la aplicación solo invoca funciones

como `procesionService.obtenerActivas(ciudadId)`, sin saber que por detrás se está llamando a un *endpoint* REST respaldado por PostgreSQL. En el backend, este mismo patrón aparece de forma directa en la capa `repository` (Sección 4.1): las interfaces de Spring Data JPA son, de hecho, una implementación literal del patrón Repository, no solo una analogía. Esta doble capa facilitaría cambiar de proveedor de base de datos en el futuro sin modificar ninguna pantalla ni ningún controlador.

- **Observer**: patrón en el que uno o varios elementos ("observadores") se suscriben a los cambios de otro elemento y son notificados automáticamente cuando este cambia, en lugar de tener que preguntar repetidamente. En este proyecto se aplica a la recepción de notificaciones *push*: el cliente se registra como observador de las notificaciones entrantes (`Notifications.setNotificationHandler` de Expo Notifications) y es notificado automáticamente cada vez que llega una, sin necesidad de preguntar al servidor si hay algo nuevo -el backend, por su parte, delega la entrega en el servicio de *push* de Expo (`exp.host`), sin hablar directamente con Firebase Cloud Messaging ni con APNs. No se aplica, en cambio, a la posición en vivo de una procesión (RNF-05), que se resuelve mediante consultas periódicas (*polling*) a la API cada 30 segundos (Sección 4.1): al ser el cliente quien pregunta activamente en lugar de ser notificado, este mecanismo es la operación inversa a Observer y no debe describirse como tal.

- **Context/Provider**: convención propia de React (no un patrón GoF[11] clásico) que permite exponer un dato o estado a cualquier pantalla de la aplicación sin tener que pasarlo manualmente de componente en componente. `AuthContext` y `LocationContext` exponen así el usuario autenticado (y su token JWT) y los permisos de ubicación a cualquier pantalla que los necesite.

- **Facade**: patrón que ofrece una única función sencilla como punto de entrada a una operación que, por detrás, involucra varios pasos o subsistemas distintos. El caso de uso "Compartir ubicación durante procesión" (RF-24) esconde tras la llamada `geoService.iniciarCompartirUbicacion(procesionId)` la solicitud de permiso de GPS, el envío periódico (`POST`) de la posición a la API y la gestión de una posible pérdida de conexión.

- **Adapter**: patrón que convierte el formato de datos de un sistema externo al formato interno que espera la aplicación, permitiendo que ambos "encajen" sin modificar ninguno de los dos. Al importar el recorrido de una procesión, el backend recibe un archivo GPX –formato estándar que exportan herramientas externas como gpx.studio, Wikiloc o Strava–, ajeno al modelo de dominio; `GpxParser` lo interpreta y lo convierte en la secuencia ordenada de puntos (latitud, longitud) que espera `RecorridoService` para construir el `Recorrido`, sin que ninguna de las dos partes -el archivo externo ni el modelo interno- tenga que adaptarse a la otra.

4.1.3. Implementacion

Se ha implementado el backend propio descrito en el apartado de diseño anterior: un proyecto Spring Boot [36] generado con Spring Initializr, organizado en los paquetes `model`, `repository`, `service`,

[11] GoF (*Gang of Four*): apodo con el que se conoce a los cuatro autores del libro *Design Patterns* [47] (1994), que popularizó el catálogo clásico de patrones de diseño orientado a objetos (Singleton, Observer, Factory Method, entre otros).

`controller`, `dto`, `security` y `exception` que siguen la estructura de capas ya justificada, y con el esquema de PostgreSQL gestionado mediante migraciones versionadas de Flyway [**?**] (un script SQL numerado por cada cambio de esquema), en lugar de dejar que Hibernate genere o modifique las tablas automáticamente; así el esquema queda documentado paso a paso y se aplica igual en local, en el servidor de despliegue y en cualquier entorno nuevo, tal y como recoge el manual de despliegue (Apéndice C).

Sobre esa base se han implementado las cuatro capas para todas las entidades necesarias para el módulo de ciudadano de esta primera iteración –`Ciudad`, `JuntaCofradias`, `Cofradia`, `Paso` y `Evento`–, cada una con su correspondiente conjunto de *endpoints* REST de consulta y gestión (`GET`, `POST`, `PUT`, `DELETE`), documentados automáticamente en `/swagger-ui.html` mediante springdoc-openapi [48]. `JuntaCofradias` se implementa en esta misma iteración por ser, igual que `Ciudad`, un dato base del que depende `Cofradia` (Apéndice B), aunque su gestión efectiva –dar de alta, editar o dar de baja una Junta– corresponda al panel de administrador de la Iteración 5.

Durante esta implementación se han detectado varias necesidades no recogidas en el diseño inicial de la base de datos, que se han ido resolviendo sobre la marcha y quedan documentadas con su justificación en el Apéndice B: entre ellas, que la información de una cofradía y de un paso necesitaba más campos de los previstos inicialmente (`web` en lugar de un archivo adjunto, `analisis_artistico` para el paso), o que `Cofradia` debía depender de `Ciudad` y no de `JuntaCofradias`, para que la información de las cofradías de una ciudad no se viera afectada por una renovación de su Junta.

Queda pendiente para el cierre de esta iteración la implementación de las pantallas de cliente correspondientes sobre esta API, que hasta ahora consumían datos de prueba (*mock*).

4.1.4. Pruebas

En esta fase del proyecto la verificación del backend ha sido manual: compilación exitosa tras cada cambio (`mvn compile`) y comprobación funcional de los *endpoints* mediante la documentación interactiva de springdoc-openapi (`/swagger-ui.html`) contra una instancia de PostgreSQL local levantada con Docker. Queda pendiente una batería de pruebas automatizadas (pruebas unitarias de la capa `service` y pruebas de integración de los controladores con `Spring Boot Test`), que se abordará más adelante y se documentará en el Capítulo 6.

4.2. 2ª Iteración

4.2.1. Analisís

4.2.2. Diseño

Interfaces de usuario

Según la planificación (Sección 2.2), esta segunda iteración se centra en el sistema de mapas, por lo que la única pantalla nueva que se diseña en Figma [45] en esta iteración es el mapa en vivo, apoyada en los mismos criterios visuales (color, tipografía y navegación) ya fijados en la Iteración 1 (Sección 4.1).

Mapa en Vivo Muestra sobre un mapa de Google Maps la posición en tiempo real de las procesiones en curso y la ubicación del propio ciudadano, junto con un listado de las procesiones actualmente en movimiento desde el que centrar el mapa sobre una de ellas.

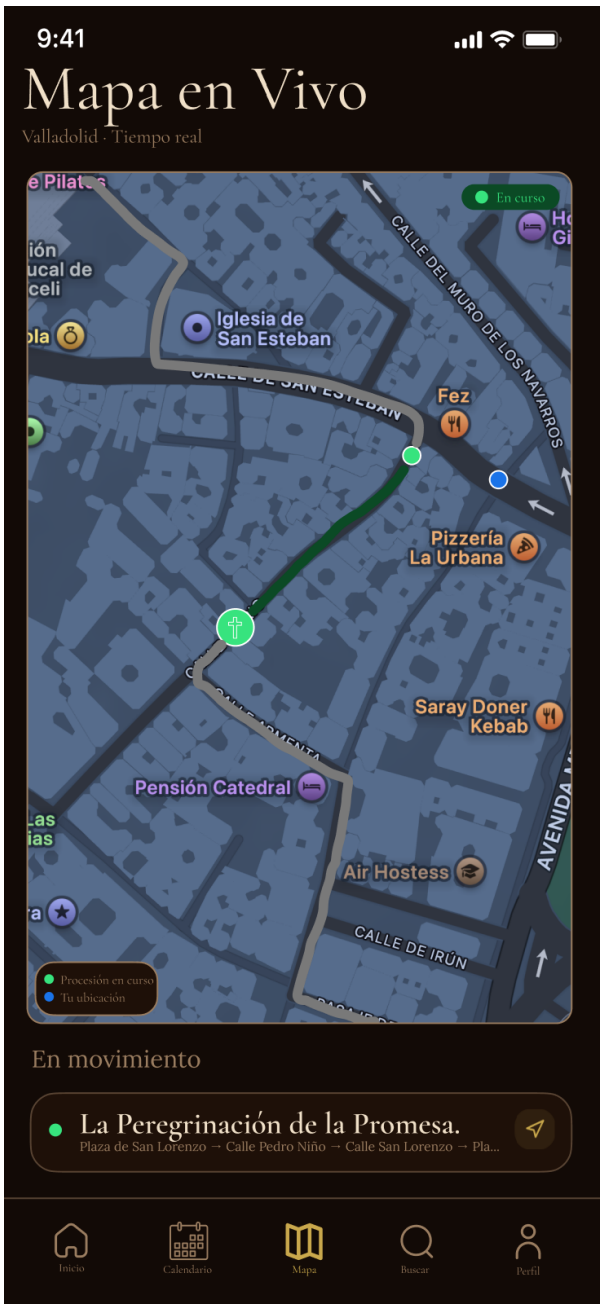


Figura 4.13: Mockup del mapa en vivo

4.2.3. Implementacion

Esta segunda iteración implementa en el backend las entidades necesarias para el sistema de mapas: `Recorrido`, `Ubicacion` y `PuntoRuta`, siguiendo la misma estructura de cuatro capas ya establecida en la Iteración 1 y documentada a nivel de campo en el Apéndice B.

`Ubicacion` es una tabla nueva, sin equivalente directo en el modelo de dominio original (Figura 3.5): agrupa en un único lugar la latitud, la longitud y la dirección que necesitan tanto `Evento` como `PuntoRuta`, para poder reutilizar el mismo punto geográfico desde varias entidades sin duplicar datos –por ejemplo, la sede de una cofradía y un punto de interés del recorrido que coincidan en el mismo

templo–.

`PuntoRuta` es, además, la raíz de una jerarquía de herencia real de tablas (estrategia `JOINED` de JPA): un punto "de paso" simple se guarda directamente en `puntos_ruta`, mientras que un punto "de interés", con nombre, descripción, imagen y una categoría, extiende esa tabla en `PuntoDeInteres`. Durante esta implementación se detectó, además, que un mismo punto de ruta puede formar parte de **varios** recorridos –varias procesiones pueden compartir un mismo tramo de calle–, así que la relación entre `Recorrido` y `PuntoRuta` se resolvió como una relación N:M mediante la tabla intermedia `recorrido_puntos_ruta`, que además guarda el orden del punto y su hora prevista de paso *en ese recorrido concreto*, ya que un mismo punto puede ocupar posiciones y horas distintas según el recorrido del que forme parte.

4.2.4. Pruebas

Al igual que en la Iteración 1, la verificación de estas entidades se ha realizado de forma manual mediante la documentación interactiva de springdoc-openapi (`/swagger-ui.html`). La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.3. 3ª Iteración

4.3.1. Analisís

4.3.2. Diseño

Interfaces de usuario

Según la planificación (Sección 2.2), esta tercera iteración cubre la geolocalización en tiempo real y el perfil de cofrade, por lo que la pantalla que se diseña en Figma [45] en esta iteración es el perfil, ya con su variante de modo cofrade, sobre los mismos criterios visuales fijados en la Iteración 1 (Sección 4.1).

Perfil Reúne la ciudad seleccionada y las cofradías, eventos, procesiones o pasos favoritos del ciudadano, el cambio entre el modo ciudadano y el modo cofrade, y un apartado de preferencias de notificaciones por tipo (procesiones en curso, incidencias). En modo cofrade se añade además el botón para activar la ubicación compartida durante una procesión. Ese apartado de preferencias por tipo, tal y como estaba planteado en el mockup, se descartó más adelante (RF-23): un interruptor propio dentro de la aplicación sería redundante con el permiso de notificaciones del propio dispositivo, que ya cubre la misma necesidad de forma más simple; a día de hoy el bloque sigue presente en la pantalla de perfil, pero es una maqueta visual sin conexión con el backend.

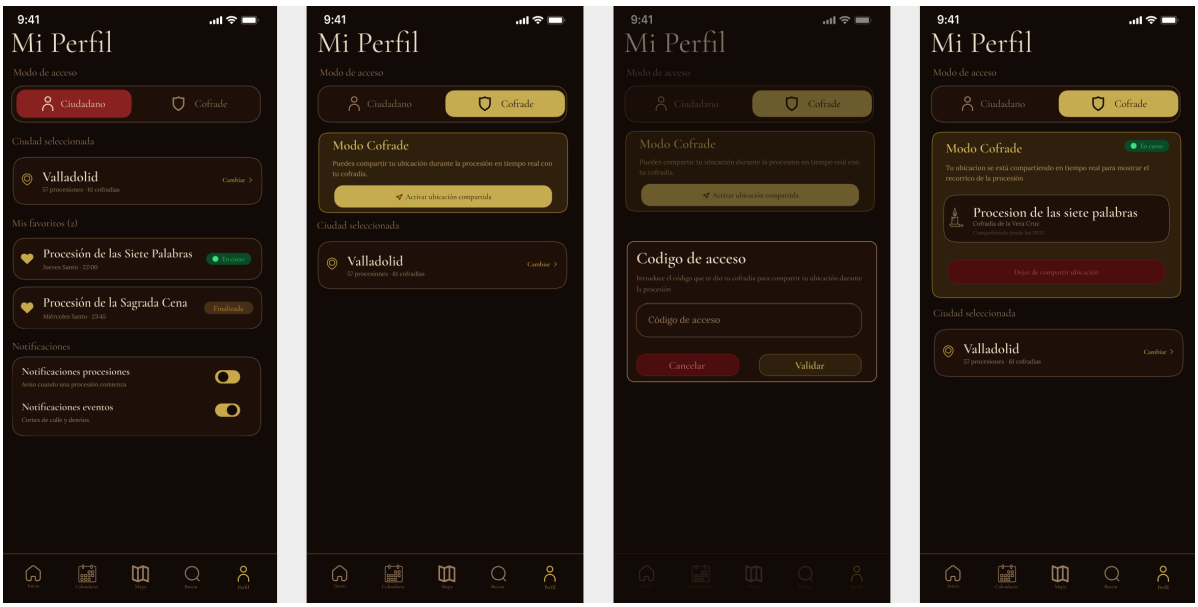


Figura 4.14: Mockup de la pantalla de perfil, en modo ciudadano y en modo cofrade

4.3.3. Implementacion

Esta tercera iteración implementa la geolocalización en tiempo real y el perfil de cofrade, lo que exige dos piezas que hasta ahora no hacían falta: la entidad `Procesion` propiamente dicha y, por primera vez en el proyecto, un mecanismo de autenticación –hasta este punto todos los *endpoints* eran de lectura o gestión pública, ya que el ciudadano nunca se registra (RI-03)–.

`Procesion` se implementa como una extensión real de `Evento` (herencia con estrategia `JOINED` de JPA, Apéndice B) en lugar de una simple referencia: hereda nombre, historia, fecha, estado, ubicación y cofradías participantes de `Evento`, y añade únicamente su fecha de inicio y fin y su `Recorrido` asociado (relación 1:1, opcional –una procesión puede programarse antes de tener ruta definida–).

Para el acceso al modo cofrade (caso de uso “Iniciar sesión con código de acceso”, Sección 3.4.1) se implementa `CodigoAcceso` junto con la infraestructura de autenticación: tokens **JWT** generados y validados con la biblioteca `jjwt`, un filtro de Spring Security que intercepta cada petición para comprobar el token, y el *endpoint* `POST /auth/codigo-acceso`, que valida el código y devuelve directamente un token identificado con la cofradía correspondiente, sin registrar ninguna fila de usuario (Apéndice B, Sección B.10). La clase `Cofrade` del modelo de dominio (Figura 3.5) se implementa así como una representación construida en memoria a partir de ese token en cada petición, no como una entidad persistida.

Por último, se implementa la funcionalidad de compartir ubicación (RF-24) mediante el *endpoint* `POST /procesiones/{id}/posiciones`, que registra cada actualización de posición del cofrade como una lectura –o *ping*– anónima en `PosicionActual` (RI-06), sin ninguna referencia a quién la envía: solo se comprueba, en el momento de recibirla, que quien la manda es un cofrade de alguna de las cofradías que participan en esa procesión. Un *ping* que cae a más de 50 metros del recorrido marcado –calculado proyectando el punto GPS sobre el tramo más cercano de la polilínea del recorrido– se descarta y no llega a guardarse, como margen frente a la deriva propia del GPS en calle y frente a lecturas de quien no va

realmente en la procesión.

A partir de ese histórico, `GET /procesiones/{id}/posiciones/actual` calcula la posición agregada de la procesión en ese momento como el promedio de los *pings* recibidos en los últimos 60 segundos, cubriendo así el requisito RNF-05. Como una procesión real no ocupa un único punto sino un tramo de calle, `GET /procesiones/{id}/estela` completa esa información con el progreso alcanzado a lo largo del recorrido por el cofrade más adelantado y por el más atrasado que están compartiendo ubicación en ese momento (RI-09) –pensado para pintarse como tres tramos del recorrido: el ya recorrido por toda la procesión, el que ocupa ahora mismo y el que todavía no ha pisado nadie–. Ese progreso se guarda como una marca de agua en la propia `Procesion`, que solo puede avanzar: si el cofrade que iba en cabeza o en cola pierde cobertura unos segundos, su último *ping* queda fuera de la ventana de 60 segundos sin que la estela retroceda ni salte hacia delante, simplemente se mantiene el último valor conocido hasta que llegue una lectura nueva que la haga avanzar más.

4.3.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual mediante springdoc-openapi (`/swagger-ui.html`), incluyendo la obtención de un token válido con `POST /auth/codigo-acceso` y su uso en las peticiones protegidas subsiguientes. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.4. 4ª Iteración

4.4.1. Analisís

4.4.2. Diseño

Interfaces de usuario

Según la planificación (Sección 2.2), esta cuarta iteración cubre el panel de la Junta de Cofradías, por lo que las pantallas que se diseñan en Figma [45] en esta iteración son el inicio de sesión y todo el panel de gestión (perfil, cofradías, procesiones, eventos y pasos), sobre los mismos criterios visuales fijados en la Iteración 1 (Sección 4.1).

Inicio de sesión A diferencia del ciudadano, que no necesita registrarse, tanto la Junta de Cofradías como el Administrador (Iteración 5) acceden con credenciales propias (*email* y contraseña). La pantalla de bienvenida distingue entre continuar como ciudadano o iniciar sesión, y esta última ofrece un selector entre los dos roles con cuenta, ya que ambos comparten la misma pantalla y el mismo *endpoint* de autenticación.

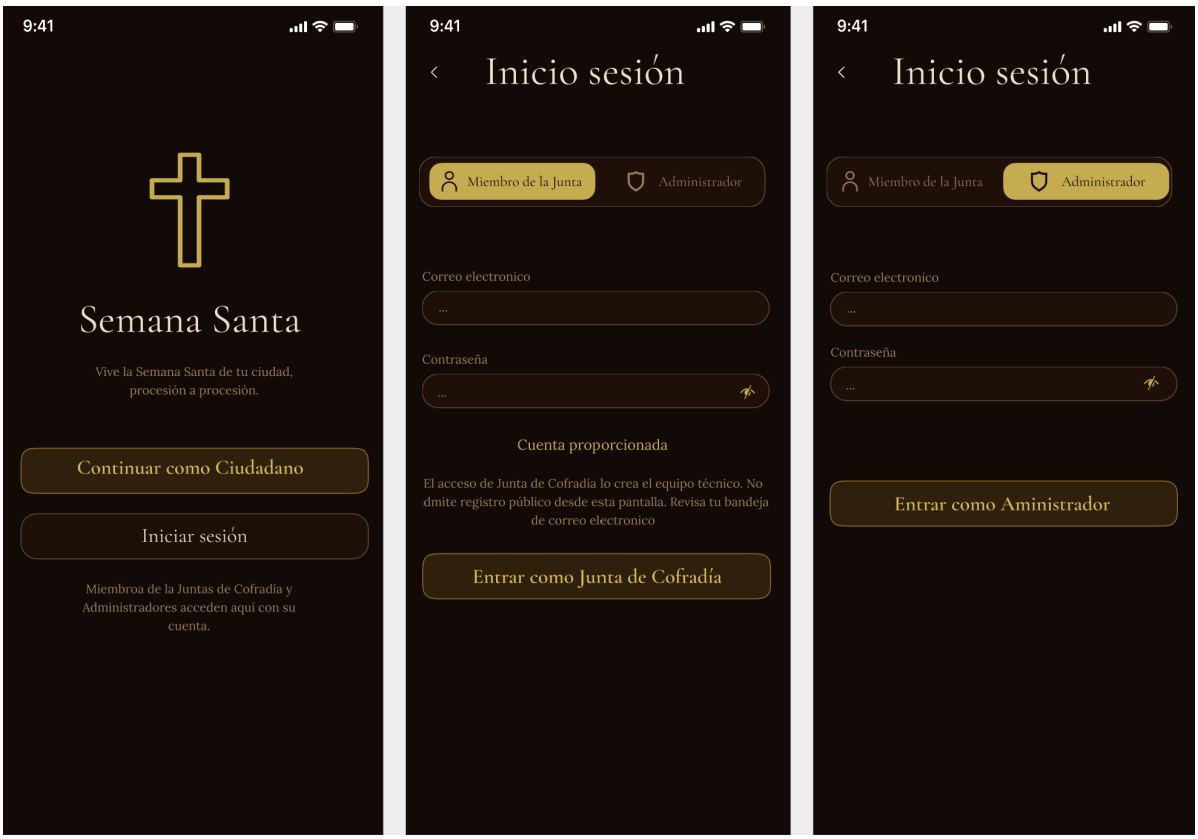


Figura 4.15: Mockup de la bienvenida y del inicio de sesión, para Junta de Cofradías y Administrador

Perfil de la Junta de Cofradías Muestra el rol y la ciudad que gestiona la Junta, y da acceso a la gestión de cofradías, procesiones, eventos y pasos, además de a la edición del propio perfil.

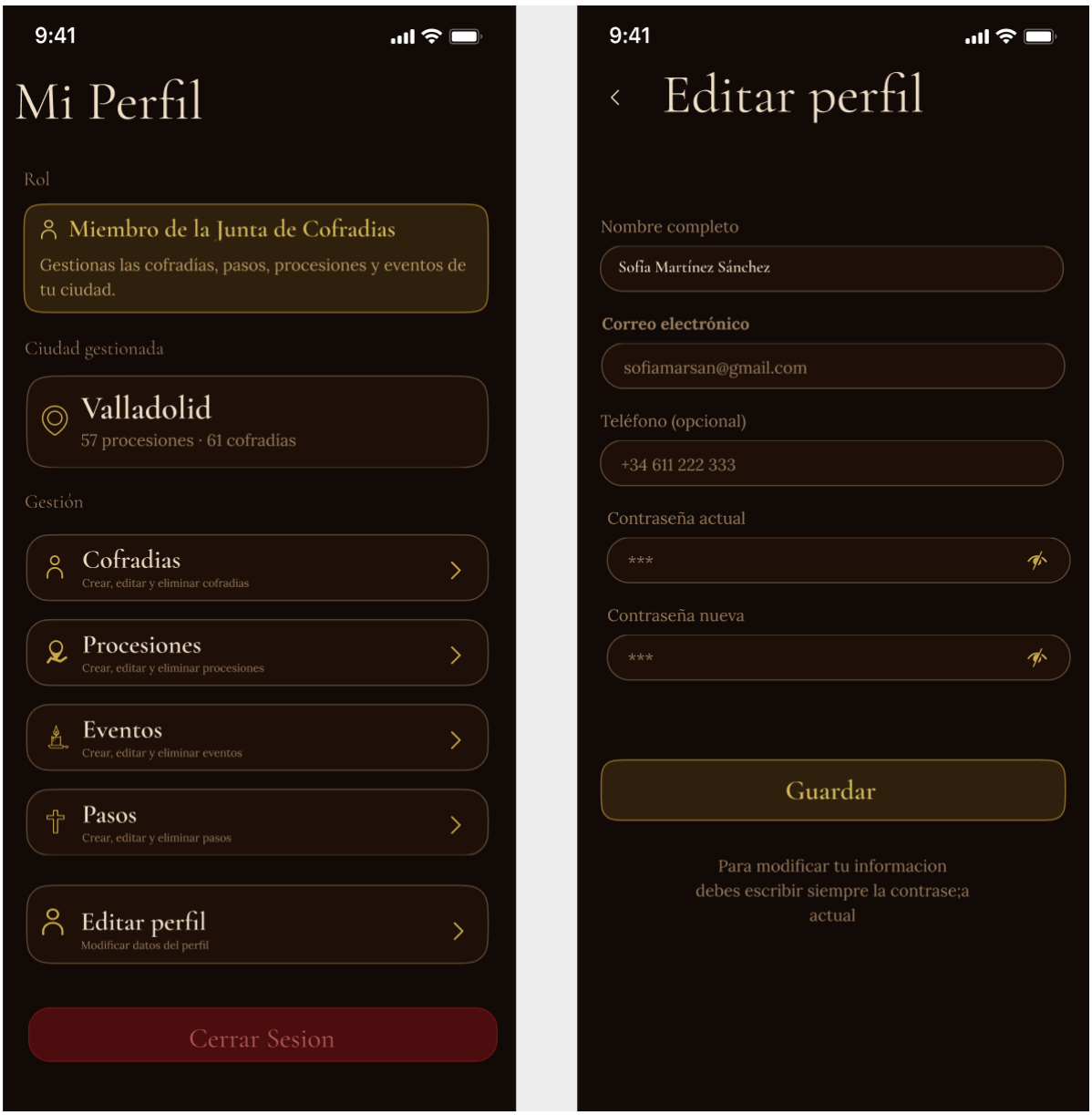


Figura 4.16: Mockup del perfil de la Junta de Cofradías y de la edición de sus datos

Cofradías Permite dar de alta una cofradía y, tras crearla, añadirle directamente procesiones, eventos o pasos, o dejarlo para más tarde. Al editar una cofradía existente se puede además consultar y gestionar esos mismos elementos ya asociados a ella.

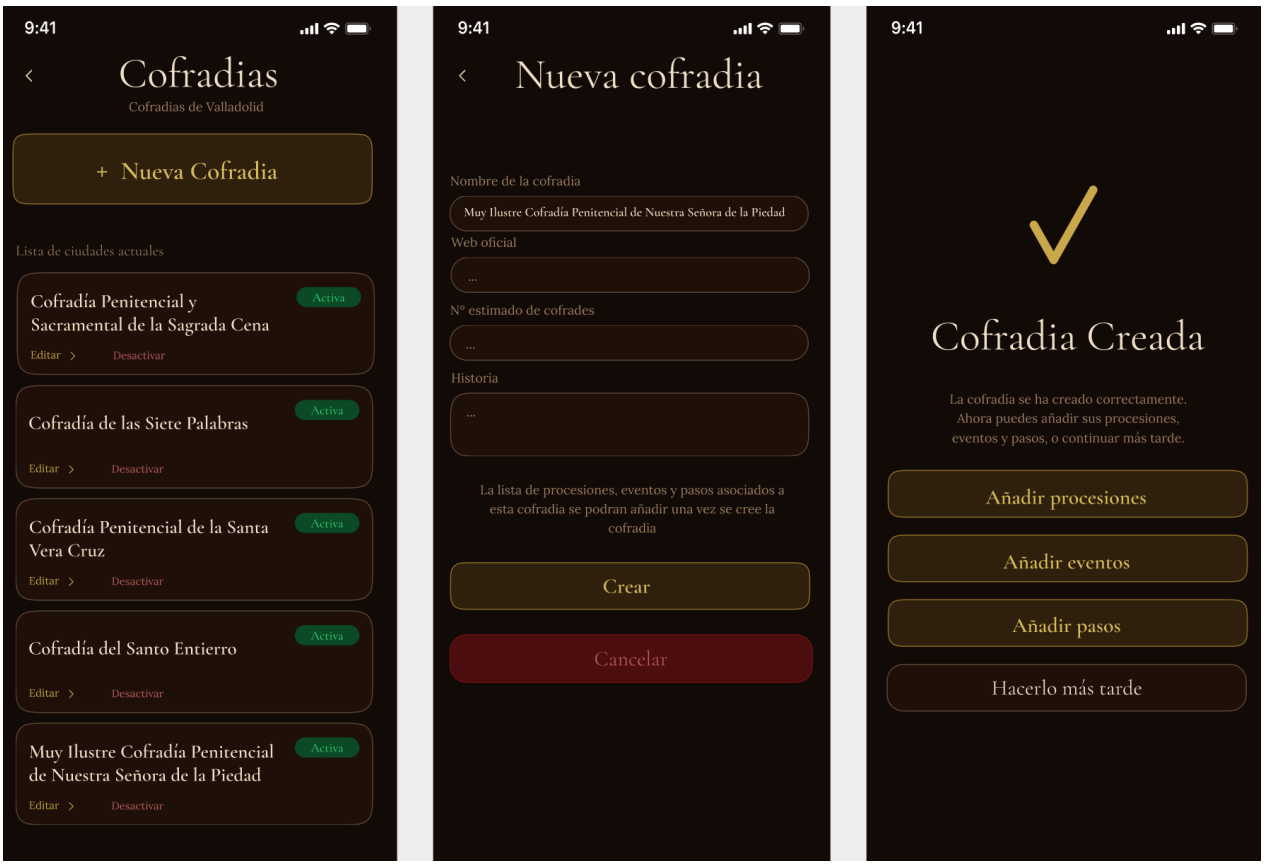


Figura 4.17: Mockup del listado de cofradías y de su creación



Figura 4.18: Mockup de la edición de una cofradía

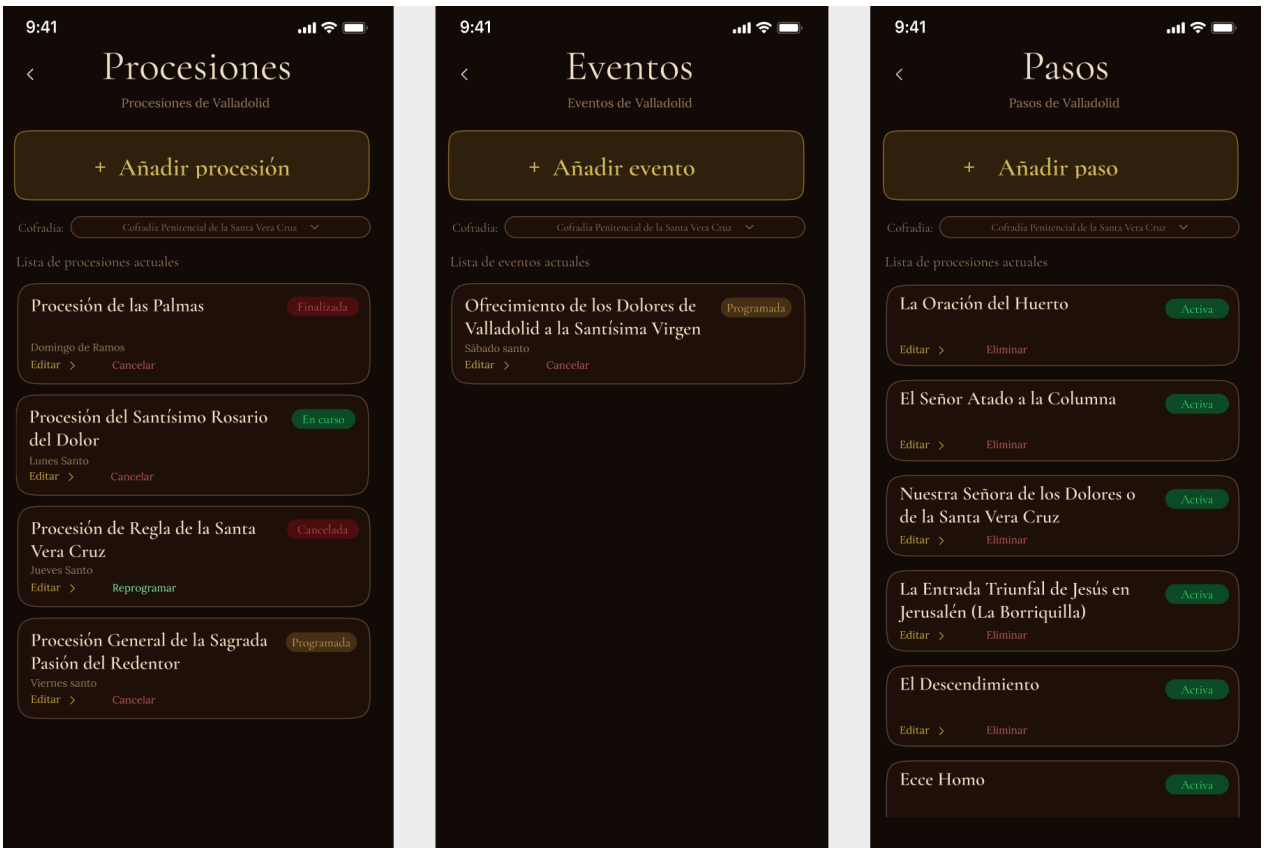


Figura 4.19: Mockup de las procesiones, eventos y pasos de una cofradía, accesibles desde su edición

Procesiones El listado de procesiones permite, además de editarlas, notificar sobre ellas (cancelación, cambio de horario o incidencia) o cancelarlas directamente. Al crear una procesión se importa su recorrido desde un archivo GPX, y tanto al crearla como al editarla se pueden asignar los pasos que participan en ella.

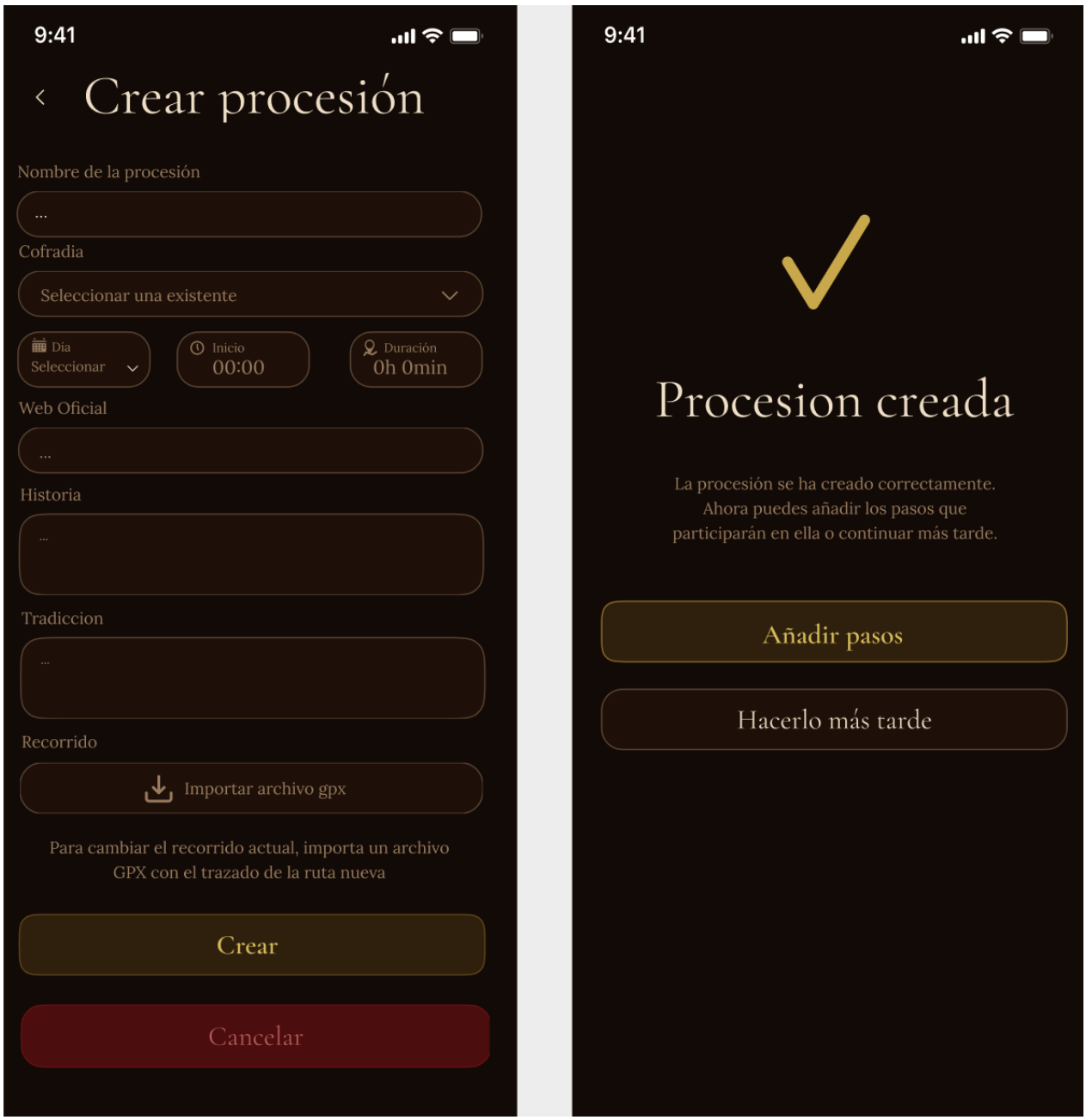


Figura 4.20: Mockup de la creación de una procesión

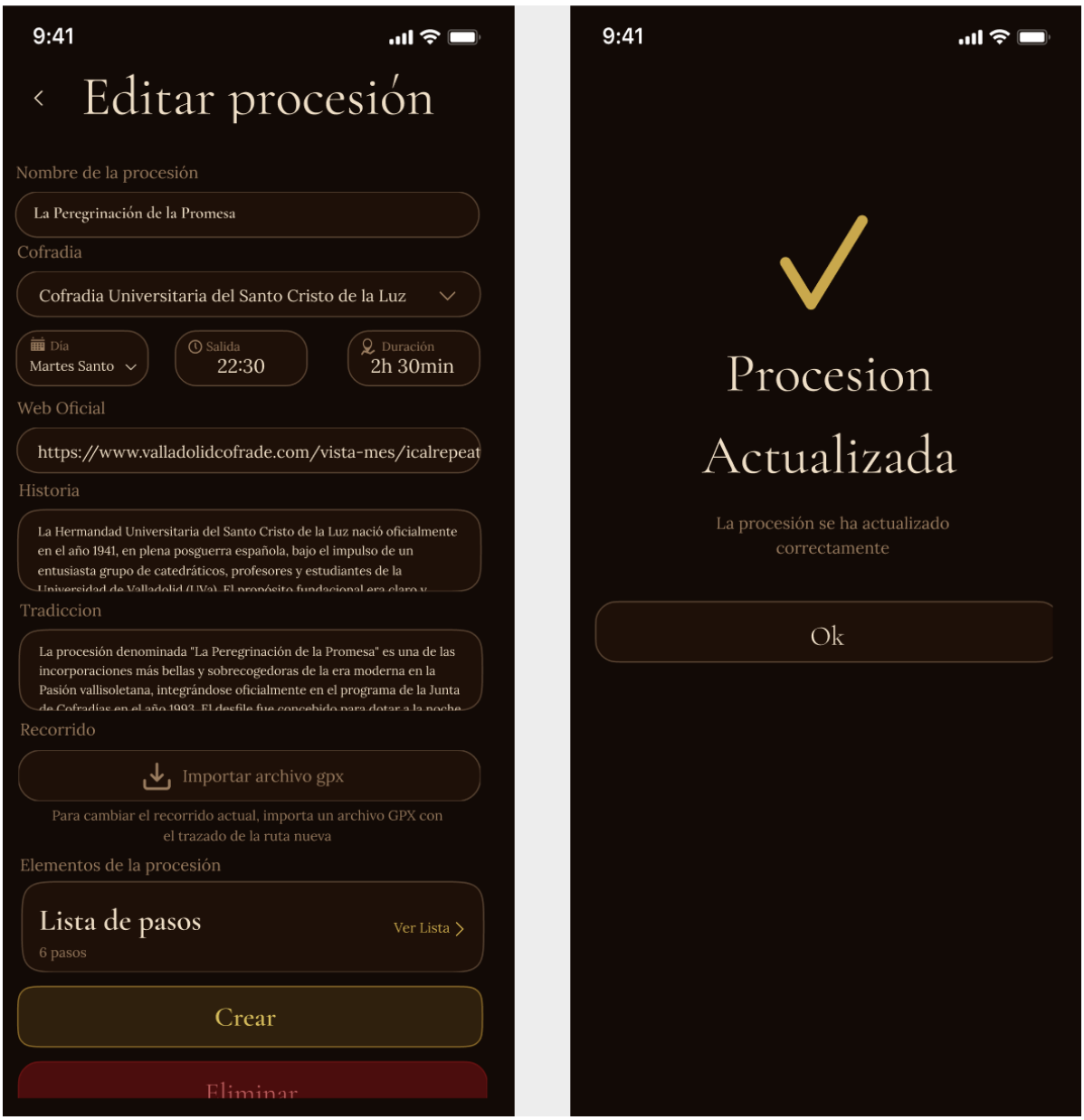


Figura 4.21: Mockup de la edición de una procesión

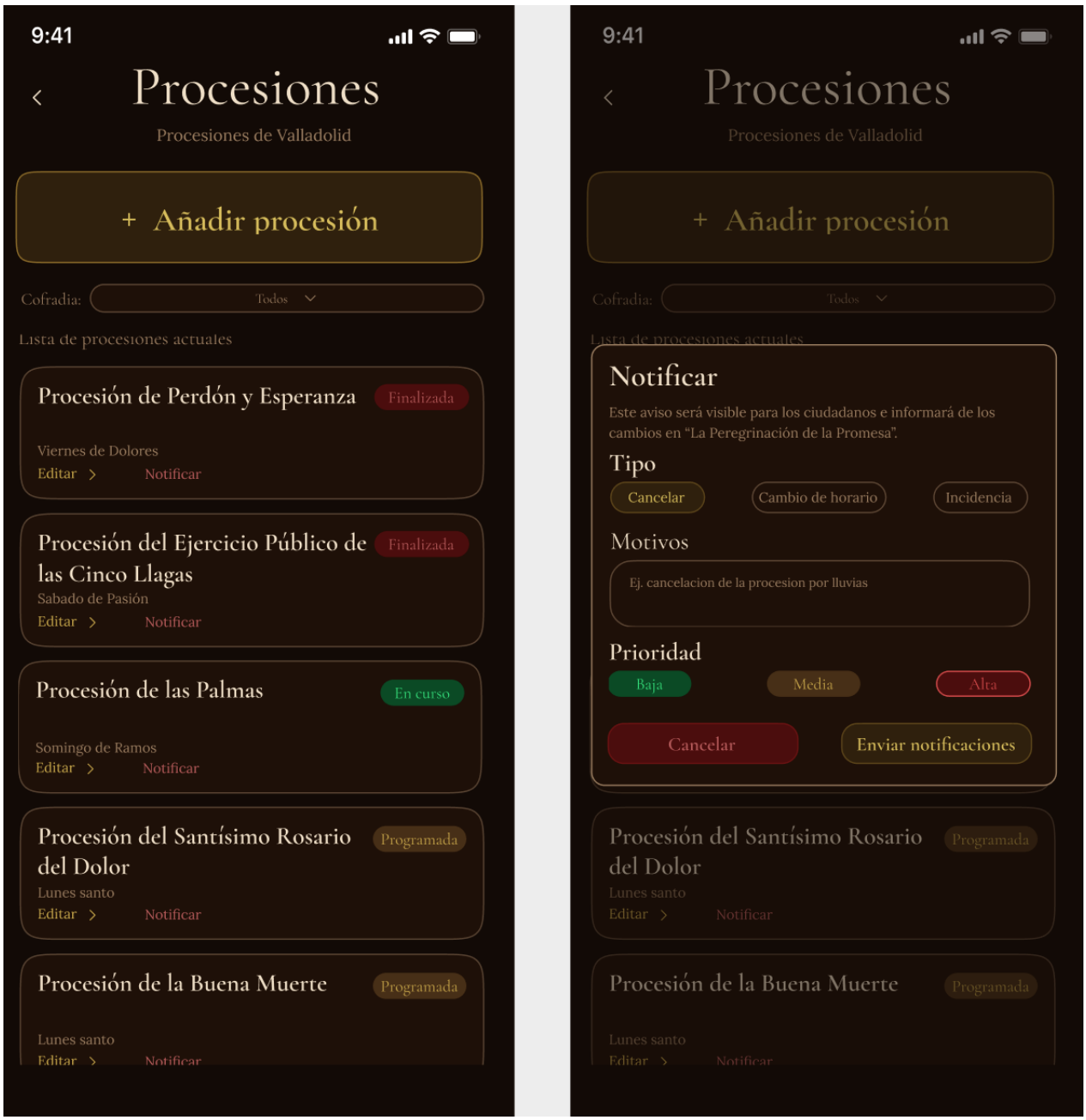


Figura 4.22: Mockup del listado de procesiones y del envío de una notificación

Eventos Análogo al de procesiones, pero sin recorrido: al crear un evento se pueden asignar directamente los pasos que participan en él, o hacerlo más tarde desde su edición.

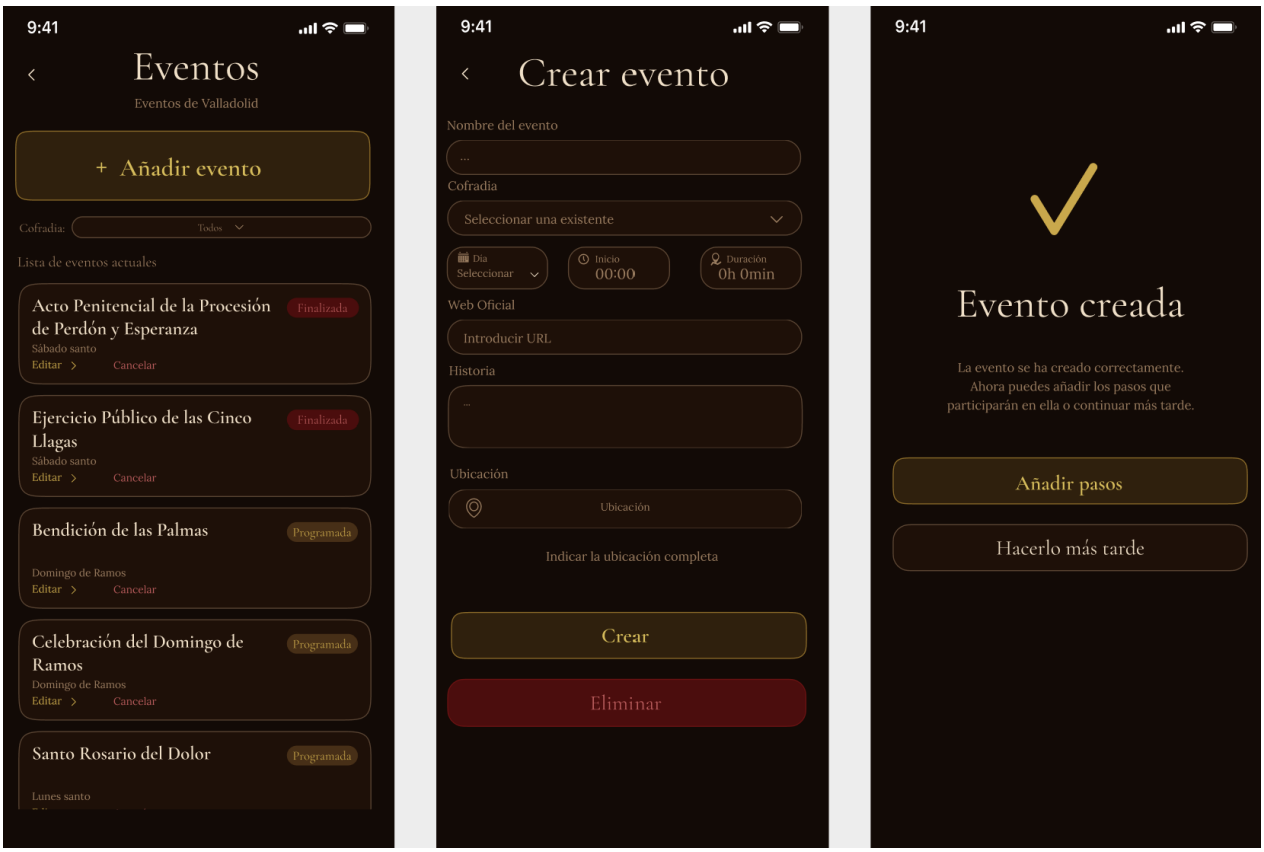


Figura 4.23: Mockup del listado de eventos y de su creación



Figura 4.24: Mockup de la edición de un evento

Pasos Cada paso pertenece a una única cofradía, y su ficha incluye su historia, un análisis artístico y una imagen, cuya URL se pega a mano en el formulario (Sección 7.1).

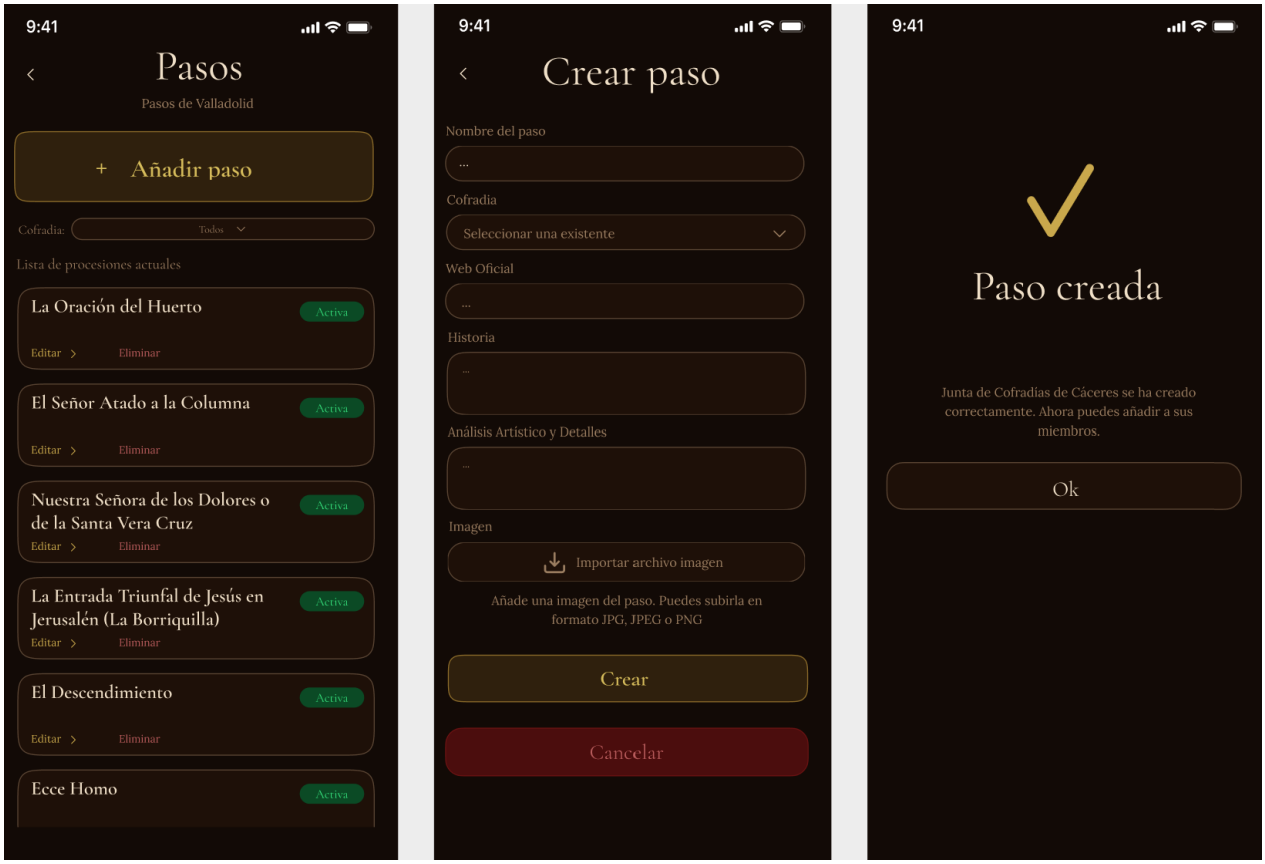


Figura 4.25: Mockup del listado de pasos y de su creación



Figura 4.26: Mockup de la edición de un paso

4.4.3. Implementacion

Esta cuarta iteración implementa el panel de la Junta de Cofradías: el acceso mediante credenciales propias y el mecanismo de control de acceso que restringe, entidad por entidad, qué puede gestionar cada Junta.

Se implementa `MiembroJuntaCofradia` –persona con credenciales que pertenece a la Junta de una ciudad, pudiendo haber varios miembros por Junta– y el *endpoint* `POST /auth/login`, que compara el `email` y la contraseña (*hash* BCrypt) recibidos y devuelve un token JWT con el identificador del miembro y su rol, siguiendo el mismo mecanismo de autenticación introducido en la Iteración 3.

Sobre esa autenticación se implementa la autorización por rol que exige el requisito RNF-03: en la capa `Service` de cada entidad de contenido –`Cofradia`, `Paso`, `Evento`, `Procesion` y `CodigoAcceso`–, las operaciones de creación, edición y eliminación comprueban que quien las solicita es un miembro de la Junta que gestiona la ciudad a la que pertenece esa cofradía, antes de permitir la operación; en caso contrario, la API responde con un código `403` (prohibido). Esta comprobación se centraliza en un único método reutilizable en lugar de repetirse en cada entidad, para evitar duplicar la misma lógica de autorización varias veces. Las entidades `Recorrido`, `Ubicacion`, `PuntoRuta` y `PuntoDeInteres`, al no pertenecer a una única cofradía –se diseñaron como recursos reutilizables entre procesiones e incluso entre ciudades distintas, Sección 4.2–, solo exigen que quien las gestione sea una Junta de Cofradías cualquiera, sin comparar ciudad.

4.4.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual: obtención de un token con `POST /auth/login` y comprobación de que las operaciones de gestión se aceptan o se rechazan (`403`) según la ciudad que gestiona cada Junta de prueba. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.5. 5ª Iteración

4.5.1. Analisís

4.5.2. Diseño

Interfaces de usuario

Según la planificación (Sección 2.2), esta quinta iteración cubre el panel de administrador, por lo que las pantallas que se diseñan en Figma [45] en esta iteración son el perfil de administrador y la gestión de ciudades, Juntas de Cofradías y sus miembros, reutilizando la misma pantalla de inicio de sesión ya diseñada para la Junta de Cofradías en la Iteración 4 (Sección 4.4).

Perfil de administrador Muestra un resumen del número de ciudades y de Juntas de Cofradías del sistema, y da acceso a la gestión de ambas, a la de los miembros de las distintas Juntas, y a la edición del propio perfil.

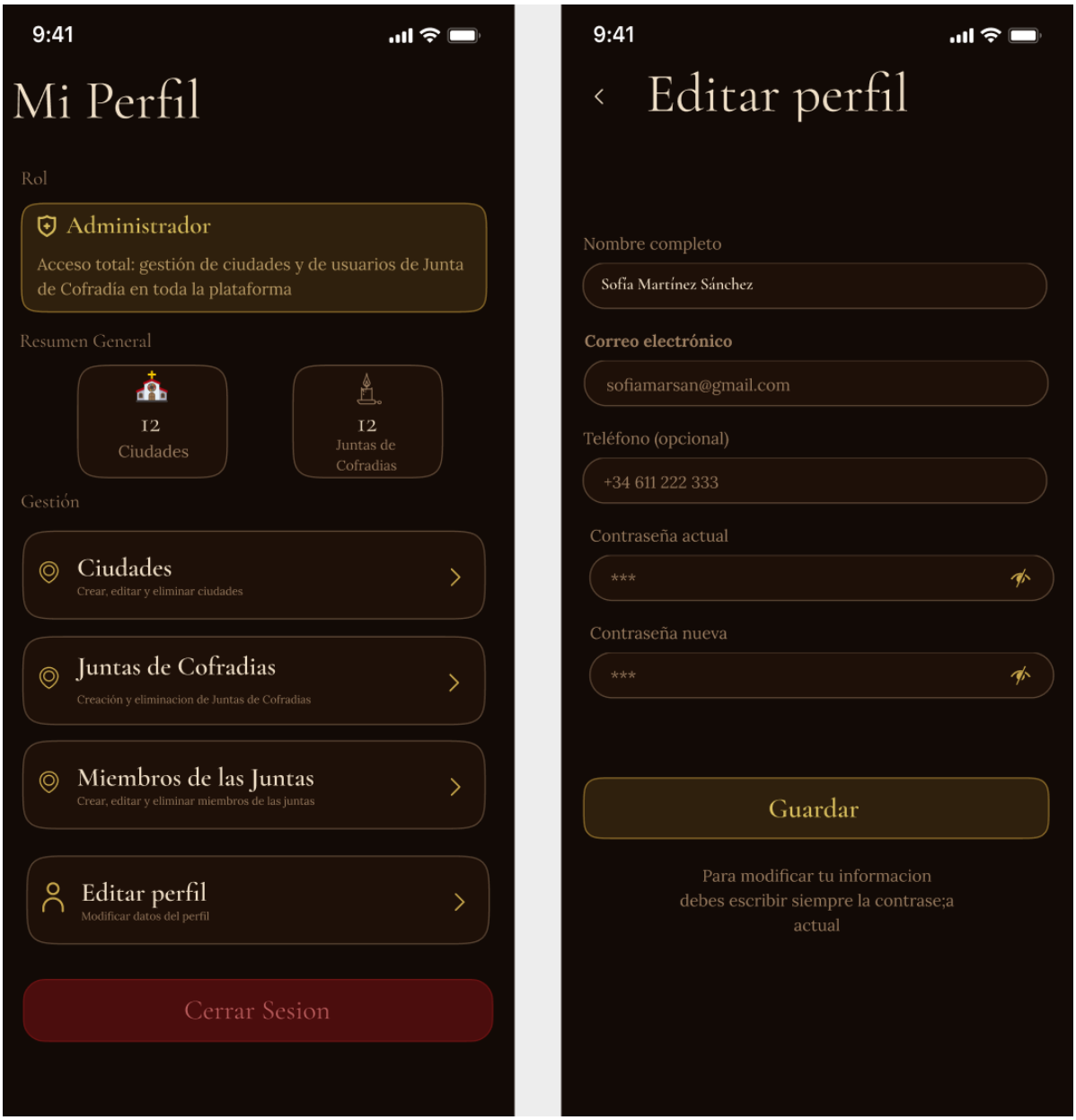


Figura 4.27: Mockup del perfil de administrador y de la edición de sus datos

Ciudades Permite dar de alta una ciudad y, sobre las ya existentes, activarlas o desactivarlas y asignarles una Junta de Cofradías.

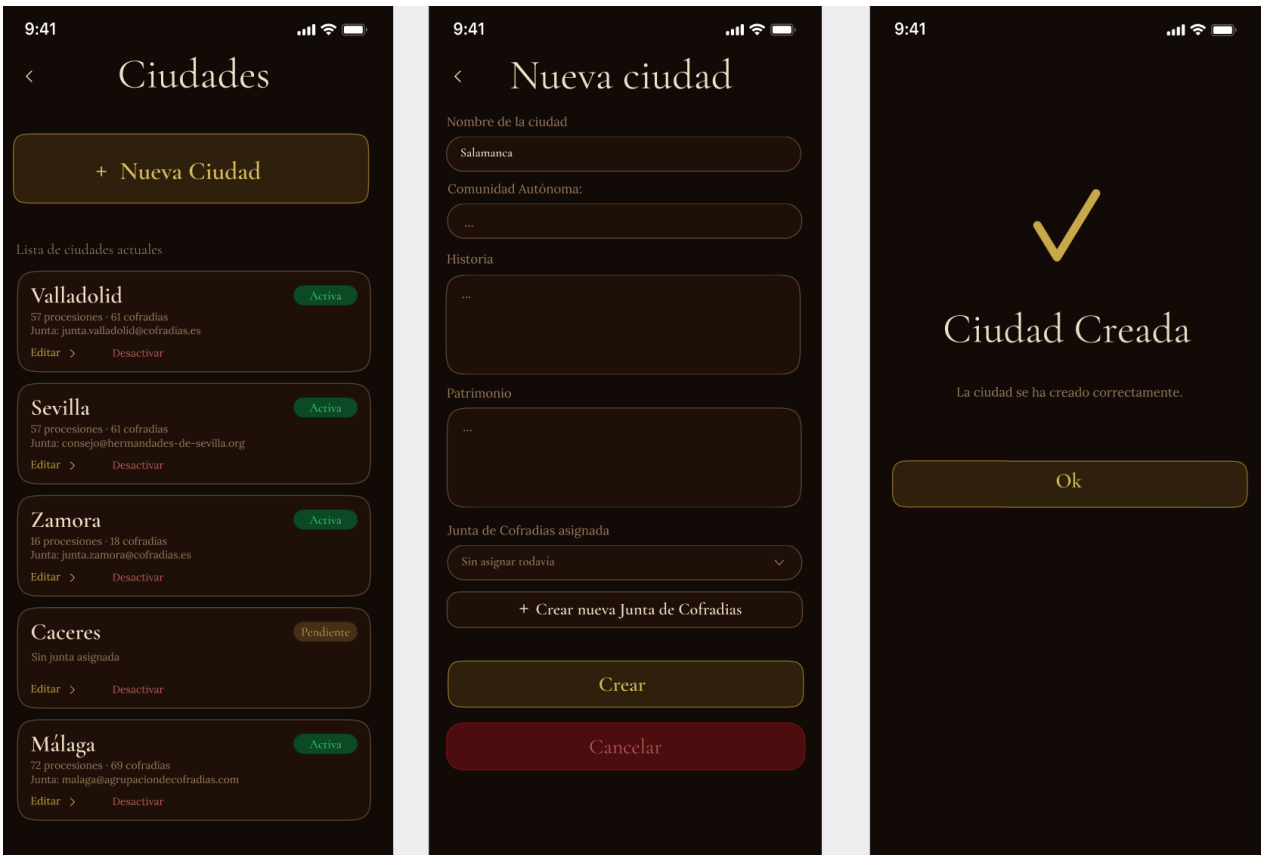


Figura 4.28: Mockup del listado de ciudades y de su creación

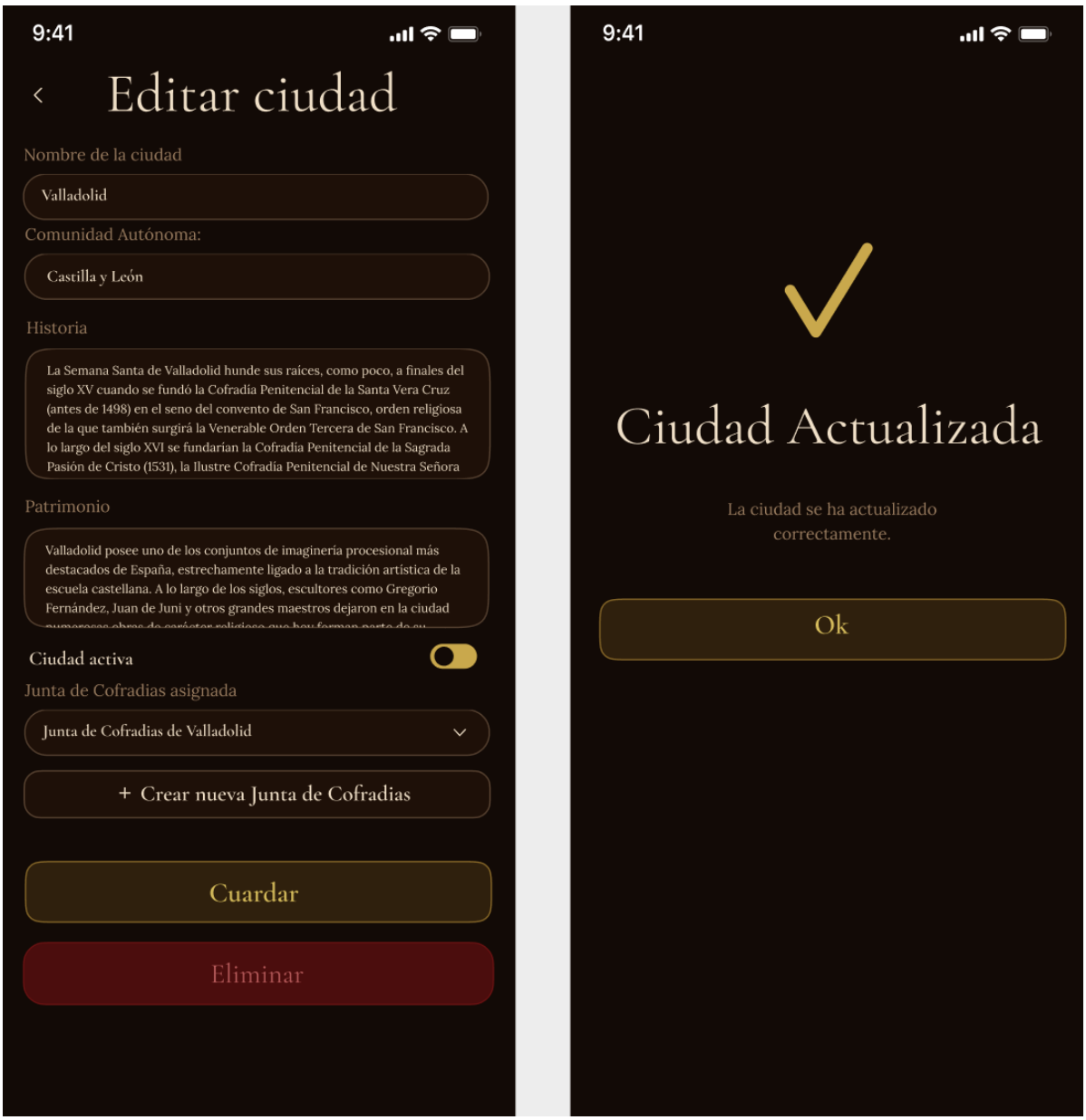


Figura 4.29: Mockup de la edición de una ciudad

Juntas de Cofradías Análogo a la gestión de ciudades: al crear una Junta se le asigna la ciudad que gestionará –solo se ofrecen ciudades que todavía no tienen una– y, tras crearla, se puede pasar directamente a añadirle miembros.

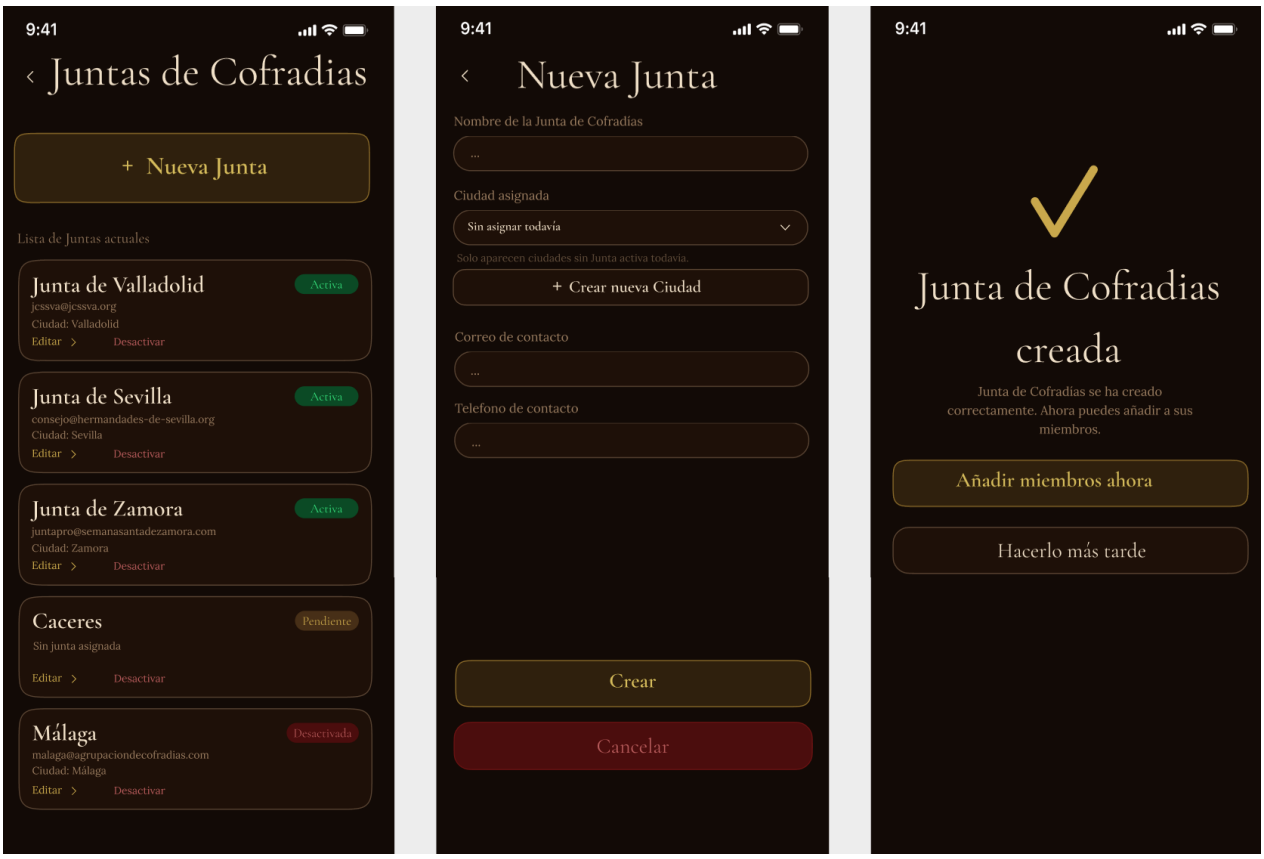


Figura 4.30: Mockup del listado de Juntas de Cofradías y de su creación



Figura 4.31: Mockup de la edición de una Junta de Cofradías

Miembros de la Junta de Cofradías Al dar de alta un miembro se le envía por correo una contraseña provisional generada automáticamente, que debe cambiar en su primer inicio de sesión; mientras no lo haga, el listado lo marca como *pendiente* y permite reenviarle la invitación. Un miembro también puede desactivarse o reactivarse sin perder sus datos.

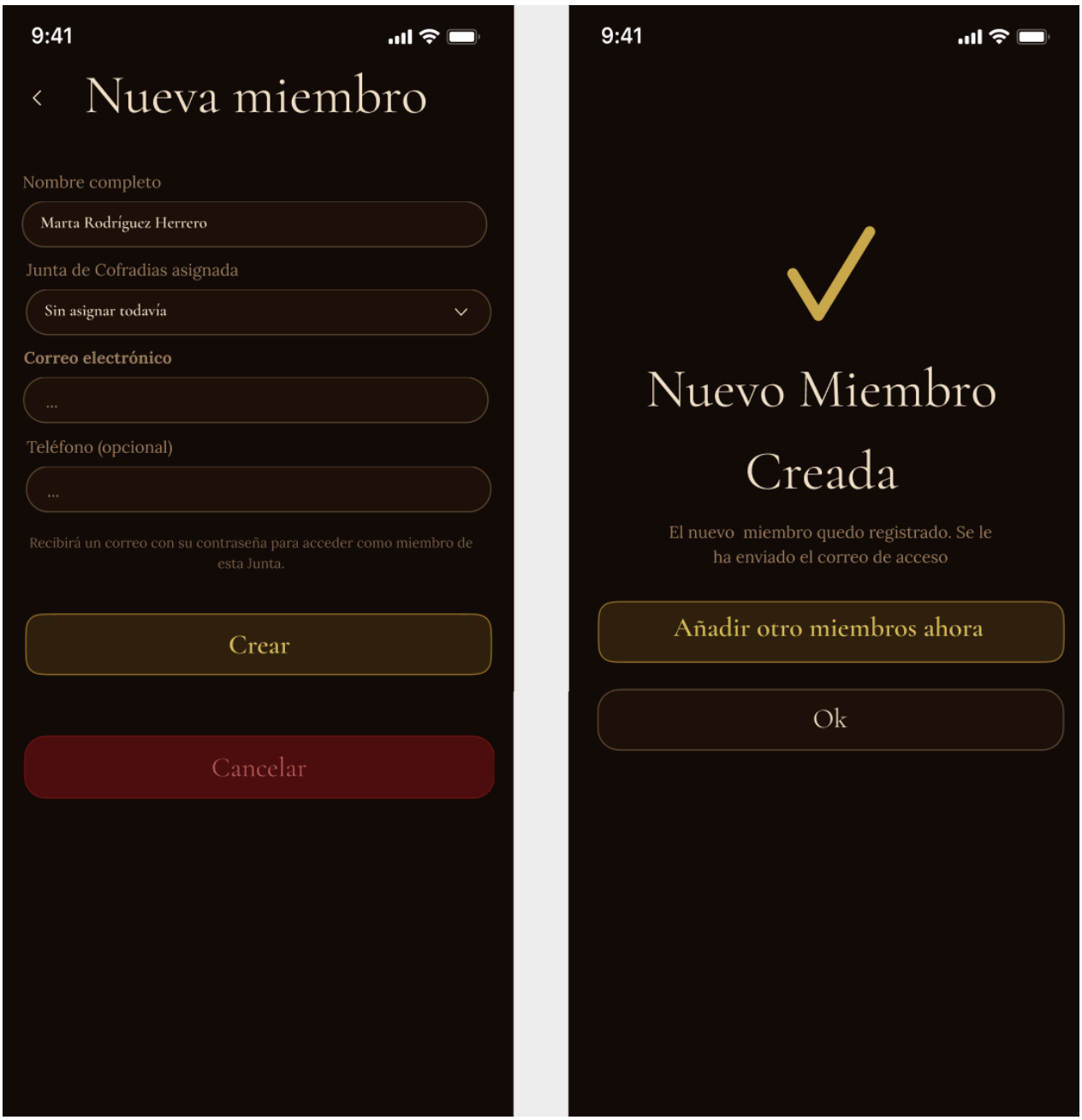


Figura 4.32: Mockup de la creación de un miembro de la Junta de Cofradías

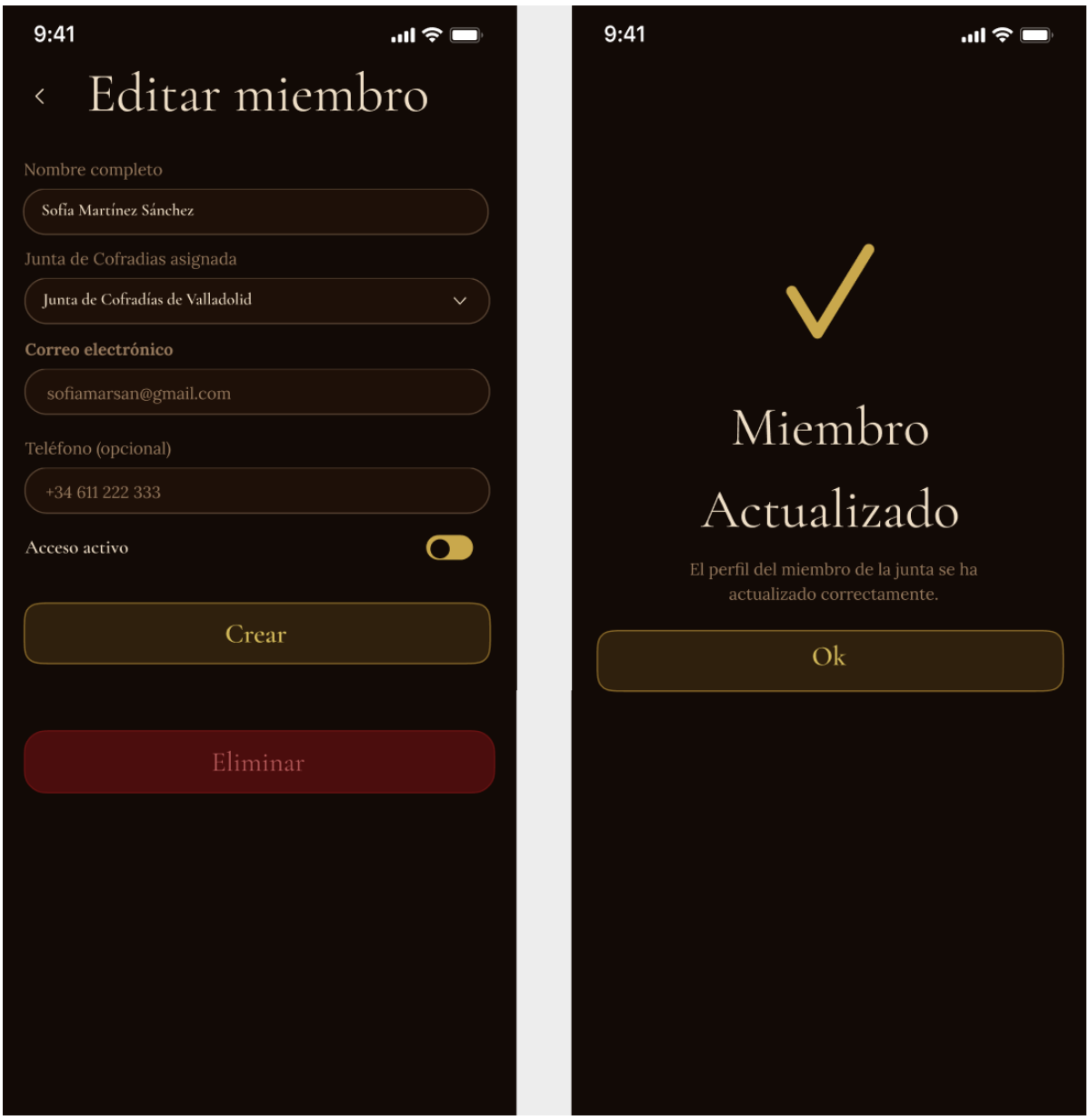


Figura 4.33: Mockup de la edición de un miembro de la Junta de Cofradías

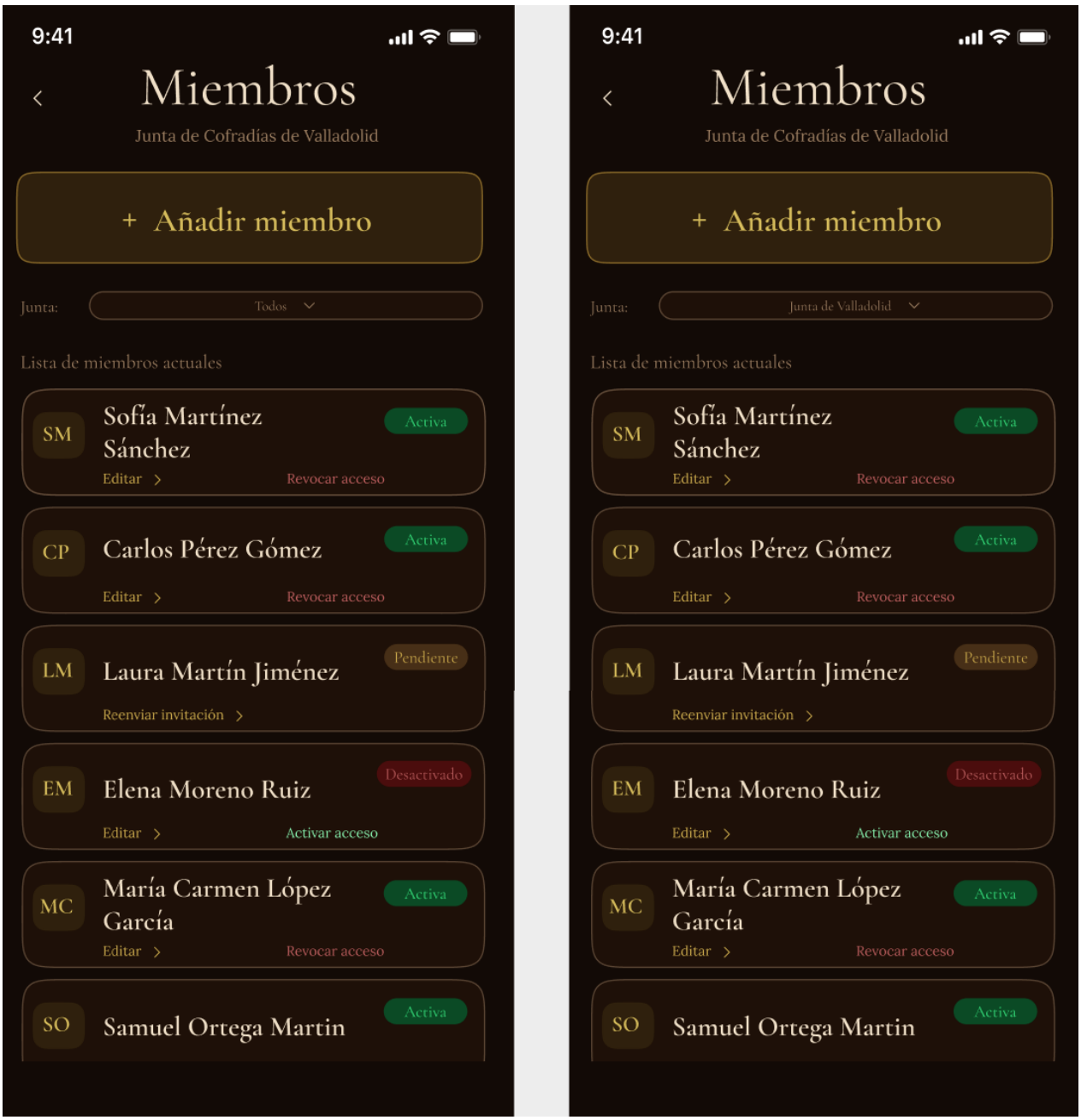


Figura 4.34: Mockup del listado de miembros, sin filtrar y filtrado por Junta de Cofradías

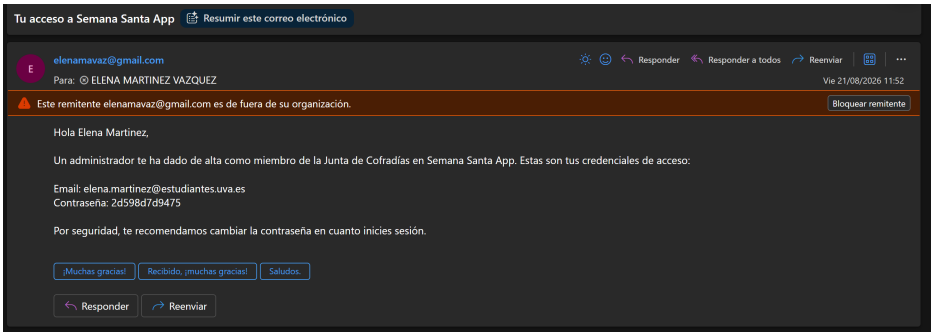


Figura 4.35: Correo de invitación recibido por un nuevo miembro de la Junta de Cofradías

4.5.3. Implementacion

Esta quinta y última iteración implementa el panel de administrador global del sistema. El sistema de notificaciones previsto para esta misma iteración queda pendiente de implementar.

Se implementa `Administrador` –rol global del sistema, sin ninguna referencia a `Ciudad`, ya que cualquier administrador puede gestionar cualquier ciudad– y su acceso mediante el mismo *endpoint* `POST /auth/login` ya utilizado por la Junta de Cofradías en la Iteración 4, que distingue el rol a partir del tipo de cuenta encontrado.

Sobre esa autenticación se completa el mecanismo de control de acceso por rol: las operaciones de creación, edición y eliminación de `Ciudad` pasan a exigir que quien las solicita sea un Administrador, sin comparar ninguna ciudad concreta –a diferencia de la comprobación introducida en la Iteración 4 para la Junta de Cofradías, aquí no hay ninguna instancia con la que comparar–. De igual forma, dar de alta, editar o dar de baja una `JuntaCofradias`, y dar de alta o dar de baja a sus `MiembroJuntaCofradia`, quedan restringidos también al Administrador, que es quien gestiona qué personas tienen acceso al panel de cada Junta.

4.5.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual: obtención de un token de Administrador con `POST /auth/login` y comprobación de que las operaciones sobre `Ciudad`, `JuntaCofradias` y `MiembroJuntaCofradia` se aceptan únicamente con ese rol. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

Capítulo 5

Estado final de la aplicación

5.0.1. Historias de usuario

ID	HU01
Nombre	Login de un usuario
Prioridad	Alta
Riesgo	Bajo
Descripción	Como usuario quiero poder acceder a la aplicación a través de una pantalla de login
Validación	- Quiero poder introducir mi nick y contraseña para acceder a mi perfil de usuario - Quiero que mi nick sea único

Tabla 5.1: Historia de usuario HU01

Capítulo 6

Pruebas

Capítulo 7

Conclusiones

7.1. Trabajo futuro

Como líneas de trabajo futuro se plantean las siguientes ampliaciones del sistema.

En primer lugar, el desarrollo de una página web para la aplicación, que permita a los usuarios acceder a determinadas funcionalidades sin necesidad de instalar la aplicación móvil.

En segundo lugar, se plantea la posibilidad de generalizar el sistema desarrollado para dar soporte no solo a la Semana Santa, sino a cualquier tipo de evento o celebración tradicional (por ejemplo, ferias, romerías o fiestas patronales). Para ello, entre otros cambios, las entidades específicas del dominio actual deberían sustituirse por clases más genéricas: las cofradías o hermandades pasarían a modelarse como *organizaciones*, sus miembros o cofrades como *miembros*, y las Juntas de Cofradías como *conjuntos de organizaciones* encargados de coordinar a varias organizaciones dentro de un mismo evento. De esta forma, el resto del sistema (geolocalización en tiempo real, gestión de recorridos, notificaciones, etc.) podría reutilizarse prácticamente sin cambios, ya que estas funcionalidades no dependen de la naturaleza religiosa del evento, sino de su estructura organizativa. Esta generalización estaría alineada con el objetivo de escalabilidad planteado inicialmente para el proyecto (Sección 1.4), y permitiría ampliar el alcance de la aplicación mucho más allá del ámbito de la Semana Santa.

En tercer lugar, la imagen de un `Paso` se guarda hoy como una URL de texto libre (Apéndice B), sin ninguna infraestructura de subida de archivos: es la Junta de Cofradías quien aloja la imagen en otro sitio y pega el enlace en el formulario, en lugar de seleccionarla directamente desde el propio dispositivo. Como mejora futura, esto podría resolverse de dos formas: guardando la imagen como un campo binario en la propia base de datos PostgreSQL –sin depender de ningún servicio externo, aunque no es la solución más eficiente para servir muchas imágenes pesadas a la vez–, o retomando el planteamiento inicial de usar **Firebase Storage** [37] –un servicio dedicado a este propósito, con mejor rendimiento a mayor escala, pero que exigiría gestionar las credenciales de un proyecto de Firebase además del backend propio ya existente–.

En cuarto lugar, la importación del recorrido desde un archivo GPX podría aprovechar más información

del propio archivo. Hoy solo se leen las coordenadas de los puntos, y los puntos de interés se marcan después uno a uno desde la aplicación, lo que resulta lento en un recorrido largo. El formato GPX permite incluir puntos con nombre (`<wpt>`) junto al trazado, de modo que la Junta podría señalar en la propia herramienta con la que dibuja la ruta los puntos de interés que quiere marcar, y la aplicación crearlos automáticamente al importar el archivo, dejando la edición posterior solo para completar o corregir su información.

En quinto lugar, las notificaciones solo pueden activarse o desactivarse en bloque, desde el permiso de notificaciones del dispositivo. Una mejora natural sería permitir al ciudadano elegir desde la propia aplicación qué tipos de aviso quiere recibir (por ejemplo, solo los de procesiones o solo los de eventos). Esto exigiría guardar esas preferencias junto a cada dispositivo registrado y asociar cada notificación al tipo de acto del que informa, para que el backend pueda filtrar a quién se la envía.

En sexto lugar, el acceso de los cofrades podría simplificarse mostrando el código de acceso de cada cofradía también como un código QR, de modo que el cofrade pudiera entrar al modo cofrade escaneándolo con la cámara del móvil en lugar de tener que escribirlo a mano.

Por último, sería interesante adaptar la interfaz en función de la Semana Santa de cada ciudad, asociando a cada una la terminología propia que se emplea localmente, de forma que la experiencia resulte más cercana y personalizada para cada usuario.

7.2. Puntos débiles

El sistema presenta una seguridad limitada en el acceso de los cofrades, que se realiza introduciendo un código de acceso compartido por toda la cofradía. No obstante, se ha priorizado la usabilidad frente a la seguridad, ya que el nivel de riesgo asociado a este método de acceso se considera asumible dado el contexto de uso de la aplicación, y la Junta de Cofradías puede revocar el código en cualquier momento.

Además, la comunicación entre la aplicación y el servidor se realiza mediante HTTP, sin cifrar, porque el servidor de despliegue no dispone de un certificado TLS. En consecuencia, las credenciales de inicio de sesión, los tokens de sesión y las posiciones que comparten los cofrades viajan por la red sin cifrar y podrían ser interceptados en una red no segura. Para un uso real, sería imprescindible servir la API mediante HTTPS, con un dominio propio y un certificado TLS (por ejemplo, de Let's Encrypt), lo que además permitiría retirar de la aplicación las excepciones que hoy permiten el tráfico HTTP.

Por otro lado, la introducción de información a través del dispositivo móvil resulta costosa para el usuario. Una alternativa más adecuada sería ofrecer una página web con capacidad de edición tanto para las juntas de cofradías como para los administradores. Sin embargo, por motivos de acotación del alcance del proyecto y de la carga de trabajo disponible, esta funcionalidad no se ha llegado a implementar.

Por otro lado, el uso de un backend propio con Spring Boot y PostgreSQL, en lugar de una plataforma *Backend-as-a-Service* como Firebase, traslada a la propia autora responsabilidades que antes asumía la plataforma de forma automática: el dimensionado y escalado de la infraestructura, las copias de seguridad de la base de datos, y la aplicación de parches de seguridad al propio servidor. Al desplegarse en

un servidor virtual gestionado directamente (Apéndice C), estas tareas, junto con la actualización del servicio a cada nueva versión, recaen por completo en la propia autora, algo que con Firebase no era necesario.

Asimismo, la posición en vivo de una procesión (RNF-05) se resuelve mediante una API REST convencional consultada periódicamente por el cliente, en lugar de mediante los *listeners* en tiempo real que ofrecía Cloud Firestore de forma nativa. Esta solución cumple el requisito tal y como está definido (una actualización cada 30 segundos como mínimo), pero no ofrece la misma sensación de instantaneidad que un mecanismo de *push*, y su eficiencia se degrada más rápido al aumentar el número de usuarios simultáneos que la consultan, ya que cada consulta es una petición HTTP independiente en lugar de una única notificación que el servidor reenvía a quien esté suscrito. Del mismo modo, la disponibilidad sin conexión de los datos estáticos (RNF-06), cubierta antes de forma automática por la caché local de Firestore, exige ahora una solución explícita en el cliente (Sección 4.1), con el consiguiente esfuerzo de implementación adicional.

Por último, el desarrollo de un backend propio con una tecnología (Spring Boot) con la que la autora contaba con una experiencia previa limitada –un único proyecto de prácticas durante el Grado– ha supuesto una curva de aprendizaje que no era necesaria con el planteamiento inicial basado en Firebase, y que se ha gestionado mediante una prueba de concepto acotada antes de comenzar la implementación completa, tal y como recoge el riesgo correspondiente (Sección 2.3).

Por último, al asociarse los códigos de acceso a nivel de cofradía y no de procesión concreta (Sección 4.1), un cofrade recibe notificaciones de todas las procesiones de su cofradía y no únicamente de aquellas en las que participa. Se considera una limitación menor, asumible dado que el número de procesiones por cofradía es reducido.

Bibliografía

[1] Google, "Maps JavaScript API — documentación oficial." https://developers.google.com/maps/documentation/javascript, 2024. Consultado el 1 de enero de 2025.

[2] Evaluando Software, "La geolocalización funciona." https://www.evaluandosoftware.com/bpm/la-geolocalizacion-funciona/, 2026. Consultado el 9 de mayo de 2026.

[3] PlayCofrade, "El penitente — aplicación móvil." https://elpenitente.app/, 2026. Consultado el 9 de mayo de 2026.

[4] PlayCofrade, "El penitente — aplicación móvil (google play)." https://play.google.com/store/apps/details?id=com.playcofrade.elpenitente&hl=es, 2026. Consultado el 9 de mayo de 2026.

[5] Valladolid Cofrade, "Lanzamos la nueva app de Valladolid Cofrade: toda la Semana Santa 2026 en tu móvil y con avisos en tiempo real." https://www.valladolidcofrade.com/noticias-2026/3660-lanzamos-la-nueva-app-de-valladolid-cofrade-toda-la-semana-santa-2026-en-tu-movil-y-con-avisos-en-tiempo-real, 2026. Consultado el 9 de mayo de 2026.

[6] Mozilla Developer Network, "¿qué es una PWA?." https://developer.mozilla.org/es/docs/Web/Progressive_web_apps, 2026. Consultado el 9 de mayo de 2026.

[7] Ayuntamiento de Zaragoza, "ssantazgz — aplicación móvil." https://www.zaragoza.es/sede/portal/turismo/post/app-semana-santa, 2026. Consultado el 9 de mayo de 2026.

[8] Junta Pro Semana Santa de Zamora, "Junta pro semana santa de zamora — aplicación móvil." https://semanasantadezamora.com/, 2026. Consultado el 9 de mayo de 2026.

[9] Uber Technologies, "Uber — página oficial." https://www.uber.com, 2024. Consultado el 1 de enero de 2025.

[10] M. Llorente Pérez, "Mejoras en la usabilidad de la aplicación móvil OrientaTree para orientación deportiva aplicada a la educación," trabajo de fin de grado, Universidad de Valladolid, 2025. https://uvadoc.uva.es/handle/10324/79500.

[11] I. Sommerville, *Software Engineering*. Boston, MA: Pearson, 10 ed., 2015. https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf.

[12] ProjectManager, "What is a Gantt chart?." https://www.projectmanager.com/guides/gantt-chart, 2026. Consultado el 9 de mayo de 2026.

[13] IEEE, "IEEE Std 29148-2018: Systems and software engineering — requirements engineering," Standard 29148-2018, Institute of Electrical and Electronics Engineers, 2018. https://ieeexplore.ieee.org/document/8559686.

[14] Object Management Group, "UML — unified modeling language, especificación oficial." https://www.omg.org/spec/UML, 2017. Versión 2.5.1. Consultado el 1 de enero de 2025.

[15] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Upper Saddle River, NJ: Prentice Hall, 3 ed., 2004. https://www.pearson.com/en-us/subject-catalog/p/applying-uml-and-patterns/P200000003228.

[16] OWASP Foundation, "Access control cheat sheet." https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html, 2024. Consultado el 1 de enero de 2025.

[17] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Newtown Square, PA: Project Management Institute, 7 ed., 2021. https://www.pmi.org/pmbok-guide-standards.

[18] B. W. Boehm, *Software Risk Management*. Los Alamitos, CA: IEEE Computer Society Press, 1989.

[19] OpenStreetMap Foundation, "Openstreetmap — mapa libre del mundo." https://www.openstreetmap.org, 2024. Consultado el 1 de mayo de 2025.

[20] GitHub, Inc., "Github — plataforma de desarrollo de software." https://github.com, 2024. Consultado el 1 de mayo de 2025.

[21] Glassdoor, "Sueldo: Jefe de proyecto en españa." https://www.glassdoor.es/Sueldos/jefe-de-proyecto-sueldo-SRCH_KO0,16_IP2.htm, 2025. Consultado el 9 de julio de 2026.

[22] Glassdoor, "Sueldo: Analista funcional en españa." https://www.glassdoor.es/Sueldos/analista-funcional-sueldo-SRCH_KO0,18.htm, 2025. Consultado el 9 de julio de 2026.

[23] Glassdoor, "Sueldo: Ux designer en españa." https://www.glassdoor.es/Sueldos/ux-designer-sueldo-SRCH_KO0,11.htm, 2025. Consultado el 9 de julio de 2026.

[24] Glassdoor, "Sueldo: Backend developer en españa." https://www.glassdoor.es/Sueldos/backend-developer-sueldo-SRCH_KO0,17.htm, 2026. Consultado el 9 de julio de 2026.

[25] Glassdoor, "Salario de desarrollador móvil junior en españa." https://www.glassdoor.es, 2025. Consultado el 1 de mayo de 2025.

[26] Glassdoor, "Sueldos para el puesto de qa tester en españa." https://www.glassdoor.es/Sueldos/qa-tester-sueldo-SRCH_KO0,9.htm, 2025. Consultado el 9 de julio de 2026.

[27] Hetzner Online GmbH, "Hetzner Cloud — planes y precios (servidor CX23)." https://www.hetzner.com/cloud, 2026. Consultado el 17 de octubre de 2026.

[28] Google, "Android studio — entorno de desarrollo oficial para android." https://developer.android.com/studio, 2024. Consultado el 1 de mayo de 2025.

[29] Microsoft, "Visual studio code — editor de código fuente." https://code.visualstudio.com, 2024. Consultado el 1 de mayo de 2025.

[30] Expo, "Expo — documentación oficial." https://docs.expo.dev/, 2024. Consultado el 7 de agosto de 2026.

[31] Dribba, "¿cuánto cuesta subir una app a google play?." https://dribba.com/blog/cuanto-cuesta-subir-una-app-al-google-play, 2026. Consultado el 9 de julio de 2026.

[32] Figma, Inc., "Planes y precios de figma." https://www.figma.com/es-es/pricing/, 2026. Consultado el 9 de julio de 2026.

[33] Atlassian, "Planes de precios de jira software." https://www.atlassian.com/es/software/jira/pricing, 2026. Consultado el 9 de julio de 2026.

[34] Fotocasa, "Oficinas de alquiler en centro, valladolid capital." https://www.fotocasa.es/es/alquiler/oficinas/valladolid-capital/centro/l, 2026. Consultado el 9 de julio de 2026.

[35] Meta Platforms, "React native — documentación oficial." https://reactnative.dev/docs/getting-started, 2024. Consultado el 15 de julio de 2026.

[36] VMware, "Spring boot — documentación oficial." https://spring.io/projects/spring-boot, 2024. Consultado el 7 de agosto de 2026.

[37] Google Firebase, "Cloud storage for firebase — documentación oficial." https://firebase.google.com/docs/storage, 2024. Consultado el 7 de agosto de 2026.

[38] Google Firebase, "Firebase cloud messaging — documentación oficial." https://firebase.google.com/docs/cloud-messaging, 2024. Consultado el 1 de enero de 2025.

[39] PostgreSQL Global Development Group, "PostgreSQL — documentación oficial." https://www.postgresql.org/docs/, 2024. Consultado el 7 de agosto de 2026.

[40] VMware, "Spring data JPA — documentación oficial." https://spring.io/projects/spring-data-jpa, 2024. Consultado el 7 de agosto de 2026.

[41] Red Hat, "Hibernate ORM — documentación oficial." https://hibernate.org/orm/documentation/, 2024. Consultado el 7 de agosto de 2026.

[42] VMware, "Spring security — documentación oficial." https://spring.io/projects/spring-security, 2024. Consultado el 7 de agosto de 2026.

[43] IETF, "RFC 7519: JSON web token (JWT)." https://datatracker.ietf.org/doc/html/rfc7519, 2015. Consultado el 7 de agosto de 2026.

[44] TanStack, "TanStack query — documentación oficial." https://tanstack.com/query/latest, 2024. Consultado el 7 de agosto de 2026.

[45] Figma, Inc., “Figma — herramienta de diseño de interfaces colaborativa.” https://www.figma.com, 2024. Consultado el 1 de mayo de 2025.

[46] Google, “Material design 3.” https://m3.material.io/, 2024. Consultado el 27 de julio de 2026.

[47] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.

[48] springdoc, “springdoc-openapi — documentación oficial.” https://springdoc.org/, 2024. Consultado el 7 de agosto de 2026.

Anexos

Apéndice A

Casos de uso adicionales

Este apéndice recoge los casos de uso adicionales del sistema que, por no considerarse esenciales para la comprensión general del proyecto, no se han incluido en el cuerpo principal de la memoria (Sección 3.3). Cada uno de ellos sigue la misma estructura que los presentados en dicha sección: resumen, actor, precondición, postcondición, secuencia base, secuencia alternativa y requisitos relacionados. Se agrupan por el rol que los realiza, en el mismo orden que los diagramas de casos de uso; los casos de uso que comparten el miembro de la Junta de Cofradías y el administrador (iniciar sesión y gestionar su propio perfil) se recogen una sola vez, junto a los del miembro de la Junta.

A.1. Ciudadano

A.2. Cofrade

A.3. Miembro de la Junta de Cofradías

A.4. Administrador

Caso de uso: Seleccionar ciudad	
Resumen	El usuario elige la ciudad de la que quiere consultar la Semana Santa.
Actor	Ciudadano
Precondición	Ninguna.
Postcondición	La ciudad elegida queda guardada en el dispositivo, y el dispositivo queda registrado para recibir las notificaciones de esa ciudad.
Secuencia Base	
[1] El sistema solicita permiso para acceder a la ubicación del dispositivo, la primera vez que se accede como ciudadano.	
[2] El usuario concede el permiso.	
[3] El sistema selecciona automáticamente la ciudad activa más cercana a la ubicación del usuario.	
[4] El sistema guarda la ciudad seleccionada y registra el dispositivo para recibir sus notificaciones.	
Secuencia Alternativa	
[2.a.1] El usuario deniega el permiso de ubicación: el sistema muestra las ciudades activas, ordenadas por nombre, y el usuario elige una; el caso de uso continúa en el paso 4.	
[3.a.1] No se puede obtener la ubicación o no hay ninguna ciudad cercana: el sistema muestra las ciudades activas y el usuario elige una.	
[3.a.2] El usuario busca una ciudad por su nombre: el sistema muestra solo las ciudades que coinciden con la búsqueda.	
[4.a.1] El usuario cambia más adelante de ciudad: el sistema muestra las ciudades activas, guarda la nueva y vuelve a registrar el dispositivo para las notificaciones de esa ciudad.	
[4.a.2] El usuario desiste de elegir ciudad la primera vez: el sistema olvida que había entrado como ciudadano y vuelve al inicio de la aplicación.	
Requisitos Relacionados	RF-02, RF-03 y RF-04.

Tabla A.1: Caso de uso: Seleccionar ciudad

Caso de uso: Consultar información de una cofradía, procesión, evento o paso	
Resumen	El usuario consulta la información detallada de una cofradía, procesión, evento o paso de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra la información detallada del elemento elegido.
Secuencia Base	
[1] El usuario elige una cofradía, procesión, evento o paso, ya sea desde un listado, desde una búsqueda, desde el calendario o desde sus favoritos.	
[2] El sistema muestra la información detallada del elemento: en una cofradía, su historia, su web, sus pasos y las procesiones y eventos en los que participa; en una procesión, su día, hora de salida, duración, estado, cofradías, pasos y el recorrido sobre el mapa; en un evento, su día, hora, duración, estado, cofradías, pasos y la ubicación sobre el mapa; en un paso, su historia, su análisis artístico, su imagen y su cofradía.	
[3] El usuario puede pasar de un elemento a otro relacionado con él (por ejemplo, de una procesión a uno de sus pasos).	
Secuencia Alternativa	
[2.a.1] La procesión todavía no tiene recorrido: el sistema indica que el recorrido está pendiente de confirmar.	
[2.a.2] La procesión está en curso: el sistema muestra además, sobre el recorrido, el tramo que ocupa en ese momento.	
[2.a.3] Sin conexión: el sistema muestra la última información consultada del elemento, si la hay, o informa de que no puede cargarla.	
Requisitos Relacionados	RF-05, RF-06, RF-07 y RF-08.

Tabla A.2: Caso de uso: Consultar información de una cofradía, procesión, evento o paso

Caso de uso: Acceder a listados de procesiones, pasos, eventos o cofradías	
Resumen	El usuario consulta el listado completo de procesiones, pasos, eventos o cofradías de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad (caso de uso «Seleccionar ciudad»).
Postcondición	Se muestra el listado del tipo de elemento elegido.
Secuencia Base	
[1] El usuario solicita el listado de procesiones, pasos, eventos o cofradías.	
[2] El sistema muestra todos los elementos de ese tipo de la ciudad seleccionada, ordenados por nombre, indicando el estado de las procesiones y los eventos.	
[3] El usuario puede escribir un texto para reducir el listado a los elementos cuyo nombre lo contiene.	
[4] El usuario elige un elemento y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] No hay elementos de ese tipo en la ciudad: el sistema informa de que no hay elementos registrados.	
[2.a.2] Error de conexión: el sistema informa del error y permite volver a intentarlo.	
[3.a.1] Ningún elemento coincide con el texto: el sistema informa de que no hay resultados.	
Requisitos Relacionados	RF-11.

Tabla A.3: Caso de uso: Acceder a listados de procesiones, pasos, eventos o cofradías

Caso de uso: Buscar cofradías, eventos, procesiones o pasos	
Resumen	El usuario busca por su nombre cofradías, eventos, procesiones o pasos de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad (caso de uso «Seleccionar ciudad»).
Postcondición	Se muestran los elementos que coinciden con la búsqueda.
Secuencia Base	
[1] El usuario solicita realizar una búsqueda.	
[2] El usuario introduce el texto que quiere buscar.	
[3] El sistema muestra las procesiones y los eventos cuyo nombre contiene el texto, ordenados por hora, junto con los pasos y las cofradías que también coinciden.	
[4] El usuario elige un resultado y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El usuario no introduce ningún texto: el sistema muestra todos los elementos de la ciudad.	
[3.a.1] El usuario quiere acotar los resultados: se realiza el caso de uso «Filtrar elementos».	
[3.a.2] No hay resultados: el sistema informa de que no se han encontrado elementos.	
Requisitos Relacionados	RF-12.

Tabla A.4: Caso de uso: Buscar cofradías, eventos, procesiones o pasos

Caso de uso: Filtrar elementos	
Resumen	El usuario acota los resultados de una búsqueda según el día de la Semana Santa y el estado de las procesiones y los eventos.
Actor	Ciudadano
Precondición	El usuario está realizando una búsqueda (caso de uso «Buscar cofradías, eventos, procesiones o pasos»).
Postcondición	Los resultados de la búsqueda se limitan a los que cumplen los filtros elegidos.
Secuencia Base	
[1] El usuario solicita filtrar los resultados.	
[2] El sistema ofrece filtrar por día de la Semana Santa y por estado (en curso, programado, finalizado o cancelado).	
[3] El usuario elige un día, un estado o ambos.	
[4] El sistema muestra solo las procesiones y los eventos que cumplen los filtros elegidos.	
Secuencia Alternativa	
[3.a.1] El usuario quita los filtros: el sistema vuelve a mostrar todos los resultados de la búsqueda.	
[4.a.1] Hay algún filtro activo: el sistema deja de mostrar los pasos y las cofradías, que no tienen día ni estado.	
[4.a.2] Ningún elemento cumple los filtros: el sistema informa de que no hay resultados.	
Requisitos Relacionados	RF-13.

Tabla A.5: Caso de uso: Filtrar elementos

Caso de uso: Consultar calendario de Semana Santa	
Resumen	El usuario consulta en un calendario mensual qué días tienen procesiones o eventos programados.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra el calendario del mes, indicando los días con procesiones o eventos.
Secuencia Base	
[1] El usuario solicita consultar el calendario.	
[2] El sistema muestra el mes de la Semana Santa e indica qué días tienen procesiones o eventos en la ciudad seleccionada.	
[3] El usuario puede avanzar o retroceder de mes.	
[4] El usuario elige un día y se realiza el caso de uso «Consultar procesiones y eventos de un día».	
Secuencia Alternativa	
[2.a.1] La ciudad no tiene procesiones ni eventos: el sistema muestra el calendario sin ningún día indicado.	
Requisitos Relacionados	RF-09.

Tabla A.6: Caso de uso: Consultar calendario de Semana Santa

Caso de uso: Consultar procesiones y eventos de un día	
Resumen	El usuario consulta las procesiones y los eventos programados en un día concreto.
Actor	Ciudadano
Precondición	El usuario ha elegido un día del calendario (caso de uso «Consultar calendario de Semana Santa»).
Postcondición	Se muestran las procesiones y los eventos de ese día.
Secuencia Base	
[1] El sistema muestra la semana del día elegido y el resumen del día: su nombre dentro de la Semana Santa, su fecha y cuántas procesiones y eventos tiene.	
[2] El sistema muestra las procesiones y los eventos de ese día, ordenados por hora, con sus cofradías, su hora y su duración.	
[3] El usuario puede pasar a otro día de la misma semana, o a la semana anterior o siguiente.	
[4] El usuario elige una procesión o un evento y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El día no tiene procesiones ni eventos: el sistema informa de que no hay nada registrado ese día.	
[4.a.1] El usuario vuelve al calendario mensual: se realiza de nuevo el caso de uso «Consultar calendario de Semana Santa».	
Requisitos Relacionados	RF-10.

Tabla A.7: Caso de uso: Consultar procesiones y eventos de un día

Caso de uso: Añadir favoritos	
Resumen	El usuario marca como favorita una cofradía, procesión, evento o paso.
Actor	Ciudadano
Precondición	El elemento no está marcado como favorito.
Postcondición	El elemento queda guardado como favorito en el dispositivo.
Secuencia Base	
[1] El usuario indica que quiere marcar como favorito un elemento que está consultando.	
[2] El sistema guarda el elemento como favorito en el dispositivo.	
[3] El sistema indica que el elemento ya es favorito.	
Secuencia Alternativa	
No se contemplan secuencias alternativas.	
Requisitos Relacionados	RF-14.

Tabla A.8: Caso de uso: Añadir favoritos

Caso de uso: Eliminar favorito	
Resumen	El usuario deja de tener como favorito una cofradía, procesión, evento o paso.
Actor	Ciudadano
Precondición	El elemento está marcado como favorito.
Postcondición	El elemento deja de estar guardado como favorito.
Secuencia Base	
[1] El usuario indica que quiere quitar de favoritos un elemento que está consultando.	
[2] El sistema quita el elemento de los favoritos guardados en el dispositivo.	
[3] El sistema indica que el elemento ya no es favorito.	
Secuencia Alternativa	
No se contemplan secuencias alternativas.	
Requisitos Relacionados	RF-14.

Tabla A.9: Caso de uso: Eliminar favorito

Caso de uso: Consultar favoritos	
Resumen	El usuario consulta reunidos todos los elementos que ha marcado como favoritos.
Actor	Ciudadano
Precondición	Ninguna.
Postcondición	Se muestra el listado de favoritos del usuario.
Secuencia Base	
[1] El usuario solicita consultar sus favoritos.	
[2] El sistema muestra cuántos favoritos tiene el usuario y de qué tipo (cofradías, procesiones, eventos y pasos).	
[3] El sistema muestra el listado de todos los favoritos, ordenados por nombre.	
[4] El usuario elige uno y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El usuario no tiene favoritos: el sistema informa de que todavía no ha marcado ninguno.	
[4.a.1] El usuario quita un elemento de favoritos desde el listado: se realiza el caso de uso «Eliminar favorito» y el sistema actualiza el listado.	
Requisitos Relacionados	RF-14.

Tabla A.10: Caso de uso: Consultar favoritos

Caso de uso: Consultar estado de las notificaciones	
Resumen	El usuario consulta si tiene activadas las notificaciones de su ciudad y, si quiere cambiarlo, accede a los ajustes del dispositivo.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	El usuario conoce si recibe o no las notificaciones de su ciudad.
Secuencia Base	
[1] El usuario solicita consultar el estado de las notificaciones.	
[2] El sistema consulta el permiso de notificaciones del dispositivo e informa de si están activadas, indicando la ciudad de la que se reciben.	
[3] El usuario indica que quiere cambiarlo.	
[4] El sistema abre los ajustes de la aplicación en el dispositivo, donde el usuario activa o desactiva las notificaciones.	
[5] Al volver a la aplicación, el sistema consulta de nuevo el permiso y actualiza el estado mostrado.	
Secuencia Alternativa	
[2.a.1] Las notificaciones están desactivadas: el sistema informa de que no recibirá los avisos de su ciudad hasta que las active en los ajustes del dispositivo.	
[3.a.1] El usuario no quiere cambiar nada: el caso de uso termina.	
Requisitos Relacionados	RF-21 y RF-22.

Tabla A.11: Caso de uso: Consultar estado de las notificaciones

Caso de uso: Generar ruta hacia procesión	
Resumen	El usuario obtiene una ruta desde su ubicación hasta un lugar relevante de una procesión (su inicio, un punto de interés o su fin), evitando las procesiones activas. **Caso de uso no implementado en la versión actual** (ver Sección 7.1).
Actor	Ciudadano
Precondición	El usuario ha concedido permiso de ubicación y la procesión tiene recorrido.
Postcondición	Se muestra sobre el mapa una ruta desde la ubicación del usuario hasta el lugar elegido.
Secuencia Base	
[1] El usuario elige una procesión y el lugar al que quiere llegar (inicio, punto de interés o fin).	
[2] El sistema obtiene la ubicación actual del usuario.	
[3] El sistema calcula una ruta a pie hasta ese lugar que evita los tramos ocupados por procesiones activas.	
[4] El sistema muestra la ruta sobre el mapa.	
Secuencia Alternativa	
[2.a.1] No se puede obtener la ubicación del usuario: el sistema informa de que no puede calcular la ruta.	
[3.a.1] No existe ninguna ruta que evite las procesiones activas: el sistema informa de ello y ofrece la ruta más corta.	
Requisitos Relacionados	RF-19 y RF-20.

Tabla A.12: Caso de uso: Generar ruta hacia procesión

Caso de uso: Elegir procesión en curso	
Resumen	El cofrade elige en qué procesión quiere compartir su ubicación cuando su cofradía tiene varias en curso a la vez.
Actor	Cofrade
Precondición	El cofrade ha solicitado compartir su ubicación y su cofradía tiene más de una procesión en curso (caso de uso «Compartir ubicación durante procesión»).
Postcondición	Queda elegida la procesión en la que se compartirá la ubicación.
Secuencia Base	
[1] El sistema muestra las procesiones en curso de la cofradía.	
[2] El cofrade elige una de ellas.	
[3] El caso de uso «Compartir ubicación durante procesión» continúa con la procesión elegida.	
Secuencia Alternativa	
[2.a.1] El cofrade no elige ninguna: no se comparte la ubicación.	
Requisitos Relacionados	RF-24.

Tabla A.13: Caso de uso: Elegir procesión en curso

Caso de uso: Dejar de compartir ubicación	
Resumen	El cofrade deja de compartir su ubicación en cualquier momento.
Actor	Cofrade
Precondición	El cofrade está compartiendo su ubicación (caso de uso «Compartir ubicación durante procesión»).
Postcondición	El sistema deja de enviar la ubicación del cofrade.
Secuencia Base	
[1] El cofrade indica que quiere dejar de compartir su ubicación.	
[2] El sistema detiene el envío de la ubicación, también el que se realiza con la pantalla bloqueada.	
[3] El sistema informa de que ya no se está compartiendo la ubicación.	
Secuencia Alternativa	
[1.a.1] El cofrade cierra la sesión mientras comparte su ubicación: el sistema deja de compartirla igualmente.	
Requisitos Relacionados	RF-24.

Tabla A.14: Caso de uso: Dejar de compartir ubicación

Caso de uso: Iniciar sesión	
Resumen	Un miembro de la Junta de Cofradías o un administrador accede a la aplicación con su correo electrónico y su contraseña.
Actor	Miembro de la Junta de Cofradías, Administrador
Precondición	El usuario tiene una cuenta dada de alta en el sistema.
Postcondición	El usuario queda identificado con su rol y accede a las funcionalidades de gestión que le corresponden.
Secuencia Base	
[1] El usuario solicita iniciar sesión e indica si es miembro de una Junta de Cofradías o administrador.	
[2] El sistema solicita el correo electrónico y la contraseña.	
[3] El usuario introduce sus credenciales.	
[4] El sistema comprueba las credenciales y que la cuenta corresponde al tipo de usuario indicado.	
[5] El sistema inicia la sesión, válida durante 24 horas, y da acceso al usuario a sus funcionalidades de gestión.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Credenciales incorrectas: el sistema informa de que el correo o la contraseña no son correctos y no inicia la sesión.	
[4.a.2] La cuenta no corresponde al tipo de usuario indicado: el sistema lo informa y no inicia la sesión.	
[5.a.1] El miembro de la Junta todavía usa la contraseña provisional que recibió por correo: se realiza el caso de uso «Cambiar contraseña provisional» antes de darle acceso.	
[5.a.2] La cuenta del miembro de la Junta está desactivada: el sistema le informa de que no puede realizar cambios y le ofrece el caso de uso «Solicitar reactivación de cuenta».	
[5.a.3] La sesión caduca mientras se usa la aplicación: el sistema la cierra e informa al usuario de que debe volver a iniciar sesión.	
Requisitos Relacionados	RF-26.

Tabla A.15: Caso de uso: Iniciar sesión

Caso de uso: Cambiar contraseña provisional	
Resumen	El miembro de la Junta de Cofradías sustituye, la primera vez que inicia sesión, la contraseña provisional que recibió por correo por una propia.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión con la contraseña provisional que el sistema le envió al darle de alta o al reenviarle la invitación.
Postcondición	La contraseña provisional queda sustituida por la nueva, el alta del miembro queda completada y el usuario accede a sus funcionalidades de gestión.
Secuencia Base	
[1] El sistema informa al usuario de que debe elegir una contraseña propia antes de continuar, y le impide realizar cualquier otra operación hasta hacerlo.	
[2] El sistema solicita la contraseña provisional y la contraseña nueva, por duplicado.	
[3] El usuario introduce las contraseñas.	
[4] El sistema comprueba que la contraseña provisional es correcta, que las dos nuevas coinciden, que la nueva es distinta de la provisional y que tiene al menos 8 caracteres, una mayúscula, un número y un carácter especial.	
[5] El sistema guarda la contraseña nueva, marca el alta del miembro como completada y le da acceso a sus funcionalidades de gestión.	
Secuencia Alternativa	
[3.a.1] El usuario decide cerrar la sesión: el sistema la cierra sin cambiar la contraseña, y la próxima vez que inicie sesión se le volverá a pedir.	
[4.a.1] Contraseña provisional incorrecta: el sistema lo informa y no cambia la contraseña.	
[4.a.2] Las contraseñas nuevas no coinciden, la nueva es igual a la provisional o no cumple las reglas: el sistema indica qué debe corregir y no cambia la contraseña.	
Requisitos Relacionados	RF-26.

Tabla A.16: Caso de uso: Cambiar contraseña provisional

Caso de uso: Solicitar reactivación de cuenta	
Resumen	Un miembro de la Junta de Cofradías cuya cuenta ha sido desactivada pide al administrador que la reactive.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión y su cuenta está desactivada.
Postcondición	Queda registrada una solicitud de reactivación pendiente, que el administrador puede aceptar o rechazar.
Secuencia Base	
[1] El sistema informa al usuario de que su cuenta está desactivada y de que no puede realizar cambios.	
[2] El usuario solicita la reactivación de su cuenta.	
[3] El sistema registra la solicitud de reactivación como pendiente.	
[4] El sistema informa al usuario de que se ha avisado al administrador y de que debe esperar a que la revise.	
Secuencia Alternativa	
[2.a.1] El usuario no la solicita y cierra la sesión: el caso de uso termina sin registrar ninguna solicitud.	
[3.a.1] La cuenta ya ha sido reactivada entretanto: el sistema informa de que la cuenta ya está activa.	
Requisitos Relacionados	RF-41.

Tabla A.17: Caso de uso: Solicitar reactivación de cuenta

Caso de uso: Gestionar su propio perfil	
Resumen	El usuario consulta y modifica los datos de su propio perfil y, si lo desea, cambia su contraseña.
Actor	Miembro de la Junta de Cofradías, Administrador
Precondición	El usuario ha iniciado sesión. Si es miembro de la Junta de Cofradías, su cuenta está activa y ya ha sustituido su contraseña provisional.
Postcondición	Los datos del perfil quedan actualizados.
Secuencia Base	
[1] El usuario solicita consultar su perfil.	
[2] El sistema muestra su nombre, su correo electrónico y su teléfono.	
[3] El usuario modifica su nombre o su teléfono y, opcionalmente, introduce una contraseña nueva.	
[4] El sistema solicita la contraseña actual para confirmar los cambios.	
[5] El usuario introduce su contraseña actual y confirma.	
[6] El sistema comprueba la contraseña actual y guarda los cambios.	
Secuencia Alternativa	
[3.a.1] El usuario intenta cambiar su correo electrónico: el sistema no lo permite, ya que es su identificador para iniciar sesión.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Contraseña actual incorrecta: el sistema lo informa y no guarda los cambios.	
[6.a.2] La contraseña nueva no tiene al menos 8 caracteres, una mayúscula, un número y un carácter especial: el sistema indica qué debe corregir y no guarda los cambios.	
Requisitos Relacionados	RF-27 y RF-40.

Tabla A.18: Caso de uso: Gestionar su propio perfil

Caso de uso: Editar información de la ciudad	
Resumen	El miembro de la Junta de Cofradías actualiza la información general sobre la Semana Santa de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	La historia y el patrimonio de la ciudad quedan actualizados.
Secuencia Base	
[1] El usuario solicita editar la información de su ciudad.	
[2] El sistema muestra la historia y el patrimonio actuales de la ciudad.	
[3] El usuario modifica la información y confirma.	
[4] El sistema guarda la información de la ciudad.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
Requisitos Relacionados	RF-29.

Tabla A.19: Caso de uso: Editar información de la ciudad

Caso de uso: Gestionar cofradía	
Resumen	El miembro de la Junta de Cofradías da de alta, edita o da de baja las cofradías de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	Las cofradías de la ciudad quedan actualizadas.
Secuencia Base	
[1] El usuario solicita gestionar las cofradías de su ciudad.	
[2] El sistema muestra las cofradías de la ciudad.	
[3] El usuario indica si quiere dar de alta una cofradía nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la cofradía: nombre, historia, web oficial, imagen y si está activa (visible para los ciudadanos) o todavía en preparación.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la cofradía en la ciudad del usuario.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la cofradía.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la cofradía.	
Requisitos Relacionados	RF-28 y RF-29.

Tabla A.20: Caso de uso: Gestionar cofradía

Caso de uso: Gestionar pasos	
Resumen	El miembro de la Junta de Cofradías da de alta, edita o da de baja los pasos de las cofradías de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Los pasos de la cofradía quedan actualizados.
Secuencia Base	
[1] El usuario solicita gestionar los pasos.	
[2] El sistema muestra los pasos de las cofradías de la ciudad.	
[3] El usuario indica si quiere dar de alta un paso nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del paso: nombre, cofradía a la que pertenece, historia, análisis artístico e imagen.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el paso.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina el paso.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el paso.	
Requisitos Relacionados	RF-30.

Tabla A.21: Caso de uso: Gestionar pasos

Caso de uso: Gestionar eventos	
Resumen	El miembro de la Junta de Cofradías da de alta, edita, cambia de estado o da de baja los eventos de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Los eventos de la ciudad quedan actualizados.
Secuencia Base	
[1] El usuario solicita gestionar los eventos de su ciudad.	
[2] El sistema muestra los eventos de la ciudad con su estado.	
[3] El usuario indica si quiere dar de alta un evento nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del evento: nombre, cofradía organizadora, día de la Semana Santa, hora de inicio, duración, ubicación (latitud, longitud y, opcionalmente, dirección), historia y web oficial; y, al editar, también su estado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el evento; al darlo de alta, queda en estado programado.	
[7] El sistema confirma los cambios y ofrece notificarlos a los ciudadanos.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina el evento.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[5.a.2] El usuario quiere elegir qué pasos participan en el evento: se realiza el caso de uso «Asignar pasos».	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el evento.	
[6.a.2] El cambio de estado contradice el horario (volver a programado una vez pasada la hora de inicio, o a en curso una vez pasada la de fin): el sistema no lo guarda e indica que se modifique la hora en lugar del estado.	
[6.a.3] El nuevo estado es en curso o finalizado: el sistema genera y envía automáticamente la notificación correspondiente a los ciudadanos de la ciudad.	
[7.a.1] El usuario quiere notificar los cambios, o ha cambiado el estado a cancelado: se realiza el caso de uso «Crear notificación».	
Requisitos Relacionados	RF-28, RF-29, RF-34 y RF-35.

Tabla A.22: Caso de uso: Gestionar eventos

Caso de uso: Gestionar procesión	
Resumen	El miembro de la Junta de Cofradías da de alta, edita, cambia de estado o da de baja las procesiones de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Las procesiones de la ciudad quedan actualizadas.
Secuencia Base	
[1] El usuario solicita gestionar las procesiones de su ciudad.	
[2] El sistema muestra las procesiones de la ciudad con su estado.	
[3] El usuario indica si quiere dar de alta una procesión nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la procesión: nombre, cofradía, día de la Semana Santa, hora de salida, duración, web oficial, historia y tradición; y, al editar, también su estado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la procesión; al darla de alta, queda en estado programado.	
[7] El sistema confirma los cambios y ofrece notificarlos a los ciudadanos.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la procesión junto con su recorrido.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[5.a.2] El usuario quiere definir el recorrido de la procesión: se realiza el caso de uso «Importar recorrido (GPX)».	
[5.a.3] El usuario quiere elegir qué pasos desfilan en la procesión: se realiza el caso de uso «Asignar pasos».	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la procesión.	
[6.a.2] El cambio de estado contradice el horario (volver a programada una vez pasada la hora de inicio, o a en curso una vez pasada la de fin): el sistema no lo guarda e indica que se modifique la hora en lugar del estado.	
[6.a.3] El nuevo estado es en curso o finalizada: el sistema genera y envía automáticamente la notificación correspondiente a los ciudadanos de la ciudad.	
[7.a.1] El usuario quiere notificar los cambios, o ha cambiado el estado a cancelada: se realiza el caso de uso «Crear notificación».	
Requisitos Relacionados	RF-28, RF-29, RF-34 y RF-35.

Tabla A.23: Caso de uso: Gestionar procesión

Caso de uso: Asignar pasos	
Resumen	El miembro de la Junta de Cofradías elige qué pasos de la cofradía participan en una procesión o un evento.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario está gestionando una procesión o un evento ya dado de alta (casos de uso «Gestionar procesión» y «Gestionar eventos»).
Postcondición	Quedan actualizados los pasos que participan en la procesión o el evento.
Secuencia Base	
[1] El usuario solicita elegir los pasos que participan.	
[2] El sistema muestra los pasos de la cofradía, indicando cuáles participan ya.	
[3] El usuario marca los pasos que deben participar y desmarca los que no.	
[4] El usuario confirma la selección.	
[5] El sistema guarda los pasos que participan en la procesión o el evento.	
Secuencia Alternativa	
[2.a.1] La cofradía no tiene pasos: el sistema informa de que antes hay que darlos de alta (caso de uso «Gestionar pasos»).	
[4.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
Requisitos Relacionados	RF-29 y RF-30.

Tabla A.24: Caso de uso: Asignar pasos

Caso de uso: Importar recorrido (GPX)	
Resumen	El miembro de la Junta de Cofradías define el recorrido de una procesión a partir de un archivo GPX preparado con una herramienta externa de trazado de rutas.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario está gestionando una procesión (caso de uso «Gestionar procesión»).
Postcondición	La procesión queda asociada a un recorrido formado por la secuencia ordenada de puntos del archivo.
Secuencia Base	
[1] El usuario solicita importar el recorrido de la procesión.	
[2] El sistema solicita el archivo GPX.	
[3] El usuario elige el archivo desde su dispositivo.	
[4] El sistema lee del archivo la secuencia ordenada de coordenadas del trazado, sin ningún nombre ni información adicional, tomando primero las del recorrido grabado (*track*), si no las de la ruta planificada (*route*) y, como último recurso, los puntos sueltos (*waypoints*).	
[5] El sistema crea el recorrido con esos puntos, calcula su distancia total y lo asocia a la procesión.	
[6] El sistema informa de que el recorrido se ha importado e indica su distancia.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] El archivo no es un GPX válido: el sistema informa del error y no importa ningún recorrido.	
[4.a.2] El archivo tiene menos de dos puntos: el sistema informa de que no hay puntos suficientes para formar un recorrido.	
[4.a.3] El archivo tiene más de 300 puntos: el sistema conserva 300 repartidos de forma uniforme, incluidos siempre el primero y el último.	
[6.a.1] El usuario quiere destacar lugares del recorrido: se realiza el caso de uso «Marcar puntos de interés».	
Requisitos Relacionados	RF-32.

Tabla A.25: Caso de uso: Importar recorrido (GPX)

Caso de uso: Marcar puntos de interés	
Resumen	El miembro de la Junta de Cofradías destaca lugares concretos del recorrido de una procesión, como un encuentro, la entrada en una iglesia o una parada para una oración.
Actor	Miembro de la Junta de Cofradías
Precondición	La procesión tiene un recorrido importado (caso de uso «Importar recorrido (GPX)»).
Postcondición	Los puntos elegidos quedan guardados como puntos de interés del recorrido, con su información.
Secuencia Base	
[1] El usuario solicita revisar el recorrido de la procesión.	
[2] El sistema muestra los puntos del recorrido en orden, indicando cuáles son ya puntos de interés.	
[3] El usuario elige un punto del recorrido.	
[4] El sistema solicita el tipo de punto de interés (monumento, iglesia, encuentro, oración, entrada de procesión o salida de procesión), su nombre y, opcionalmente, una descripción y una imagen.	
[5] El usuario introduce los datos y confirma.	
[6] El sistema guarda el punto como punto de interés del recorrido.	
Secuencia Alternativa	
[3.a.1] El punto elegido ya es un punto de interés: el sistema muestra su información para que el usuario la modifique.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Faltan el tipo o el nombre: el sistema indica que son obligatorios y no guarda el punto.	
Requisitos Relacionados	RF-33.

Tabla A.26: Caso de uso: Marcar puntos de interés

Caso de uso: Gestionar códigos de acceso	
Resumen	El miembro de la Junta de Cofradías emite y revoca los códigos de acceso con los que los cofrades de una cofradía acceden al modo cofrade.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	Los códigos de acceso de la cofradía quedan actualizados.
Secuencia Base	
[1] El usuario elige una cofradía de su ciudad y solicita gestionar sus códigos de acceso.	
[2] El sistema muestra los códigos emitidos para la cofradía, con su estado (emitido, ya usado o revocado).	
[3] El usuario solicita emitir un código nuevo.	
[4] El sistema genera automáticamente un código único y lo asocia a la cofradía.	
[5] El sistema muestra el código para que el usuario lo comparta con los cofrades.	
Secuencia Alternativa	
[3.a.1] El usuario solicita revocar un código: el sistema pide confirmación y, si el usuario confirma, lo revoca; los cofrades que lo estén usando dejan de poder compartir su ubicación.	
[3.a.2] El usuario no quiere hacer ningún cambio: el caso de uso termina.	
Requisitos Relacionados	RF-31.

Tabla A.27: Caso de uso: Gestionar códigos de acceso

Caso de uso: Gestionar una Junta de Cofradías	
Resumen	El administrador da de alta, edita o da de baja las Juntas de Cofradías del sistema.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. Existe al menos una ciudad.
Postcondición	Las Juntas de Cofradías quedan actualizadas en el sistema.
Secuencia Base	
[1] El usuario solicita gestionar las Juntas de Cofradías.	
[2] El sistema muestra las Juntas registradas, con la ciudad que gestiona cada una.	
[3] El usuario indica si quiere dar de alta una Junta nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la Junta: nombre, correo electrónico, teléfono, ciudad que gestiona y si está activa.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la Junta.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la Junta.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] La ciudad elegida ya tiene una Junta de Cofradías: el sistema informa de que cada ciudad solo puede tener una y no guarda la Junta.	
[6.a.2] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la Junta.	
[6.a.3] La Junta se desactiva: sus miembros dejan de poder realizar cambios hasta que se vuelva a activar.	
Requisitos Relacionados	RF-38.

Tabla A.28: Caso de uso: Gestionar una Junta de Cofradías

Caso de uso: Gestionar a los miembros de las Juntas de Cofradías	
Resumen	El administrador da de alta, edita, desactiva o da de baja a los miembros de las Juntas de Cofradías.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. Existe al menos una Junta de Cofradías.
Postcondición	Los miembros de las Juntas quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario elige una Junta de Cofradías y solicita gestionar sus miembros.	
[2] El sistema muestra los miembros de la Junta con su situación: activo, desactivado o con la invitación pendiente (todavía no ha cambiado su contraseña provisional).	
[3] El usuario indica si quiere dar de alta un miembro nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del miembro: nombre, correo electrónico y teléfono; y, al editar, también si está activo o desactivado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el miembro.	
[7] Al dar de alta al miembro, el sistema genera una contraseña provisional y se la envía por correo electrónico junto a un mensaje de bienvenida.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina al miembro.	
[3.a.2] El miembro tiene la invitación pendiente y no encuentra el correo: se realiza el caso de uso «Reenviar invitación».	
[4.a.1] Al editar, el correo electrónico no se puede modificar, ya que es el identificador con el que el miembro inicia sesión.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] El correo electrónico ya pertenece a otra cuenta: el sistema lo informa y no guarda el miembro.	
[6.a.2] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el miembro.	
[7.a.1] Error al enviar el correo: el sistema informa del error y no da de alta al miembro, para no dejar una cuenta cuya contraseña nadie ha recibido.	
Requisitos Relacionados	RF-39.

Tabla A.29: Caso de uso: Gestionar a los miembros de las Juntas de Cofradías

Caso de uso: Reenviar invitación	
Resumen	El administrador vuelve a enviar a un miembro de la Junta de Cofradías el correo con una contraseña provisional, porque no lo recibió o lo ha perdido.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. El miembro tiene la invitación pendiente: todavía no ha sustituido su contraseña provisional.
Postcondición	El miembro recibe por correo una contraseña provisional nueva, y la anterior deja de ser válida.
Secuencia Base	
[1] El usuario solicita reenviar la invitación a un miembro con la invitación pendiente.	
[2] El sistema genera una contraseña provisional nueva, que invalida la anterior.	
[3] El sistema envía de nuevo el correo de bienvenida con la contraseña provisional nueva.	
[4] El sistema informa de que la invitación se ha reenviado.	
Secuencia Alternativa	
[3.a.1] Error al enviar el correo: el sistema informa del error y mantiene la contraseña provisional anterior.	
Requisitos Relacionados	RF-39.

Tabla A.30: Caso de uso: Reenviar invitación

Caso de uso: Gestionar solicitudes de reactivación	
Resumen	El administrador revisa las solicitudes de los miembros desactivados que piden reactivar su cuenta, y las acepta o las rechaza.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador.
Postcondición	Las solicitudes revisadas dejan de estar pendientes; las aceptadas reactivan la cuenta del miembro.
Secuencia Base	
[1] El usuario solicita consultar las solicitudes de reactivación.	
[2] El sistema muestra las solicitudes pendientes, con el nombre, el correo electrónico y la Junta de cada miembro.	
[3] El usuario elige una solicitud y decide aceptarla.	
[4] El sistema reactiva la cuenta del miembro y marca la solicitud como resuelta.	
[5] El sistema actualiza las solicitudes pendientes.	
Secuencia Alternativa	
[2.a.1] No hay solicitudes pendientes: el sistema informa de ello.	
[3.a.1] El usuario decide rechazar la solicitud: el sistema la marca como resuelta y la cuenta sigue desactivada.	
Requisitos Relacionados	RF-41.

Tabla A.31: Caso de uso: Gestionar solicitudes de reactivación

Apéndice B

Esquema de la base de datos

Este apéndice recoge, a nivel de detalle de campo, el esquema completo de las tablas de PostgreSQL [39] introducidas en el apartado Diseño de la base de datos (Sección 4.1), incluyendo los tipos de dato de cada campo y las claves primarias y foráneas. Los tipos de dato mostrados corresponden a tipos nativos de PostgreSQL (`BIGINT`, `VARCHAR`, `TEXT`, `BOOLEAN`, `TIMESTAMP`, `DOUBLE PRECISION`) o a tipos enumerados propios (`ENUM`) definidos para este esquema.

B.1. Ciudad (`ciudades`)

Requisito: RI-01

Agrupa las procesiones y actos de cada localidad.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`nombre`	`VARCHAR`	
`comunidad_autonoma`	`VARCHAR`	
`descripcion`	`TEXT`	
`activa`	`BOOLEAN`	visible para el ciudadano o no
`latitud`	`DOUBLE PRECISION`	opcional, para preseleccionar la ciudad más cercana por GPS (RF-04)
`longitud`	`DOUBLE PRECISION`	opcional, para preseleccionar la ciudad más cercana por GPS (RF-04)

Tabla B.1: Campos de `ciudades`

B.2. Junta de Cofradías (`juntas_cofradias`)

Requisito: RI-02

Junta que gestiona una ciudad. Relación 1:1 con ciudades.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
email	VARCHAR	
telefono	VARCHAR	
ciudad_id	BIGINT	FK → ciudades(id), UNIQUE

Tabla B.2: Campos de juntas_cofradias

B.3. Cofradía / Hermandad (cofradias)

Requisito: RI-04

Asociación religiosa que organiza eventos y procesiones. Pertenece a la ciudad –nunca a la Junta de Cofradías que la gestiona–: Ciudad es quien agrupa estructuralmente a sus cofradías (relación 1:N), mientras que JuntaCofradias es solo la cuenta de gestión de esa ciudad, sustituible sin afectar a los datos de las cofradías que agrupa.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
web	VARCHAR	URL del sitio propio de la cofradía
fecha_creacion	TIMESTAMP	
activa	BOOLEAN	visible para el ciudadano, o todavía en preparación
ciudad_id	BIGINT	FK → ciudades(id)

Tabla B.3: Campos de cofradias

B.4. Paso (pasos)

Requisito: RI-10 Pasos que posee la cofradía. A diferencia del planteamiento inicial, en el que la relación N:M era solo con procesiones, "los eventos también pueden tener pasos": la relación se generalizó a eventos (Sección B.7), del que procesiones hereda, resuelta mediante la tabla intermedia pasos_eventos.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
analisis_artistico	TEXT	
imagen	VARCHAR	URL de texto libre, pegada a mano por la Junta (sin infraestructura de subida propia; ver trabajo futuro)
cofradia_id	BIGINT	FK → cofradias(id)

Tabla B.4: Campos de pasos

B.4.1. Paso-Evento (pasos_eventos) — tabla intermedia N:M

Campo	Tipo	Nota
paso_id	BIGINT	PK compuesta, FK → pasos(id)
evento_id	BIGINT	PK compuesta, FK → eventos(id)

Tabla B.5: Campos de pasos_eventos

B.5. Código de Acceso (codigos_acceso)

Requisito: RI-13 Códigos que emite la Junta de Cofradías en nombre de una cofradía de su ciudad (Cofradia no es un actor con cuenta propia, no puede emitir nada por sí misma). Quien lo valida obtiene un token identificado con esa cofradía y el rol de cofrade, sin que ello genere ningún registro de usuario –ver la explicación completa en la Sección B.10–. El código no se invalida al usarse por primera vez: sigue sirviendo para volver a autenticarse mientras no esté revocado.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
codigo	VARCHAR	UNIQUE, generado por el servidor
estado	ENUM	EMITIDO, VALIDADO, REVOCADO
cofradia_id	BIGINT	FK → cofradias(id)

Tabla B.6: Campos de codigos_acceso

B.6. Ubicación (ubicaciones)

Punto geográfico reutilizable (latitud, longitud y dirección textual), referenciado por eventos y por puntos_ruta. Se modela como tabla independiente, en lugar de como columnas repetidas en cada entidad que necesita una posición, para poder compartir la misma ubicación entre varias entidades –por ejemplo, la sede de una cofradía y un punto de interés del recorrido que coincidan en el mismo templo– sin duplicar datos.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
latitud	DOUBLE PRECISION	
longitud	DOUBLE PRECISION	
direccion	VARCHAR	

Tabla B.7: Campos de ubicaciones

B.7. Evento (eventos)

Requisito: RI-05

Acto que puede tener más de una cofradía participando (por ejemplo, un Vía Crucis conjunto o una procesión con pasos de varias hermandades): la relación con cofradias es N:M, no 1:N, resuelta mediante la tabla intermedia eventos_cofradias. Evento es además la tabla raíz de la que procesiones hereda sus columnas comunes (Sección B.8).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
tradicion	TEXT	
fecha	TIMESTAMP	
estado	ENUM	PROGRAMADO, EN_CURSO, FINALIZADO, CANCELADO
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla B.8: Campos de eventos

B.7.1. Evento-Cofradía (eventos_cofradias) — tabla intermedia N:M

Campo	Tipo	Nota
evento_id	BIGINT	PK compuesta, FK → eventos(id)
cofradia_id	BIGINT	PK compuesta, FK → cofradias(id)

Tabla B.9: Campos de eventos_cofradias

B.8. Procesión (procesiones)

Requisito: RI-09, RI-06

A diferencia del planteamiento inicial, en el que procesiones referenciaba a eventos mediante una clave foránea 1:N, la traducción final a PostgreSQL usa **herencia real de tablas** (estrategia JOINED de JPA): Procesion *es* un Evento –hereda nombre, historia, tradicion, fecha, estado, ubicacion_id y sus

cofradías participantes (`eventos_cofradias`)– y añade únicamente sus columnas propias en esta tabla. El `id` de `procesiones` es a la vez su clave primaria y una clave foránea a `eventos(id)`: al guardar una procesión, el motor inserta primero la fila común en `eventos` y reutiliza el identificador generado para la fila de `procesiones`, en lugar de generar uno nuevo. Esta decisión evita mantener columnas duplicadas (`cofradia_id`, `estado`...) sincronizadas manualmente entre las dos tablas.

Campo	Tipo	Nota
`id`	`BIGINT`	PK y FK → `eventos(id)`
`fecha_inicio`	`TIMESTAMP`	
`fecha_fin`	`TIMESTAMP`	
`recorrido_id`	`BIGINT`	FK → `recorridos(id)`, `UNIQUE` (1:1), opcional –puede programarse una procesión antes de tener ruta definida–

Tabla B.10: Campos de `procesiones`

B.8.1. Posiciones actuales (`posiciones_actuales`) — histórico de pings anónimos

A diferencia del planteamiento inicial, esta tabla no guarda una única fila por procesión que se sobrescribe con cada actualización (`UPSERT`), sino un **histórico de lecturas GPS sueltas** (*pings*): cada vez que un cofrade comparte su ubicación se inserta una fila nueva, sin sobrescribir ninguna anterior. Cada fila es además **anónima**: no lleva ninguna referencia a la persona que la envió, solo a la procesión (Sección B.10 explica por qué). La "posición actual" de una procesión ya no es un dato guardado como tal, sino un valor **calculado** en el momento de la consulta: el promedio de latitud y longitud de los *pings* recibidos en los últimos 60 segundos, siendo el número de *pings* recientes una aproximación al número de cofrades compartiendo ubicación en ese instante. La aplicación cliente consulta este valor calculado mediante la API propia (`GET /procesiones/{id}/ubicacion`) con la cadencia que exige el requisito RNF-05, tal y como se explica en el apartado Decisiones tecnológicas.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`latitud`	`DOUBLE PRECISION`	
`longitud`	`DOUBLE PRECISION`	
`timestamp`	`TIMESTAMP`	instante de la lectura
`procesion_id`	`BIGINT`	FK → `procesiones(id)`

Tabla B.11: Campos de `posiciones_actuales`

B.9. Recorrido (`recorridos`)

Requisito: RI-07, RI-11

Itinerario compuesto por puntos de ruta ordenados. Sin campo `nombre`: un recorrido se identifica por la procesión a la que pertenece, no por un nombre propio.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
distancia_total	DOUBLE PRECISION	metros
tiempo_estimado	INTEGER	minutos

Tabla B.12: Campos de recorridos

B.9.1. Punto de Ruta (puntos_ruta)

A diferencia del planteamiento inicial, un mismo punto de ruta puede formar parte de **varios** recorridos –varias procesiones pueden compartir un mismo tramo de calle o pasar por el mismo templo–, así que puntos_ruta ya no referencia directamente a un recorrido: la relación es N:M y se resuelve en la tabla intermedia recorrido_puntos_ruta (Sección B.9.3). PuntoRuta es además la tabla raíz de una segunda jerarquía de herencia real de tablas: un punto "de paso" simple se guarda directamente aquí, mientras que un punto "de interés" con detalle añade sus columnas propias en puntos_de_interes (Sección B.9.2).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla B.13: Campos de puntos_ruta

B.9.2. Punto de Interés (puntos_de_interes)

Subtipo de PuntoRuta con detalle: nombre, descripción, imagen y una categoría (RI-11). El tipo enumerado se llama TipoPuntoInteres –es una categoría de punto de interés, no de cualquier punto de ruta– e incluye tanto categorías de interés turístico o religioso como puntos operativos del propio recorrido de una procesión (inicio y fin de la salida procesional, o el lugar donde se celebra el evento asociado).

Campo	Tipo	Nota
id	BIGINT	PK y FK → puntos_ruta(id)
tipo	ENUM	MONUMENTO, IGLESIA, ENCUENTRO, ORACCION, ENTRADAPROCESION, SALIDAPROCESION, UBICACIONEVENTO
nombre	VARCHAR	
descripcion	TEXT	
imagen	VARCHAR	URL de texto libre, pegada a mano por la Junta (sin infraestructura de subida propia; ver trabajo futuro)

Tabla B.14: Campos de puntos_de_interes

B.9.3. Recorrido-Punto de Ruta (`recorrido_puntos_ruta`) — tabla intermedia N:M

El orden de un punto dentro de un recorrido y la hora prevista de paso ya no son propiedades del punto en sí (un mismo punto puede estar en la posición 3 de un recorrido y en la posición 7 de otro, a horas distintas), sino de **esta relación**. Lleva un identificador propio en lugar de una clave primaria compuesta –a diferencia de `pasos_eventos` o `eventos_cofradias`– porque, además de las dos claves foráneas, tiene columnas propias; dos restricciones `UNIQUE` garantizan que un punto no se repita dentro del mismo recorrido y que dos puntos no compartan posición.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`recorrido_id`	`BIGINT`	FK → `recorridos(id)`
`punto_ruta_id`	`BIGINT`	FK → `puntos_ruta(id)`
`orden`	`INTEGER`	`UNIQUE` junto con `recorrido_id`
`hora_prevista`	`TIMESTAMP`	hora prevista de paso

Tabla B.15: Campos de `recorrido_puntos_ruta`

B.10. Usuario (`usuarios`)

Requisito: RI-03

A diferencia del planteamiento inicial, en el que `usuarios` era una única tabla con una columna `rol` que distinguía entre cofrade, junta de cofradías y administrador, la traducción final a PostgreSQL usa de nuevo **herencia real de tablas**: `usuarios` es la tabla raíz, con las columnas que de verdad comparten sus subtipos, y cada subtipo añade sus propias columnas en su propia tabla. Sin columna `rol`: con herencia real el tipo de usuario lo determina en qué tabla de subtipo tiene fila, no un valor guardado.

El cambio más relevante respecto al planteamiento inicial es que **el cofrade ha dejado de ser un** `Usuario`: ya no se guarda ninguna fila –ni en `usuarios` ni en ninguna tabla propia– para las personas que comparten ubicación durante una procesión. Validar un código de acceso (Sección B, `codigos_acceso`) no registra a nadie: el servidor comprueba que el código no está revocado y devuelve directamente un token JWT identificado con la **cofradía** cuyo código se validó, no con una persona. Esta decisión responde a que la aplicación no necesita conocer la identidad de cada cofrade para cumplir su función –agregar y mostrar la posición de una procesión–, y evita cargar el sistema con el registro de personas cuya participación es, además, estacional (solo durante los días de procesión). `Cofrade` sigue existiendo como clase en el modelo de dominio (Figura 3.5) y en el código del backend, pero como una representación construida en memoria a partir del token en cada petición, nunca persistida.

Por tanto, `usuarios` solo contiene los perfiles con credenciales propias –junta de cofradías y administrador–, que sí se autentican con `email` y una contraseña almacenada como *hash*[1] mediante BCrypt, el algoritmo utilizado por defecto por Spring Security [42]. El ciudadano tampoco se registra, igual que en el plantea-

[1]Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

miento inicial.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
email	VARCHAR	UNIQUE
password_hash	VARCHAR	*hash* BCrypt
fecha_ingreso	TIMESTAMP	

Tabla B.16: Campos de usuarios

B.10.1. Miembro de Junta de Cofradías (miembros_junta_cofradia)

Persona con credenciales que pertenece a la Junta de Cofradías de una ciudad –una Junta puede tener varios miembros, cada uno con su propio inicio de sesión–. La referencia es a la JuntaCofradias, no directamente a la Ciudad: así se reutiliza la relación 1:1 ya existente entre JuntaCofradias y Ciudad en lugar de duplicarla, y de ahí –miembro.juntaCofradias.ciudad– es de donde la capa de servicio de la API obtiene qué ciudad gestiona cada miembro a la hora de comprobar la autorización por rol (RNF-03). activo lo alterna el Administrador: desactivado no significa sin acceso –el *login* le sigue funcionando–, sino que el backend le rechaza cualquier escritura. password_provisional vale TRUE hasta que el miembro cambia la contraseña que se le generó y envió por correo al darlo de alta, como señal de que la invitación sigue pendiente. solicitud_reactivacion_pendiente la activa un miembro desactivado que pide volver a estar activo, y la resuelve el Administrador aceptándola o rechazándola.

Campo	Tipo	Nota
id	BIGINT	PK y FK → usuarios(id)
nombre	VARCHAR	
telefono	VARCHAR	opcional
junta_cofradias_id	BIGINT	FK → juntas_cofradias(id)
activo	BOOLEAN	
password_provisional	BOOLEAN	
solicitud_reactivacion_pendiente	BOOLEAN	

Tabla B.17: Campos de miembros_junta_cofradia

B.10.2. Administrador (administradores)

Rol global del sistema, sin referencia a Ciudad: cualquier administrador puede gestionar cualquier ciudad, no tiene sentido atarlo de antemano a una que todavía no exista. nombre y telefono son opcionales –no se piden al crear la cuenta, solo se completan cuando el propio administrador edita su perfil–.

Campo	Nota
id	PK y FK → usuarios(id)
nombre	opcional
telefono	opcional

Tabla B.18: Campos de administradores

B.11. Notificación (`notificaciones`)

Requisito: RI-08

A diferencia del planteamiento inicial, no hay una jerarquía Aviso/Alerta: una única clase cubre los dos casos, con un tipo y una prioridad en vez de una distinción de subtipos –al revisar qué se usaba de verdad se vio que el cliente pinta cada notificación solo a partir de su prioridad, nunca de un subtipo más fino, así que esa jerarquía no se estaba ganando su sitio–. Pertenece a una `Ciudad`: es el ámbito con el que el ciudadano filtra qué notificaciones ver, y con el que la Junta que las crea comprueba su autorización.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`titulo`	`VARCHAR`	
`mensaje`	`TEXT`	opcional
`fecha_creacion`	`TIMESTAMP`	
`ciudad_id`	`BIGINT`	FK → `ciudades(id)`
`tipo`	`ENUM`	`INICIO`, `FIN` (los genera el propio sistema al cambiar el estado de un `Evento`), `INCIDENCIA`, `CAMBIO_HORARIO`, `CANCELACION` (los crea la Junta a mano)
`prioridad`	`ENUM`	`BAJA`, `MEDIA`, `ALTA`; obligatoria salvo en `INICIO/FIN`
`fecha_expiracion`	`TIMESTAMP`	opcional, para cualquier tipo

Tabla B.19: Campos de `notificaciones`

Sin tabla de notificación entregada: no se registra si un ciudadano concreto ha leído cada notificación o no –no tiene cuenta con la que asociar esa lectura (Sección B.10)–, simplemente se consulta la lista de notificaciones vigentes de su ciudad.

B.12. Dispositivo Push (`dispositivos_push`)

Requisito: RI-12

Dispositivo del ciudadano registrado para recibir notificaciones *push* de una ciudad. Sin usuario al que pertenecer –el ciudadano nunca se autentica (Sección B.10)–: la única pertenencia real es a qué ciudad le interesa a ese token, que puede cambiar si el ciudadano cambia de ciudad seleccionada (el mismo token se reregistra, actualizando `ciudad_id` en vez de crear una fila nueva). El `token` es de Expo Push (`ExponentPushToken[...]`), no un token FCM/APNs en crudo: el backend solo habla con la API de Expo Push (`exp.host`), nunca directamente con Firebase Cloud Messaging ni con APNs.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
token	VARCHAR	UNIQUE, token de Expo Push
ciudad_id	BIGINT	FK → ciudades(id)
fecha_registro	TIMESTAMP	
fecha_actualizado	TIMESTAMP	

Tabla B.20: Campos de dispositivos_push

B.13. Relaciones entre tablas

La Tabla B.21 recoge las relaciones entre tablas de este esquema, todas ellas garantizadas por claves foráneas con restricción de integridad referencial.

De	A	Relación	Cardinalidad
ciudades	cofradias	agrupa	1:N
ciudades	juntas_cofradias	gestionada por	1:1
juntas_cofradias	miembros_junta_cofradia	tiene	1:N
cofradias	eventos	participa en	N:M
cofradias	pasos	posee	1:N
cofradias	codigos_acceso	emite	1:N
eventos	procesiones	es extendido por (herencia)	1:1
eventos	ubicaciones	se celebra en	N:1
procesiones	recorridos	tiene	1:1
procesiones	posiciones_actuales	recibe	1:N
recorridos	puntos_ruta	compuesto por	N:M
puntos_ruta	puntos_de_interes	es extendido por (herencia)	1:1
puntos_ruta	ubicaciones	está en	N:1
pasos	eventos	participa en	N:M
miembros_junta_cofradia	usuarios	es extendido por (herencia)	1:1
administradores	usuarios	es extendido por (herencia)	1:1
ciudades	notificaciones	recibe	1:N
ciudades	dispositivos_push	recibe registros de	1:N

Tabla B.21: Relaciones entre tablas de PostgreSQL

Apéndice C

Manual de despliegue

Este apéndice describe los pasos necesarios para desplegar el backend propio (API Spring Boot y base de datos PostgreSQL) en el servidor virtual proporcionado por la Escuela de Ingeniería Informática de la Universidad de Valladolid, así como la configuración necesaria en el cliente React Native para que la aplicación completa funcione en un entorno nuevo. El proyecto no depende de ningún servicio de Firebase (Sección 4.1.2): las notificaciones *push* se envían a través de Expo Push Notifications, sin necesidad de credenciales propias.

C.1. Requisitos previos

- Acceso por SSH al servidor virtual (`virtual.lab.inf.uva.es`), con un usuario con permisos de `sudo`. El servidor es una máquina virtual Linux cuyos puertos de entrada asignados están en el rango 20101–20109: el 20101 se usa para SSH y el 20102 para la API.
- Java 17 o superior, Git y PostgreSQL instalados en el servidor.
- Repositorio del proyecto en GitHub [20], desde el que se descarga el código en el servidor.
- Cuenta de Expo [30] con un proyecto EAS creado, para generar el ejecutable de la aplicación móvil y para que `expo-notifications` pueda emitir tokens de Expo Push válidos.
- Clave de API de Google Maps Platform [1] para el SDK de Maps en Android, restringida al paquete de la aplicación y a la huella SHA-1 del certificado con el que EAS firma el ejecutable. En iOS los mapas usan Apple Maps y no necesitan clave.
- Java 17 o superior y Node.js instalados en el entorno de desarrollo local.

C.2. Despliegue del backend en el servidor

1. Conectarse al servidor por SSH:

```
ssh -p 20101 usuario@virtual.lab.inf.uva.es
```

2. Crear en PostgreSQL la base de datos y el usuario que utilizará la API. No es necesario crear ninguna tabla: al arrancar, Spring Boot ejecuta automáticamente las migraciones versionadas del esquema (Apéndice B) antes de empezar a atender peticiones.

3. Clonar el repositorio y compilar el backend, lo que genera el ejecutable `.jar` de la API:

```
git clone <URL del repositorio> ~/TFG
cd ~/TFG/backend && ./mvnw package -DskipTests
```

4. Crear el fichero de configuración `/etc/semanasanta/backend.env`, legible solo por el administrador del servidor, con las variables de entorno de la API:

   - `PORT`: puerto en el que escucha la API (`20102`).
   - `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD`: conexión con la base de datos PostgreSQL creada en el paso anterior.
   - `JWT_SECRET`: clave secreta utilizada para firmar los tokens JWT (Sección 4.1). Debe ser una cadena aleatoria de suficiente longitud y no debe compartirse ni subirse al repositorio.
   - `JWT_EXPIRATION`: tiempo de validez del token, en milisegundos.
   - `ADMIN_BOOTSTRAP_SECRET`: secreto que permite crear el primer Administrador del sistema (`POST /administradores/bootstrap`) cuando todavía no existe ninguno.
   - `MAIL_USERNAME` y `MAIL_PASSWORD`: cuenta SMTP de Gmail (con una contraseña de aplicación) desde la que se envía el correo de bienvenida a los miembros de las Juntas de Cofradía.

   No hace falta configurar ninguna credencial para las notificaciones *push*: el backend envía cada notificación con una petición HTTP corriente al servicio de Expo Push (`https://exp.host/-/api/v2/push/send`), sin autenticación propia.

5. Registrar la API como servicio del sistema (`semanasanta`) mediante systemd, con `backend.env` como fichero de entorno, para que arranque sola al iniciar el servidor y se reinicie automáticamente si se detiene por un error.

6. Arrancar el servicio y comprobar su estado:

```
sudo systemctl start semanasanta
systemctl status semanasanta
sudo journalctl -u semanasanta -f
```

   Una vez arrancada, la API queda accesible en `http://virtual.lab.inf.uva.es:20102`, que es la URL base que consume la aplicación cliente. El servidor no dispone de un certificado TLS, por lo que la comunicación se realiza mediante HTTP; por este motivo el cliente permite expresamente el tráfico HTTP hacia ese dominio (véase la sección de configuración del cliente).

7. Crear el primer Administrador llamando, desde el propio servidor, al *endpoint* de *bootstrap* con el secreto configurado en `ADMIN_BOOTSTRAP_SECRET`. A partir de ahí, el resto de cuentas se crean desde la propia aplicación.

C.2.1. Actualización a una nueva versión

El despliegue de una nueva versión se realiza manualmente desde el servidor: se descarga el código más reciente del repositorio, se vuelve a compilar el backend y se reinicia el servicio. Si la nueva versión incluye migraciones del esquema, se aplican automáticamente al arrancar.

```
cd ~/TFG && git pull
cd backend && ./mvnw package -DskipTests
sudo systemctl restart semanasanta
```

C.3. Documentación de la API

Una vez desplegado el backend, la documentación interactiva de la API, generada automáticamente por springdoc-openapi [48] a partir del propio código, queda disponible en `/swagger-ui.html` sobre la URL del backend, y permite consultar y probar cada *endpoint* sin necesidad de herramientas adicionales.

C.4. Configuración del cliente React Native

1. La URL base de la API se fija con la variable de entorno `EXPO_PUBLIC_API_URL`. En los ejecutables generados con EAS la define `eas.json` en los perfiles `preview` y `production` (`http://virtual.lab.inf.uva.es`); durante el desarrollo puede fijarse en un fichero `.env.local` para trabajar contra el servidor, o dejarse sin definir para trabajar contra un backend local.

2. Al usar HTTP, el cliente debe permitir expresamente ese tráfico: en Android mediante la opción `usesCleartextTraffic` del plugin `expo-build-properties`, y en iOS mediante una excepción de App Transport Security para `virtual.lab.inf.uva.es` en `ios.infoPlist`, ambas en `app.json`.

3. Comprobar que `app.json` tiene configurado el `projectId` del proyecto EAS (campo `extra.eas.projectId`) y el plugin `expo-notifications`: son los que permiten que el dispositivo obtenga un token de Expo Push válido y lo registre en la API propia (`POST /dispositivos-push`).

4. Registrar la clave de Google Maps como variable de entorno de EAS (`GOOGLE_MAPS_API_KEY`) para los entornos `preview` y `production`. El fichero `app.config.js` la lee y la pasa al plugin de `react-native-maps`, de modo que la clave nunca se guarda en el repositorio.

C.5. Generación del ejecutable de la aplicación móvil

El paquete de instalación se genera en los servidores de Expo mediante EAS Build, sin necesidad de compilar en local. El perfil `preview` produce un `.apk` de Android que puede instalarse directamente en el dispositivo a partir del enlace que devuelve la compilación:

```
eas build --profile preview --platform android
```

El perfil `production` genera el paquete destinado a su publicación en las tiendas de aplicaciones. En el caso de iOS, instalar el ejecutable en un dispositivo físico requiere además una cuenta del Apple Developer Program.

Apéndice D

Manual de uso

Apéndice E

Contenido del TFG